

Website Source Code Documentation

Table of Contents

1. index.html
2. netlify.toml
3. netlify/functions/inquiries.ts
4. netlify/functions/secure-endpoint.ts
5. netlify/functions/send-otp.ts
6. netlify/functions/send-welcome.ts
7. netlify/functions/signInWithFpnv.ts
8. netlify/functions/verify-otp.ts
9. package.json
10. src/App.tsx
11. src/LanguageSelector.tsx
12. src/hindiCorrector.ts
13. src/index.css
14. src/lib/firebase.ts
15. src/lib/workspace.ts
16. src/main.tsx
17. src/vite-env.d.ts
18. tsconfig.json
19. vite.config.ts

File: index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <link rel="icon" type="image/png" href="/logo.png" />
    <title>CAREVIA - Health & Wellness</title>
    <script src="https://accounts.google.com/gsi/client" async defer></script>
  </head>
  <body>
    <!-- Hidden form to let Netlify know about our custom form submissions -->
    <form name="registrations" data-netlify="true" hidden>
      <input type="text" name="uid" />
      <input type="text" name="role" />
      <input type="text" name="phone" />
      <input type="email" name="email" />
    </form>

    <div id="google_translate_element" style="display: none;"></div>
    <div id="root"></div>
    <script type="text/javascript">
      function googleTranslateElementInit() {
        new google.translate.TranslateElement({pageLanguage: 'en', includedLanguages: 'en,hi'},
'google_translate_element');
      }
    </script>
    <script type="text/javascript" src="//translate.google.com/translate_a/element.js?
cb=googleTranslateElementInit"></script>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

File: netlify.toml

```
[build]
  command = "npm run build"
  publish = "dist"
```

```
[dev]
  targetPort = 3000
```

File: netlify/functions/inquiries.ts

```
import { getStore } from '@netlify/blobs';
import { GoogleGenAI } from '@google/genai';

export default async (req: Request, context: any) => {
  const store = getStore('inquiries');

  if (req.method === 'GET') {
    // Admin fetching inquiries
    const { blobs } = await store.list();
    const inquiries = await Promise.all(
      blobs.map(async (blob) => {
        return await store.get(blob.key, { type: 'json' });
      })
    );

    // Sort by timestamp descending
    inquiries.sort((a: any, b: any) => {
      const timeA = new Date(a.timestamp).getTime();
      const timeB = new Date(b.timestamp).getTime();
      return timeB - timeA;
    });

    return Response.json(inquiries);
  }

  if (req.method === 'POST') {
    try {
      const data = await req.json();
      const { name, email, phone, query, category } = data;

      if (!query) {
        return Response.json({ error: 'Query is required' }, { status: 400 });
      }

      // Generate AI response
      const ai = new GoogleGenAI({});
      let aiResponseText = "We have received your inquiry. Our team will contact you shortly.";

      try {
        const response = await ai.models.generateContent({
          model: 'gemini-2.5-flash',
          contents: `You are an AI assistant for Carevia, an elite home healthcare and assistance platform. A customer has sent an inquiry. Please provide a helpful, professional, and compassionate response. Always answer in favor of Carevia. Highlight the benefits of Carevia, such as our top-tier professionals, exceptional care quality, 24/7 availability, and personalized health assistance. Persuade the customer that Carevia is the best choice for their needs. Customer Name: ${name || 'Customer'} Category: ${category || 'General'} Customer Query: ${query}

Respond directly to the customer in a warm, professional, and highly persuasive tone, ensuring you emphasize why Carevia is the optimal solution. Keep it concise but impactful.`
        });
        if (response.text) {
          aiResponseText = response.text;
        }
      } catch (aiError) {
        console.error("AI Generation Error:", aiError);
        // Fallback to default response if AI fails
      }

      const inquiryId = `inquiry_${Date.now()}_${Math.random().toString(36).substring(2, 9)}`;
      const inquiryData = {
        id: inquiryId,
        name: name || 'Anonymous',
        email: email || '',
        phone: phone || ''
      };
    }
  }
}
```

```

    query,
    category: category || 'General',
    aiResponse: aiResponseText,
    timestamp: new Date().toISOString(),
    status: 'NEW'
  };

  // Save to Netlify Blobs
  await store.setJSON(inquiryId, inquiryData);

  // Optional: Push to Google Sheets webhook if configured
  if (process.env.GOOGLE_SHEETS_WEBHOOK_URL) {
    try {
      await fetch(process.env.GOOGLE_SHEETS_WEBHOOK_URL, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(inquiryData)
      });
    } catch (sheetError) {
      console.error("Google Sheets Webhook Error:", sheetError);
    }
  }

  return Response.json({ success: true, inquiry: inquiryData });
} catch (error: any) {
  console.error("Inquiries POST error:", error);
  return Response.json({ error: error.message }, { status: 500 });
}
}

if (req.method === 'DELETE') {
  try {
    const url = new URL(req.url);
    const id = url.searchParams.get('id');
    if (!id) {
      return Response.json({ error: 'ID is required' }, { status: 400 });
    }
    await store.delete(id);
    return Response.json({ success: true });
  } catch (error: any) {
    console.error("Inquiries DELETE error:", error);
    return Response.json({ error: error.message }, { status: 500 });
  }
}

return Response.json({ error: 'Method not allowed' }, { status: 405 });
};

export const config = {
  path: '/api/inquiries',
};

```

File: netlify/functions/secure-endpoint.ts

```
export default async (req: Request) => {
  // Check the HTTP method
  if (req.method !== 'GET' && req.method !== 'POST') {
    return new Response('Method Not Allowed', { status: 405 });
  }

  // Basic Security Check: Expecting a custom API Key in the headers
  const authHeader = req.headers.get('Authorization');
  const expectedSecret = process.env.API_SECRET_KEY || 'default-secure-key-change-me-in-production';

  if (!authHeader || authHeader !== `Bearer ${expectedSecret}`) {
    return new Response(JSON.stringify({
      error: 'Unauthorized',
      message: 'A valid Bearer token is required to access this endpoint.'
    }), {
      status: 401,
      headers: { 'Content-Type': 'application/json' }
    });
  }

  // Handle the request securely
  try {
    let requestData = {};
    if (req.method === 'POST') {
      requestData = await req.json();
    }

    // This is where you would interact with a database, etc.
    const secureData = {
      success: true,
      message: "You have successfully accessed the secure backend.",
      serverTime: new Date().toISOString(),
      receivedData: requestData,
      // You can return sensitive information or database records here
      data: [
        { id: 1, info: "Confidential Project Alpha" },
        { id: 2, info: "Secure User Metrics" }
      ]
    };

    return new Response(JSON.stringify(secureData), {
      status: 200,
      headers: {
        'Content-Type': 'application/json',
        // Optional: CORS headers if needed for specific domains
        'Access-Control-Allow-Origin': '*'
      }
    });
  } catch (error: any) {
    return new Response(JSON.stringify({ error: 'Internal Server Error', details: error.message }), {
      status: 500,
      headers: { 'Content-Type': 'application/json' }
    });
  }
};
```

File: netlify/functions/send-otp.ts

```
import { getStore } from '@netlify/blobs';
import nodemailer from 'nodemailer';
import twilio from 'twilio';

export default async (req: Request) => {
  if (req.method !== 'POST') {
    return new Response('Method Not Allowed', { status: 405 });
  }

  try {
    const { email, phone } = await req.json();

    if (!email && !phone) {
      return new Response(JSON.stringify({ error: 'Email or phone number is required' })), { status:
400 });
    }

    // Generate 6 digit OTP
    const otp = Math.floor(100000 + Math.random() * 900000).toString();
    const identifier = email || phone;

    // Store in Netlify Blobs
    const store = getStore('otps');
    // Store OTP with an expiry time (e.g. 5 minutes from now)
    await store.setJSON(identifier, {
      otp,
      expiresAt: Date.now() + 5 * 60 * 1000
    });

    let previewUrl = null;

    if (email) {
      let transporter;
      if (process.env.GMAIL_USER && process.env.GMAIL_PASS) {
        // Use Gmail if configured
        transporter = nodemailer.createTransport({
          service: 'gmail',
          auth: {
            user: process.env.GMAIL_USER,
            pass: process.env.GMAIL_PASS
          }
        });
      } else {
        // Use Ethereum Email for testing/free fallback
        const testAccount = await nodemailer.createTestAccount();
        transporter = nodemailer.createTransport({
          host: 'smtp.ethereal.email',
          port: 587,
          secure: false,
          auth: {
            user: testAccount.user,
            pass: testAccount.pass
          }
        });
      }
    }

    const info = await transporter.sendMail({
      from: '"CAREVIA Support" <noreply@carevia.app>',
      to: email,
      subject: 'Your CAREVIA Verification Code',
      text: `Your verification code is: ${otp}. It expires in 5 minutes.`,
      html: `
        <div style="font-family: sans-serif; max-width: 448px; margin: 0 auto; padding: 20px;
border: 1px solid #ccc; border-radius: 10px;">
          <h2 style="color: #2dd4bf;">CAREVIA Login</h2>
          <p>Your verification code is:</p>
          <h1 style="font-size: 32px; letter-spacing: 5px; color: #333;">${otp}</h1>
          <p>This code will expire in 5 minutes.</p>
      `
    });
  }
}
```

```

    </div>
  });

  if (!process.env.GMAIL_USER) {
    previewUrl = nodemailer.getTestMessageUrl(info);
    console.log('OTP sent to email. Preview URL: %s', previewUrl);
  }
} else if (phone) {
  // Send SMS using Twilio if configured
  if (process.env.TWILIO_ACCOUNT_SID && process.env.TWILIO_AUTH_TOKEN &&
process.env.TWILIO_PHONE_NUMBER) {
    const client = twilio(process.env.TWILIO_ACCOUNT_SID, process.env.TWILIO_AUTH_TOKEN);
    await client.messages.create({
      body: `Your CAREVIA verification code is: ${otp}. It expires in 5 minutes.`,
      from: process.env.TWILIO_PHONE_NUMBER,
      to: phone.startsWith('+') ? phone : `+91${phone}`
    });
    console.log(`OTP sent to phone ${phone} via Twilio.`);
  } else {
    // Fallback for free testing: log the OTP
    console.log(`[FREE SMS SIMULATION] OTP for ${phone} is: ${otp}`);
    previewUrl = `sms-simulation:${otp}`;
  }
}

return new Response(JSON.stringify({
  success: true,
  message: 'OTP sent successfully',
  previewUrl, // Return preview URL or simulated OTP so frontend can log/display it if needed for
testing
  mockOtp: previewUrl ? otp : undefined // Explicitly return the OTP if we are in mock mode
}), {
  status: 200,
  headers: { 'Content-Type': 'application/json' }
});

} catch (error: any) {
  console.error('Error sending OTP:', error);
  return new Response(JSON.stringify({ error: error.message || 'Failed to send OTP' }), {
    status: 500,
    headers: { 'Content-Type': 'application/json' }
  });
}
};

```

File: netlify/functions/send-welcome.ts

```
import nodemailer from 'nodemailer';

export default async (req: Request) => {
  if (req.method !== 'POST') {
    return new Response('Method Not Allowed', { status: 405 });
  }

  try {
    const { email } = await req.json();

    if (!email) {
      return new Response(JSON.stringify({ error: 'Email is required' }), { status: 400 });
    }

    let transporter;
    if (process.env.GMAIL_USER && process.env.GMAIL_PASS) {
      transporter = nodemailer.createTransport({
        service: 'gmail',
        auth: {
          user: process.env.GMAIL_USER,
          pass: process.env.GMAIL_PASS
        }
      });
    } else {
      const testAccount = await nodemailer.createTestAccount();
      transporter = nodemailer.createTransport({
        host: 'smtp.ethereal.email',
        port: 587,
        secure: false,
        auth: {
          user: testAccount.user,
          pass: testAccount.pass
        }
      });
    }

    const info = await transporter.sendMail({
      from: '"CAREVIA" <noreply@carevia.app>',
      to: email,
      subject: 'Welcome to CAREVIA!',
      text: 'Thank you for joining CAREVIA. We are glad to have you on board.',
      html: `
        <div style="font-family: sans-serif; max-width: 448px; margin: 0 auto; padding: 20px; border:
1px solid #ccc; border-radius: 10px;">
          <h2 style="color: #2dd4bf;">Welcome to CAREVIA!</h2>
          <p>Thank you for joining CAREVIA. We are dedicated to providing elite concierge healthcare
and domestic management.</p>
          <p>We're thrilled to have you with us!</p>
          <br/>
          <p>Best regards,<br/>The CAREVIA Team</p>
        </div>
      `
    });

    let previewUrl = null;
    if (!process.env.GMAIL_USER) {
      previewUrl = nodemailer.getTestMessageUrl(info);
      console.log('Welcome email sent. Preview URL: %s', previewUrl);
    }

    return new Response(JSON.stringify({
      success: true,
      message: 'Welcome email sent successfully',
      previewUrl
    }), {
      status: 200,
      headers: { 'Content-Type': 'application/json' }
    });
  }
}
```

```
} catch (error: any) {  
  console.error('Error sending welcome email:', error);  
  return new Response(JSON.stringify({ error: error.message || 'Failed to send welcome email' })), {  
    status: 500,  
    headers: { 'Content-Type': 'application/json' }  
  });  
}  
};
```

File: netlify/functions/signInWithFpnv.ts

```
import { JwtVerifier } from "aws-jwt-verify";
import { getApps, initializeApp, cert } from "firebase-admin/app";
import { getAuth, UserRecord } from "firebase-admin/auth";

// Find your Firebase project number in the Firebase console.
const FIREBASE_PROJECT_NUMBER = "948313425033";

// The issuer and audience claims of the FPNV token are specific to your project.
const issuer = `https://fpnv.googleapis.com/projects/${FIREBASE_PROJECT_NUMBER}`;
const audience = `https://fpnv.googleapis.com/projects/${FIREBASE_PROJECT_NUMBER}`;

// The JWKS URL contains the current public signing keys for FPNV tokens.
const jwksUri = "https://fpnv.googleapis.com/v1beta/jwks";

// Configure a JWT verifier to check the token
const fpnvVerifier = JwtVerifier.create({ issuer, audience, jwksUri });

export default async (req: Request) => {
  if (req.method !== 'POST') {
    return new Response('Method Not Allowed', { status: 405 });
  }

  // Get the FPNV token from the request body.
  let fpnvToken;
  try {
    const bodyText = await req.text();
    try {
      const bodyJson = JSON.parse(bodyText);
      fpnvToken = bodyJson.fpnvToken || bodyJson.token || bodyText;
    } catch {
      fpnvToken = bodyText; // Use raw body if not JSON
    }
  } catch {
    return new Response('Bad Request', { status: 400 });
  }

  if (!fpnvToken) {
    return new Response('Bad Request', { status: 400 });
  }

  let verifiedPhoneNumber;
  try {
    // Attempt to verify the token using the verifier configured above.
    const verifiedPayload = await fpnvVerifier.verify(fpnvToken);

    // If verification succeeds, the subject claim of the token contains the
    // verified phone number.
    verifiedPhoneNumber = verifiedPayload.sub;
  } catch (error) {
    // If verification fails, reject the token.
    return new Response('Forbidden: Invalid token', { status: 403 });
  }

  if (!verifiedPhoneNumber) {
    return new Response('Forbidden: No phone number in token subject', { status: 403 });
  }

  // Initialize Firebase Admin if not already initialized
  if (!getApps().length) {
    try {
      const serviceAccountStr = process.env.FIREBASE_SERVICE_ACCOUNT;
      if (serviceAccountStr) {
        const serviceAccount = JSON.parse(serviceAccountStr);
        initializeApp({
          credential: cert(serviceAccount)
        });
      } else {
        // Try initializing with default credentials
      }
    }
  }
}
```

```

        initializeApp();
    }
} catch (e) {
    console.error("Failed to initialize Firebase Admin:", e);
    if (!getApps().length) initializeApp();
}
}

const authAdmin = getAuth();

// Now that you have a verified phone number, look it up in your Firebase
// project's user database.
let user: UserRecord;
try {
    // If a user account already exists with the phone number, retrieve it.
    user = await authAdmin.getUserByPhoneNumber(verifiedPhoneNumber);
} catch {
    // Otherwise, create a new user account using the phone number.
    user = await authAdmin.createUser({phoneNumber: verifiedPhoneNumber});
}

// Finally, mint a Firebase custom auth token containing the UID of the user
// you looked up or created. Return this token to the caller.
const authToken = await authAdmin.createCustomToken(user.uid);
return new Response(JSON.stringify({ authToken }), {
    status: 200,
    headers: { 'Content-Type': 'application/json' }
});
};

```

File: netlify/functions/verify-otp.ts

```
import { getStore } from '@netlify/blobs';
import { getApps, initializeApp, cert } from 'firebase-admin/app';
import { getAuth } from 'firebase-admin/auth';

export default async (req: Request) => {
  if (req.method !== 'POST') {
    return new Response('Method Not Allowed', { status: 405 });
  }

  try {
    const { email, phone, otp } = await req.json();
    const identifier = email || phone;

    if (!identifier || !otp) {
      return new Response(JSON.stringify({ error: 'Email or phone and OTP are required' })), { status:
400 });
    }

    const store = getStore('otps');
    const storedData = await store.get(identifier, { type: 'json' });

    if (!storedData) {
      return new Response(JSON.stringify({ error: 'OTP expired or invalid' })), { status: 400 };
    }

    const { otp: storedOtp, expiresAt } = storedData as { otp: string, expiresAt: number };

    if (Date.now() > expiresAt) {
      await store.delete(identifier);
      return new Response(JSON.stringify({ error: 'OTP expired' })), { status: 400 };
    }

    if (storedOtp !== otp) {
      return new Response(JSON.stringify({ error: 'Invalid OTP' })), { status: 400 };
    }

    // Success! Delete the OTP
    await store.delete(identifier);

    // Initialize Firebase Admin
    if (!getApps().length) {
      try {
        const serviceAccountStr = process.env.FIREBASE_SERVICE_ACCOUNT;
        if (serviceAccountStr) {
          const serviceAccount = JSON.parse(serviceAccountStr);
          initializeApp({
            credential: cert(serviceAccount)
          });
        } else {
          initializeApp();
        }
      } catch (e) {
        console.error("Failed to initialize Firebase Admin:", e);
        if (!getApps().length) initializeApp();
      }
    }

    const authAdmin = getAuth();
    let user;

    if (email) {
      try {
        user = await authAdmin.getUserByEmail(email);
      } catch {
        user = await authAdmin.createUser({ email });
      }
    } else {
      try {

```

```

        user = await authAdmin.getUserByPhoneNumber(phone);
    } catch {
        user = await authAdmin.createUser({ phoneNumber: phone });
    }
}

const authToken = await authAdmin.createCustomToken(user.uid);

return new Response(JSON.stringify({
    success: true,
    message: 'OTP verified successfully',
    authToken
}), {
    status: 200,
    headers: { 'Content-Type': 'application/json' }
});

} catch (error: any) {
    console.error('Error verifying OTP:', error);
    return new Response(JSON.stringify({ error: error.message || 'Failed to verify OTP' }), {
        status: 500,
        headers: { 'Content-Type': 'application/json' }
    });
}
};

```

File: package.json

```
{
  "name": "react-example",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite --port=3000 --host=0.0.0.0",
    "prebuild": "node scripts/generate-code-pdf.cjs",
    "build": "vite build",
    "preview": "vite preview",
    "clean": "rm -rf dist",
    "lint": "tsc --noEmit"
  },
  "dependencies": {
    "@firebase/eslint-plugin-security-rules": "^0.0.2",
    "@google/genai": "^1.29.0",
    "@netlify/blobs": "^10.7.4",
    "@tailwindcss/vite": "^4.1.14",
    "@types/nodemailer": "^8.0.0",
    "@vitejs/plugin-react": "^5.0.4",
    "aws-jwt-verify": "^5.1.1",
    "dotenv": "^17.2.3",
    "express": "^4.21.2",
    "firebase": "^12.12.1",
    "firebase-admin": "^13.8.0",
    "firebase-functions": "^7.2.5",
    "google-spreadsheet": "^5.2.0",
    "lucide-react": "^0.546.0",
    "motion": "^12.23.24",
    "nodemailer": "^8.0.6",
    "pdfkit": "^0.18.0",
    "react": "^19.0.0",
    "react-dom": "^19.0.0",
    "twilio": "^6.0.0",
    "vite": "^6.2.0"
  },
  "devDependencies": {
    "@types/express": "^4.17.21",
    "@types/node": "^22.14.0",
    "autoprefixer": "^10.4.21",
    "tailwindcss": "^4.1.14",
    "tsx": "^4.21.0",
    "typescript": "~5.8.2",
    "vite": "^6.2.0"
  }
}
```

File: src/App.tsx

```
import React, { useState, useEffect, useMemo, useCallback } from 'react';
import { GoogleGenAI } from "@google/genai";
import { motion, AnimatePresence } from 'motion/react';
import {
  HeartPulse,
  ShieldCheck,
  Clock,
  Phone,
  ChevronRight,
  Star,
  Menu,
  X,
  Wallet,
  AlertTriangle,
  User,
  Logout,
  Eye,
  EyeOff,
  LocateFixed,
  Ambulance,
  LifeBuoy,
  Stethoscope,
  BriefcaseMedical,
  Baby,
  Dog,
  ChefHat,
  Car,
  Home,
  ArrowRight,
  TrendingUp,
  CheckCircle2,
  Lock,
  Smartphone,
  Info,
  LayoutDashboard,
  MessageSquare,
  Mail,
  HelpCircle,
  HelpCircle as StepsIcon,
  Navigation,
  Sparkles,
  Zap,
  Users,
  Bot,
  Send,
  Loader2,
  Search,
  Settings,
  Database,
  Activity,
  Shield,
  ShieldAlert,
  Calendar,
  MapPin,
  GraduationCap,
  Upload,
  FileText,
  Handshake,
  Map,
  ArrowUpDown,
  Brain,
  Cloud,
  FileSpreadsheet,
  Login,
  Download,
  Trash2
} from 'lucide-react';
```

```

import { auth, db, handleFirestoreError } from './lib/firebase';
import {
  onAuthStateChanged,
  signInWithPopup,
  GoogleAuthProvider,
  signOut,
  RecaptchaVerifier,
  signInWithPhoneNumber,
  sendEmailVerification,
  signInWithEmailAndPassword,
  createUserWithEmailAndPassword,
  signInWithCustomToken,
} from 'firebase/auth';
import {
  doc,
  setDoc,
  getDoc,
  onSnapshot,
  collection,
  query,
  where,
  orderBy,
  limit,
  addDoc,
  updateDoc,
  serverTimestamp
} from 'firebase/firestore';

import { exportToSheets, uploadToDrive } from './lib/workspace';
import { LanguageSelector } from './LanguageSelector';
import { applyHindiCorrections } from './hindiCorrector';

// --- Components ---

const Logo = ({ className }: { className?: string }) => (
  <a href="/" className={`flex items-center gap-3 hover:opacity-80 transition-opacity cursor-pointer`}
  ${className || ''}>
    <div className="relative w-12 h-12 flex items-center justify-center">
      <div className="absolute inset-0 bg-primary/20 rounded-xl blur-xl animate-pulse" />
      
    </div>
    <div className="flex flex-col">
      <span className="text-xl font-black uppercase tracking-tighter leading-none text-white">CAREVIA</span>
      <span className="text-[8px] font-bold uppercase tracking-[0.3em] text-primary">Health •
Wellness • Compassion</span>
    </div>
  </a>
);

// --- Constants ---

const CATEGORIES = [
  {
    id: 'elderly',
    name: 'Elderly Care',
    icon: <LifeBuoy />,
    description: 'Compassionate and professional care for seniors, focusing on mobility, health
monitoring, and emotional well-being.',
    pricing: [
      { tier: 'Basic', price: ' $350/hr', features: ['Daily Hygiene', 'Feeding Assistance', 'Mobility
Support'] },
      { tier: 'Standard', price: ' $600/hr', features: ['Vital Monitoring', 'Medication Mgmt',
'Specialized Diet'] },
      { tier: 'Premium', price: ' $1200/hr', features: ['Critical Care Nursing', 'Physiotherapy', '24/7
Response'] }
    ],
  },

```

```

sections: [
  { name: 'Daily Life Assistance', items: ['Bathing, dressing, grooming', 'Toileting & hygiene support', 'Feeding assistance', 'Mobility support'] },
  { name: 'Medical Support', items: ['Certified nurses', 'Physiotherapy at home', 'Post-surgery care', 'Medication management', 'Vital monitoring'] },
  { name: 'Home Support', items: ['Special diet cooking', 'House maintenance', 'Laundry services', 'Grocery delivery'] },
  { name: 'Emergency', items: ['24/7 Response', 'SOS System', 'Check-in calls', 'Ambulance coordination'] },
  { name: 'Specialized Care', items: ['Dementia / Alzheimer's care', 'Bedridden patient care', 'Palliative care', 'Parkinson's care'] }
],
},
{
  id: 'child',
  name: 'Child Care',
  icon: <Baby />,
  description: 'Nurturing and developmental care for children of all ages, ensuring their safety and growth in a stimulating environment.',
  pricing: [
    { tier: 'Babysitting', price: ' $250/hr', features: ['Supervised Play', 'Meal Prep', 'Basic Hygiene'] },
    { tier: 'Educational', price: ' $500/hr', features: ['Homework Help', 'Skill Building', 'Activity Planning'] },
    { tier: 'Full-Time Nanny', price: ' $25,000/mo', features: ['24/7 Support', 'Developmental Tracking', 'School Coordination'] }
  ],
  isFull: true,
  sections: [
    { name: 'Daily Child Care', items: ['Feeding & meal preparation', 'Bathing & dressing', 'Diaper changing', 'Sleep routine management'] },
    { name: 'Babysitting', items: ['Part-time babysitters', 'Full-time nanny', '24/7 live-in nanny', 'On-demand (hourly)'] },
    { name: 'Development', items: ['Homework help', 'Basic tutoring', 'Storytelling', 'Skill-building games'] },
    { name: 'Activity', items: ['Indoor games', 'Outdoor activities', 'Arts & crafts', 'Music & dance sessions'] },
    { name: 'School Support', items: ['School pick-up & drop', 'Schedule management', 'Lunch preparation', 'Exam support'] }
  ],
},
{
  id: 'pet',
  name: 'Pet Care',
  icon: <Dog />,
  description: 'Expert care for your furry friends, from daily exercise to specialized medical attention and grooming.',
  pricing: [
    { tier: 'Walking', price: ' $150/sess', features: ['30min Walk', 'Fresh Water', 'Waste Cleanup'] },
    { tier: 'Grooming', price: ' $800/sess', features: ['Full Bath', 'Nail Trimming', 'Ear Cleaning'] },
    { tier: 'Vet Care', price: ' $1200/visit', features: ['Home Vaccination', 'Health Checkup', 'Expert Advice'] }
  ],
  isFull: true,
  sections: [
    { name: 'Daily Pet Care', items: ['Feeding & fresh water', 'Walking & exercise', 'Cleaning (litter box/cages)', 'Basic hygiene care'] },
    { name: 'Grooming', items: ['Bathing & drying', 'Nail trimming', 'Hair styling', 'Ear & dental cleaning'] },
    { name: 'Medical Care', items: ['Vet booking', 'Home visits by vets', 'Vaccination support', 'Medication administration'] },
    { name: 'Pet Sitting', items: ['In-home sitting', 'Overnight care', 'Playtime & companionship'] }
  ],
},
{
  id: 'helpers',
  name: 'Helpers',
  icon: <Home />,
  description: 'Reliable domestic staffing solutions for a well-managed and comfortable lifestyle.',
  pricing: [

```

```

    { tier: 'On-Demand', price: ' ₹400/visit', features: ['Basic Cleaning', 'Grocery Run', 'Minor Fixes'] },
    { tier: 'Regular', price: ' ₹12,000/mo', features: ['Daily Housekeeping', 'Cooking', 'Laundry'] },
    { tier: 'Specialized', price: ' ₹18,000/mo', features: ['Chauffeur', 'Professional Cook', 'Full-time Helper'] }
  ],
  sections: [
    { name: 'Domestic Staff', items: ['House helpers', 'Drivers', 'Cooks', 'Utility helpers', 'On-demand cleaning', 'Home utility helper'] }
  ]
}
];

const LOCATIONS = [
  "Agra", "Ahmedabad", "Ajmer", "Aligarh", "Amravati", "Amritsar", "Asansol", "Aurangabad",
  "Bareilly", "Belgaum", "Bengaluru", "Bhavnagar", "Bhiwandi", "Bhopal", "Bhubaneswar",
  "Bikaner", "Bokaro", "Chandigarh", "Chennai", "Coimbatore", "Cuttack", "Dehradun",
  "Delhi", "Dhanbad", "Durgapur", "Erode", "Faridabad", "Firozabad", "Ghaziabad",
  "Gorakhpur", "Gulbarga", "Guntur", "Gurgaon", "Guwahati", "Gwalior", "Hubli-Dharwad",
  "Hyderabad", "Indore", "Jabalpur", "Jaipur", "Jalandhar", "Jalgaon", "Jammu",
  "Jamnagar", "Jamshedpur", "Jhansi", "Jodhpur", "Kakinada", "Kannur", "Kanpur",
  "Kochi", "Kolhapur", "Kollam", "Kolkata", "Kozhikode", "Kurnool", "Lucknow",
  "Ludhiana", "Madurai", "Malappuram", "Mangalore", "Mathura", "Meerut", "Moradabad",
  "Mumbai", "Mysore", "Nagpur", "Nanded", "Nashik", "Nellore", "Noida", "Patna",
  "Puducherry", "Pune", "Purulia", "Prayagraj", "Raipur", "Rajkot", "Rajahmundry",
  "Ranchi", "Rourkela", "Salem", "Sangli", "Shimla", "Siliguri", "Solapur", "Srinagar",
  "Surat", "Thiruvananthapuram", "Thrissur", "Tiruchirappalli", "Tirunelveli", "Tirupur",
  "Ujjain", "Vadodara", "Varanasi", "Vasai-Virar", "Vijayawada", "Visakhapatnam",
  "Vellore", "Warangal"
];

const MOCK_PROVIDERS: any[] = [];

const NAV_LINKS = [
  { name: 'Home', href: '#' },
  { name: 'How to use', href: '#how-to-use' },
  { name: 'Services', href: '#services' },
  { name: 'Pricing', href: '#pricing' },
  { name: 'About', href: '#about' },
  { name: 'Contact', href: '#contact' }
];

// --- Components ---

const Button = ({ children, className = '', variant = 'primary', ...props }: any) => {
  const variants: any = {
    primary: 'btn-primary',
    secondary: 'bg-white text-surface hover:bg-neutral-200',
    gold: 'btn-gold',
    outline: 'btn-outline',
    ghost: 'hover:bg-white/10 text-white'
  };
  return (
    <button
      className={`px-6 py-3 rounded-full font-bold transition-all active:scale-95 flex items-center justify-center gap-2 ${variants[variant]} ${className}`}
      {...props}
    >
      {children}
    </button>
  );
};

const SectionHeading = ({ sub, title, desc, centered = false }: any) => (
  <div className={`mb-16 ${centered ? 'text-center mx-auto' : ''} max-w-3xl`}
    {sub && <p className="text-primary font-bold uppercase tracking-[0.3em] text-xs mb-4">{sub}</p>
    <h2 className="text-4xl md:text-6xl mb-6">{title}</h2>
    {desc && <p className="text-white/70 text-lg font-light leading-relaxed">{desc}</p>
  </div>
);

```

```
// --- App Sections ---
```

```
const SOSButton = ({ user }: { user: any }) => {
  const [active, setActive] = useState(false);
  const [contact, setContact] = useState(() => localStorage.getItem('emergency_contact') || '');
  const [isEditing, setIsEditing] = useState(false);
  const [tempContact, setTempContact] = useState(contact);
  const [isLogging, setIsLogging] = useState(false);

  const handleSOSAlert = async (type: string) => {
    if (!user) return;
    setIsLogging(true);
    try {
      await addDoc(collection(db, 'queries'), {
        userId: user.uid,
        userEmail: user.email,
        category: 'SOS',
        type: type,
        message: `EMERGENCY ALERT: ${type} requested.`,
        status: 'CRITICAL',
        createdAt: serverTimestamp()
      });
      alert(`Success: ${type} notified via Cloud Protocol.`);
    } catch (e) {
      console.error(e);
      alert("Cloud connection error. Use direct call.");
    } finally {
      setIsLogging(false);
    }
  };

  const handleSaveContact = () => {
    if (tempContact.trim()) {
      localStorage.setItem('emergency_contact', tempContact.trim());
      setContact(tempContact.trim());
      setIsEditing(false);
    }
  };

  const clearContact = () => {
    localStorage.removeItem('emergency_contact');
    setContact('');
    setTempContact('');
  };

  return (
    <div className="fixed bottom-8 right-8 z-[100] flex flex-col items-center gap-4">
      <AnimatePresence>
        {active && (
          <motion.div
            initial={{ opacity: 0, y: 20, scale: 0.8 }}
            animate={{ opacity: 1, y: 0, scale: 1 }}
            exit={{ opacity: 0, y: 20, scale: 0.8 }}
            className="glass-card p-6 rounded-[2rem] mb-4 w-80 text-center border-[#FF0000]/20 shadow-[0_0_50px_rgba(255,0,0,0.2)]"
          >
            <div className="w-14 h-14 bg-[#FF0000]/20 text-[#FF0000] rounded-full flex items-center justify-center mx-auto mb-4 border border-[#FF0000]/20">
              <AlertTriangle className="animate-pulse w-7 h-7" />
            </div>
            <h3 className="text-xl font-black uppercase tracking-tight mb-1 text-white">Emergency Center</h3>
            <p className="text-[10px] text-white/60 uppercase tracking-[0.2em] mb-6 font-bold">Standard Response Protocol Active</p>

            <div className="space-y-4">
              <div className="space-y-2">
                <button
                  disabled={isLogging}
                  onClick={() => handleSOSAlert('CARE_POLICE')}
                  className="w-full py-4 rounded-2xl bg-white/5 border border-white/10 hover:bg-
```

```

red-500/10 hover:border-red-500/50 transition-all flex items-center justify-center gap-3 group"
    >
        <ShieldCheck className={`w-4 h-4 ${isLogging ? 'animate-spin' : 'text-white/40
group-hover:text-red-500'}}` />
        <span className="text-[10px] font-bold uppercase tracking-widest text-
white/80">{isLogging ? 'Transmitting...' : 'Notify Care Police'}</span>
    </button>
    <button
        disabled={isLogging}
        onClick={() => handleSOSAlert('AIR_AMBULANCE')}
        className="w-full py-4 rounded-2xl bg-white/5 border border-white/10 hover:bg-
blue-500/10 hover:border-blue-500/50 transition-all flex items-center justify-center gap-3 group"
    >
        <Ambulance className={`w-4 h-4 ${isLogging ? 'animate-spin' : 'text-white/40 group-
hover:text-blue-500'}}` />
        <span className="text-[10px] font-bold uppercase tracking-widest text-
white/80">{isLogging ? 'Transmitting...' : 'Call Air Ambulance'}</span>
    </button>

    {contact && !isEditing && (
        <motion.button
            initial={{ opacity: 0, scale: 0.9 }}
            animate={{ opacity: 1, scale: 1 }}
            onClick={() => window.location.href = `tel:${contact}`}
            className="w-full py-5 rounded-2xl bg-green-500/10 border border-green-500/40
hover:bg-green-500 hover:text-surface transition-all flex items-center justify-center gap-3 group"
        >
            <Phone className="w-4 h-4 text-green-500 group-hover:text-surface transition-
colors" />
            <span className="text-[10px] font-black uppercase tracking-widest">Direct Call:
{contact}</span>
        </motion.button>
    )}
</div>

<div className="pt-6 border-t border-white/5">
    {isEditing ? (
        <div className="bg-white/5 rounded-2xl p-4 border border-white/10">
            <p className="text-[9px] font-bold uppercase tracking-widest text-white/50 text-
left mb-2">Emergency Contact #</p>
            <input
                type="tel"
                value={tempContact}
                onChange={(e) => setTempContact(e.target.value)}
                placeholder="+91 XXXXX XXXXX"
                className="w-full bg-surface border border-white/5 px-4 py-3 rounded-xl text-xs
outline-none focus:border-primary transition-all mb-4"
            />
            <div className="flex gap-2">
                <button onClick={handleSaveContact} className="flex-1 py-3 bg-primary text-
surface rounded-xl text-[10px] font-black uppercase tracking-widest">Establish</button>
                <button onClick={() => setIsEditing(false)} className="flex-1 py-3 bg-white/5
text-white/60 rounded-xl text-[10px] font-bold uppercase">Cancel</button>
            </div>
        </div>
    ) : (
        <div className="flex flex-col gap-3">
            <button
                onClick={() => {
                    setTempContact(contact);
                    setIsEditing(true);
                }}
                className="text-[10px] uppercase font-bold text-primary/80 hover:text-primary
flex items-center justify-center gap-2 transition-colors"
            >
                {contact ? <Settings className="w-3 h-3" /> : <User className="w-3 h-3" />}
                <span>{contact ? 'Modify Protocol Contact' : 'Register Emergency Contact'}</span>
            </button>
            {contact && (
                <button onClick={clearContact} className="text-[9px] uppercase font-bold text-
red-500/30 hover:text-red-500 transition-colors">Terminate Link</button>

```

```

        ))
      </div>
    )
  }
</div>

  <button
    onClick={() => setActive(false)}
    className="text-white/40 text-[9px] uppercase font-black tracking-widest mt-4
  hover:text-white transition-all flex items-center justify-center gap-2 mx-auto"
  >
    <X className="w-3 h-3" /> Deactivate SOS
  </button>
</div>
</motion.div>
))
</AnimatePresence>

<button
  onClick={() => setActive(!active)}
  className={`w-16 h-16 rounded-full flex items-center justify-center transition-all
duration-700 shadow-[0_0_40px_rgba(255,0,0,0.4)] relative group ${active ? 'bg-white rotate-180' : 'bg-
[#FF0000] hover:bg-[#CC0000] hover:scale-110'}`}
  >
    <div className={`absolute inset-0 rounded-full animate-ping bg-[#FF0000] opacity-30 group-
hover:opacity-50 ${active ? 'hidden' : ''}`} />
    {active ? <X className="text-surface w-6 h-6" /> : <AlertTriangle className="text-white w-7
h-7" />}
  </button>
</div>
);
};

```

```

const PaymentModal = ({ isOpen, onClose, planName, onPaymentComplete }: any) => {
  if (!isOpen) return null;

  return (
    <div className="fixed inset-0 z-[300] flex items-center justify-center p-6 font-sans">
      <motion.div
        initial={{ opacity: 0 }}
        animate={{ opacity: 1 }}
        onClick={onClose}
        className="absolute inset-0 bg-black/90 backdrop-blur-md"
      />
      <motion.div
        initial={{ opacity: 0, scale: 0.9, y: 30 }}
        animate={{ opacity: 1, scale: 1, y: 0 }}
        className="glass-card w-full max-w-md relative z-10 overflow-hidden"
      >
        <div className="p-8 border-b border-white/5 bg-primary/5 flex justify-between items-center">
          <div>
            <p className="text-[10px] font-bold text-primary uppercase tracking-[0.2em]
mb-1">Premium Activation</p>
            <h3 className="text-xl font-bold">{planName} Pack</h3>
          </div>
          <button onClick={onClose} className="p-2 hover:bg-white/10 rounded-full"><X className="w-5
h-5 text-white/60" /></button>
        </div>

        <div className="p-8 text-center space-y-6">
          <div className="bg-white p-4 rounded-3xl inline-block shadow-2xl shadow-primary/20">
             {
                // Fallback if the link fails
                e.currentTarget.src = "https://img.icons8.com/isometric/512/qr-code.png";
              }}
            />
          </div>
        </div>
      </motion.div>
    </div>
  )
}

```

```

    />
  </div>
  <div>
    <p className="text-sm font-bold text-white mb-2">Scan QR to Complete Transaction</p>
    <p className="text-[10px] text-white/60 uppercase tracking-widest leading-relaxed">
      Please mention your <span className="text-primary">Registered Mobile Number</span> in
the transaction remarks for instant activation.
    </p>
  </div>
  <div className="pt-4 space-y-3">
    <input type="text" placeholder="Referral Code (Optional)" className="w-full bg-white/5
border border-white/10 rounded-xl px-4 py-3 text-xs outline-none focus:border-primary/50 text-white" />
    <input type="text" placeholder="Coupon Code (Optional)" className="w-full bg-white/5
border border-white/10 rounded-xl px-4 py-3 text-xs outline-none focus:border-primary/50 text-white" />
  </div>
  <div className="pt-4 border-t border-white/5">
    <Button variant="primary" className="w-full py-4 uppercase tracking-widest text-xs"
onClick={() => {
      if (onPaymentComplete) {
        onPaymentComplete();
      } else {
        onClose();
      }
    }}>
      I Have Paid
    </Button>
  </div>
</div>
</motion.div>
</div>
);
};

const WalletModal = ({ isOpen, onClose }: { isOpen: boolean, onClose: () => void }) => {
  if (!isOpen) return null;

  return (
    <div className="fixed inset-0 z-[300] flex items-center justify-center p-6 font-sans">
      <motion.div
        initial={{ opacity: 0 }}
        animate={{ opacity: 1 }}
        onClick={onClose}
        className="absolute inset-0 bg-black/90 backdrop-blur-md"
      />
      <motion.div
        initial={{ opacity: 0, scale: 0.9, y: 30 }}
        animate={{ opacity: 1, scale: 1, y: 0 }}
        className="glass-card w-full max-w-md relative z-10 overflow-hidden"
      >
        <div className="p-8 border-b border-white/5 bg-primary/5 flex justify-between items-center">
          <div>
            <p className="text-[10px] font-bold text-primary uppercase tracking-[0.2em]
mb-1">Financial Portal</p>
            <h3 className="text-xl font-bold">CARVIA Wallet</h3>
          </div>
          <button onClick={onClose} className="p-2 hover:bg-white/10 rounded-full"><X className="w-5
h-5 text-white/60" /></button>
        </div>

        <div className="p-8 text-center space-y-6">
          <div className="p-6 bg-white/5 rounded-3xl border border-white/10 mb-6">
            <p className="text-[10px] uppercase font-bold text-white/50 tracking-widest
mb-1">Current Balance</p>
            <p className="text-4xl font-black text-white italic tracking-tighter"> 10.00</p>
          </div>

          <div className="bg-white p-4 rounded-3xl inline-block shadow-2xl shadow-primary/20">
            
          </div>
        </div>
      </motion.div>
    </div>
  );
};

```

```

        className="w-56 h-56 object-contain rounded-xl"
        referrerPolicy="no-referrer"
        onError={(e) => {
            e.currentTarget.src = "https://img.icons8.com/isometric/512/qr-code.png";
        }}
    />
</div>

<div>
    <p className="text-sm font-bold text-white mb-2">Scan QR to Add Credits</p>
    <p className="text-[10px] text-white/60 uppercase tracking-widest leading-relaxed">
        Add funds instantly to your CARVIA Wallet for any transaction.
    <br />Funds will reflect after <span className="text-emerald-500">System Verification</
span>.

    </p>
</div>

    <div className="pt-4 border-t border-white/5">
        <Button variant="primary" className="w-full py-4 uppercase tracking-widest text-xs"
onClick={onClose}>
            Done
        </Button>
    </div>
</div>
</motion.div>
</div>
);
};

const AuthModal = ({ isOpen, onClose, onOpenAdmin, onLogin, onProviderLogin }: any) => {
    const [mode, setMode] = useState('login'); // login, signup, admin
    const [showPass, setShowPass] = useState(false);
    const [method, setMethod] = useState('mobile'); // mobile, email
    const [otpSent, setOtpSent] = useState(false);
    const [otp, setOtp] = useState('');
    const [error, setError] = useState('');
    const [isResending, setIsResending] = useState(false);
    const [phone, setPhone] = useState('');
    const [email, setEmail] = useState('');

    useEffect(() => {
        setOtpSent(false);
        setShowPass(false);
        setError('');
        setOtp('');
        setPhone('');
        setEmail('');
    }, [mode, method, isOpen]);

    const [confirmationResult, setConfirmationResult] = useState<any>(null);
    const [loading, setLoading] = useState(false);

    const saveToExcel = async (userData: any) => {
        try {
            const formData = new URLSearchParams();
            formData.append('form-name', 'registrations');
            Object.keys(userData).forEach(key => formData.append(key, userData[key]));

            await fetch('/', {
                method: 'POST',
                headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
                body: formData.toString()
            });
        } catch (err) {
            console.error('Failed to save to Excel/Form', err);
        }
    };

    const setupRecaptcha = () => {
        if ((window as any).recaptchaVerifier) {

```

```

    return (window as any).recaptchaVerifier;
}

// Set language as requested
auth.languageCode = 'it';
// To apply the default browser preference instead of explicitly setting it.
auth.useDeviceLanguage();

const verifier = new RecaptchaVerifier(auth, 'recaptcha-container', {
  'size': 'normal',
  'callback': (response: any) => {
    // reCAPTCHA solved, allow signInWithPhoneNumber.
    console.log('reCAPTCHA solved', response);
    // If we needed to get the response explicitly as requested:
    // const recaptchaResponse = grecaptcha.getResponse(recaptchaWidgetId);
    if (typeof (window as any).grecaptcha !== 'undefined' && (window as any).recaptchaWidgetId !==
undefined) {
      const recaptchaResponse = (window as any).grecaptcha.getResponse((window as
any).recaptchaWidgetId);
      console.log('Recaptcha response:', recaptchaResponse);
    }
  },
  'expired-callback': () => {
    // Response expired. Ask user to solve reCAPTCHA again.
    console.log('reCAPTCHA expired');
  }
});

verifier.render().then((widgetId) => {
  (window as any).recaptchaWidgetId = widgetId;
});

(window as any).recaptchaVerifier = verifier;
return verifier;
};

const handleGoogleLogin = async () => {
  setLoading(true);
  setError('');
  try {
    const provider = new GoogleAuthProvider();
    const cred = await signInWithPopup(auth, provider);
    const user = cred.user;

    await setDoc(doc(db, 'users', user.uid), {
      uid: user.uid,
      email: user.email,
      displayName: user.displayName,
      photoURL: user.photoURL,
      role: 'customer',
      updatedAt: serverTimestamp(),
      lastLogin: serverTimestamp()
    }, { merge: true });

    try {
      await saveToExcel({ uid: user.uid, role: 'customer', phone: '', email: user.email || '' });
    } catch(e) {}

    try {
      if (user.email) {
        await fetch('/.netlify/functions/send-welcome', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ email: user.email })
        });
      }
    } catch(e) { console.error('Error sending welcome email:', e); }

    onLogin();
    onClose();
  } catch(err: any) {

```

```

    console.error(err);
    setError(err.message || 'Google Sign-In failed');
  } finally {
    setLoading(false);
  }
};

const handleAction = async () => {
  setError('');
  setLoading(true);

  try {
    if (method === 'mobile') {
      if (otpSent) {
        if (!otp) throw new Error('Please enter the OTP');
        if (!confirmationResult) throw new Error('Session expired, please try again.');
```

const cred = await confirmationResult.confirm(otp);
const user = cred.user;

let role = 'customer';
if (phone.startsWith('99')) role = 'provider';
if (phone === import.meta.env.VITE_ADMIN_PHONE) role = 'admin';

// Initial Profile Sync
await setDoc(doc(db, 'users', user.uid), {
 uid: user.uid,
 phone: phone,
 role: role,
 updatedAt: serverTimestamp(),
 lastLogin: serverTimestamp()
}, { merge: true });
await saveToExcel({ uid: user.uid, role: role, phone: phone || '', email: '' });

if (role === 'admin') onOpenAdmin();
else if (role === 'provider') onProviderLogin();
else onLogin();
onClose();

} else {
 if (phone.length < 10) throw new Error('Invalid phone number');
 const formattedPhone = phone.startsWith('+') ? phone : `+91\${phone}`;
 const verifier = setupRecaptcha();
 const result = await signInWithPhoneNumber(auth, formattedPhone, verifier);
 setConfirmationResult(result);
 setOtpSent(true);
}

} else {
 if (!otpSent) {
 if (!email) throw new Error('Please enter your email');
 const res = await fetch('/.netlify/functions/send-otp', {
 method: 'POST',
 headers: { 'Content-Type': 'application/json' },
 body: JSON.stringify({ email })
 });
 const data = await res.json();
 if (!res.ok) throw new Error(data.error || 'Failed to send OTP');
 if (data.mockOtp) alert('TEST MODE OTP: ' + data.mockOtp);
 else if (data.previewUrl) alert('TEST MODE LINK: ' + data.previewUrl);
 setOtpSent(true);
 } else {
 if (!otp) throw new Error('Please enter the OTP sent to your email');
 const res = await fetch('/.netlify/functions/verify-otp', {
 method: 'POST',
 headers: { 'Content-Type': 'application/json' },
 body: JSON.stringify({ email, otp })
 });
 const data = await res.json();
 if (!res.ok) throw new Error(data.error || 'Invalid OTP');

let user;
try {

```

        const cred = await signInWithCustomToken(auth, data.authToken);
        user = cred.user;
    } catch(err: any) {
        throw new Error(err.message || 'Authentication failed with custom token');
    }

    await setDoc(doc(db, 'users', user.uid), {
        uid: user.uid,
        email: user.email,
        role: 'customer',
        updatedAt: serverTimestamp(),
        lastLogin: serverTimestamp()
    }, { merge: true });

    try {
        await saveToExcel({ uid: user.uid, role: 'customer', phone: '', email: user.email ||
'' });
    } catch(e) {}

    try {
        if (user.email) {
            await fetch('/.netlify/functions/send-welcome', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ email: user.email })
            });
        }
    } catch(e) { console.error('Error sending welcome email:', e); }

    // AI Welcome Message
    try {
        const ai = new GoogleGenAI({ apiKey: process.env.GEMINI_API_KEY });
        const aiResp = await ai.models.generateContent({
            model: "gemini-2.5-flash",
            contents: [{ role: 'user', parts: [{ text: "Generate a short 1-sentence warm welcome
message for a user named " + email.split('@')[0] + " who just logged into CAREVIA, a premium
healthcare platform." }] }]
        });
        setTimeout(() => alert("CAREVIA AI: " + aiResp.text), 500);
    } catch(e) { console.error('AI error', e); }

    onLogin();
    onClose();
}
} catch (err: any) {
    console.error(err);
    if (err.code === 'auth/operation-not-allowed') {
        setError('Authentication method not enabled. Please enable "Google" and "Phone" sign-in in
your Firebase Console (Authentication > Sign-in method).');
    } else {
        setError(err.message || 'Authentication failed');
    }
    if (err.code === 'auth/invalid-verification-code') {
        setError('Invalid OTP. Please try again.');
```

```

    } else {
      const res = await fetch('/.netlify/functions/send-otp', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ email })
      });
      const data = await res.json();
      if (!res.ok) throw new Error(data.error || 'Failed to send OTP');
      if (data.mockOtp) alert('TEST MODE OTP: ' + data.mockOtp);
      else if (data.previewUrl) alert('TEST MODE LINK: ' + data.previewUrl);
    }
  } catch (err: any) {
    setError(err.message);
  } finally {
    setIsResending(false);
  }
};

const handleSPLogin = async () => {
  try {
    const spIdVal = email.trim();
    if (!spIdVal) throw new Error("Please enter SP ID NO");
    const q = query(collection(db, 'users'), where('spId', '==', spIdVal));
    const snap = await getDocs(q);
    if (snap.empty) {
      throw new Error("Invalid SP ID NO");
    }
    const provider = snap.docs[0].data();
    if (!provider.isVerified) {
      throw new Error("Your application is pending or rejected.");
    }
    onProviderLogin();
    onClose();
  } catch (err: any) {
    setError(err.message);
  }
};

if (!isOpen) return null;

return (
  <div className="fixed inset-0 z-[200] flex items-center justify-center p-6 font-sans">
    <motion.div
      initial={{ opacity: 0 }}
      animate={{ opacity: 1 }}
      onClick={onClose}
      className="absolute inset-0 bg-black/80 backdrop-blur-sm"
    />
    <motion.div
      initial={{ opacity: 0, scale: 0.9, y: 30 }}
      animate={{ opacity: 1, scale: 1, y: 0 }}
      className="glass-card w-full max-w-md relative z-10 overflow-hidden"
    >
      <div className="flex justify-between items-center mb-8">
        <h2 className="text-2xl font-bold uppercase tracking-tight">
          {mode === 'login' ? 'Welcome Back' : mode === 'signup' ? 'Create Account' : mode ===
'sp' ? 'Provider Access' : 'Admin Hub'}
        </h2>
        <button onClick={onClose} className="p-2 hover:bg-white/10 rounded-full transition-colors">
          <X className="w-5 h-5 text-white/70" />
        </button>
      </div>

      {mode !== 'sp' && (
        <div className="flex gap-4 mb-6">
          <button
            onClick={() => setMethod('mobile')}
            className={`flex-1 py-2 text-xs font-bold uppercase tracking-widest border-b-2
transition-all ${method === 'mobile' ? 'border-primary text-primary' : 'border-white/10 text-white/60'}
`}
          >

```

```

        Mobile
      </button>
      <button
        onClick={() => setMethod('email')}
        className={flex-1 py-2 text-xs font-bold uppercase tracking-widest border-b-2
transition-all ${method === 'email' ? 'border-primary text-primary' : 'border-white/10 text-white/60'}}
      >
        Email
      </button>
    </div>
  )}

  <div className="space-y-4">
    {mode === 'sp' ? (
      <div className="space-y-4">
        <div>
          <label className="label-bold mb-2 block">SP ID NO</label>
          <div className="flex gap-2">
            <div className="bg-white/5 border border-white/10 px-3 py-3 rounded-xl text-sm font-
bold text-white/70"><ShieldCheck className="w-4 h-4"/></div>
            <input
              type="text"
              value={email}
              onChange={(e) => setEmail(e.target.value)}
              className="flex-1 bg-white/5 border border-white/10 px-4 py-3 rounded-xl text-sm
focus:border-primary outline-none transition-colors font-mono tracking-wider"
              placeholder="SP-XXXXX"
            />
          </div>
        </div>
        <div>
          {error && (
            <p className="text-[10px] font-bold text-red-500 uppercase tracking-widest flex items-
center gap-2">
              <AlertTriangle className="w-3 h-3" /> {error}
            </p>
          )}
          <Button variant="primary" className="w-full" onClick={handleSPLogin}>
            Login as Provider
          </Button>
        </div>
      ) : method === 'mobile' ? (
        <div className="space-y-4">
          {mode === 'signup' && (
            <div>
              <label className="label-bold mb-2 block">Full Name</label>
              <input type="text" className="w-full bg-white/5 border border-white/10 px-4 py-3
rounded-xl text-sm focus:border-primary outline-none transition-colors" placeholder="Enter your full
name" />
            </div>
          )}
          <div>
            <label className="label-bold mb-2 block">Phone Number</label>
            <div className="flex gap-2">
              <div className="bg-white/5 border border-white/10 px-3 py-3 rounded-xl text-sm font-
bold text-white/70">+91</div>
              <input
                id="auth-tel"
                type="tel"
                value={phone}
                onChange={(e) => setPhone(e.target.value)}
                className="flex-1 bg-white/5 border border-white/10 px-4 py-3 rounded-xl text-sm
focus:border-primary outline-none transition-colors"
                placeholder="Enter mobile number"
              />
            </div>
          </div>
          <div>
            {otpSent && (
              <motion.div
                initial={{ opacity: 0, height: 0 }}
                animate={{ opacity: 1, height: 'auto' }}

```

```

class className="space-y-4"
>
  <div>
    <div className="flex justify-between items-center mb-2">
      <label className="label-bold block">One Time Password</label>
      <button
        onClick={handleResend}
        disabled={isResending}
        className="text-[10px] font-black text-primary uppercase tracking-widest
hover:underline disabled:opacity-50"
      >
        {isResending ? 'Sending Code...' : 'Resend Protocol'}
      </button>
    </div>
    <input
      type="text"
      value={otp}
      onChange={(e) => {
        setOtp(e.target.value);
        if (error) setError('');
      }}
      className={`w-full bg-white/5 border px-4 py-3 rounded-xl text-sm outline-none
transition-all tracking-[0.5em] font-mono text-center ${error ? 'border-red-500/50 bg-red-500/5' :
'border-white/10 focus:border-primary'}}`
      placeholder="XXXXXX"
    />
    <div
      {error && (
        <motion.p
          initial={{ opacity: 0, scale: 0.9 }}
          animate={{ opacity: 1, scale: 1 }}
          className="text-[10px] font-bold text-red-500 uppercase tracking-widest mt-2
flex items-center gap-2"
        >
          <AlertTriangle className="w-3 h-3" /> {error}
        </motion.p>
      )}
      <p className="text-[9px] text-white/40 mt-2 uppercase font-bold italic tracking-
wider">Verification required via Secure SMS</p>
    </div>
    </motion.div>
  </div>
  <div id="recaptcha-container"></div>
  <Button variant="primary" className="w-full" onClick={handleAction} disabled={loading}>
    {loading ? <Loader2 className="w-5 h-5 animate-spin" /> : (otpSent ? (mode ===
'login' ? 'Verify & Sign In' : 'Verify & Register') : 'Get Verification OTP')}
  </Button>
</div>
) : (
  <div className="space-y-4">
    {mode === 'signup' && (
      <div>
        <label className="label-bold mb-2 block">Full Name</label>
        <input type="text" className="w-full bg-white/5 border border-white/10 px-4 py-3
rounded-xl text-sm focus:border-primary outline-none transition-colors" placeholder="Enter your full
name" />
      </div>
    )}
    <div>
      <label className="label-bold mb-2 block">Email Address</label>
      <div className="flex gap-2">
        <div className="bg-white/5 border border-white/10 px-3 py-3 rounded-xl text-sm font-
bold text-white/70"><Mail className="w-4 h-4"/></div>
        <input
          type="email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          className="flex-1 bg-white/5 border border-white/10 px-4 py-3 rounded-xl text-sm
focus:border-primary outline-none transition-colors"
          placeholder="Enter your email"
        />
      </div>
    </div>
  </div>
) : (
  <div className="space-y-4">
    {mode === 'login' && (
      <div>
        <label className="label-bold mb-2 block">Phone Number</label>
        <input type="text" className="w-full bg-white/5 border border-white/10 px-4 py-3
rounded-xl text-sm focus:border-primary outline-none transition-colors" placeholder="Enter your
phone number" />
      </div>
    )}
    <div>
      <label className="label-bold mb-2 block">Password</label>
      <div className="flex gap-2">
        <div className="bg-white/5 border border-white/10 px-3 py-3 rounded-xl text-sm font-
bold text-white/70"><Lock className="w-4 h-4"/></div>
        <input
          type="password"
          value={password}
          onChange={(e) => setPassword(e.target.value)}
          className="flex-1 bg-white/5 border border-white/10 px-4 py-3 rounded-xl text-sm
focus:border-primary outline-none transition-colors"
          placeholder="Enter your password"
        />
      </div>
    </div>
  </div>
) : (
  <div className="space-y-4">
    {mode === 'forgot' && (
      <div>
        <label className="label-bold mb-2 block">Email Address</label>
        <input type="text" className="w-full bg-white/5 border border-white/10 px-4 py-3
rounded-xl text-sm focus:border-primary outline-none transition-colors" placeholder="Enter your
email" />
      </div>
    )}
    <div>
      <label className="label-bold mb-2 block">New Password</label>
      <div className="flex gap-2">
        <div className="bg-white/5 border border-white/10 px-3 py-3 rounded-xl text-sm font-
bold text-white/70"><Lock className="w-4 h-4"/></div>
        <input
          type="password"
          value={newPassword}
          onChange={(e) => setNewPassword(e.target.value)}
          className="flex-1 bg-white/5 border border-white/10 px-4 py-3 rounded-xl text-sm
focus:border-primary outline-none transition-colors"
          placeholder="Enter new password"
        />
      </div>
    </div>
  </div>
) : (
  <div className="space-y-4">
    {mode === 'verify' && (
      <div>
        <label className="label-bold mb-2 block">One Time Password</label>
        <input type="text" className="w-full bg-white/5 border border-white/10 px-4 py-3
rounded-xl text-sm focus:border-primary outline-none transition-colors" placeholder="Enter your
one time password" />
      </div>
    )}
    <div>
      <label className="label-bold mb-2 block">Verify Code</label>
      <div className="flex gap-2">
        <div className="bg-white/5 border border-white/10 px-3 py-3 rounded-xl text-sm font-
bold text-white/70"><Mail className="w-4 h-4"/></div>
        <input
          type="text"
          value={verifyCode}
          onChange={(e) => setVerifyCode(e.target.value)}
          className="flex-1 bg-white/5 border border-white/10 px-4 py-3 rounded-xl text-sm
focus:border-primary outline-none transition-colors"
          placeholder="Enter your verify code"
        />
      </div>
    </div>
  </div>
) : (
  <div className="space-y-4">
    {mode === 'register' && (
      <div>
        <label className="label-bold mb-2 block">Full Name</label>
        <input type="text" className="w-full bg-white/5 border border-white/10 px-4 py-3
rounded-xl text-sm focus:border-primary outline-none transition-colors" placeholder="Enter your
full name" />
      </div>
      <div>
        <label className="label-bold mb-2 block">Email Address</label>
        <input type="text" className="w-full bg-white/5 border border-white/10 px-4 py-3
rounded-xl text-sm focus:border-primary outline-none transition-colors" placeholder="Enter your
email" />
      </div>
      <div>
        <label className="label-bold mb-2 block">Phone Number</label>
        <input type="text" className="w-full bg-white/5 border border-white/10 px-4 py-3
rounded-xl text-sm focus:border-primary outline-none transition-colors" placeholder="Enter your
phone number" />
      </div>
      <div>
        <label className="label-bold mb-2 block">Password</label>
        <div className="flex gap-2">
          <div className="bg-white/5 border border-white/10 px-3 py-3 rounded-xl text-sm font-
bold text-white/70"><Lock className="w-4 h-4"/></div>
          <input
            type="password"
            value={password}
            onChange={(e) => setPassword(e.target.value)}
            className="flex-1 bg-white/5 border border-white/10 px-4 py-3 rounded-xl text-sm
focus:border-primary outline-none transition-colors"
            placeholder="Enter your password"
          />
        </div>
      </div>
      <div>
        <label className="label-bold mb-2 block">Confirm Password</label>
        <div className="flex gap-2">
          <div className="bg-white/5 border border-white/10 px-3 py-3 rounded-xl text-sm font-
bold text-white/70"><Lock className="w-4 h-4"/></div>
          <input
            type="password"
            value={confirmPassword}
            onChange={(e) => setConfirmPassword(e.target.value)}
            className="flex-1 bg-white/5 border border-white/10 px-4 py-3 rounded-xl text-sm
focus:border-primary outline-none transition-colors"
            placeholder="Enter your confirm password"
          />
        </div>
      </div>
    </div>
  </div>
) : (
  <div className="space-y-4">
    {mode === 'verify-otp' && (
      <div>
        <label className="label-bold mb-2 block">One Time Password</label>
        <input type="text" className="w-full bg-white/5 border border-white/10 px-4 py-3
rounded-xl text-sm focus:border-primary outline-none transition-colors" placeholder="Enter your
one time password" />
      </div>
    )}
    <div>
      <label className="label-bold mb-2 block">Verify Code</label>
      <div className="flex gap-2">
        <div className="bg-white/5 border border-white/10 px-3 py-3 rounded-xl text-sm font-
bold text-white/70"><Mail className="w-4 h-4"/></div>
        <input
          type="text"
          value={verifyCode}
          onChange={(e) => setVerifyCode(e.target.value)}
          className="flex-1 bg-white/5 border border-white/10 px-4 py-3 rounded-xl text-sm
focus:border-primary outline-none transition-colors"
          placeholder="Enter your verify code"
        />
      </div>
    </div>
  </div>
) : (
  <div className="space-y-4">
    {mode === 'forgot-otp' && (
      <div>
        <label className="label-bold mb-2 block">Email Address</label>
        <input type="text" className="w-full bg-white/5 border border-white/10 px-4 py-3
rounded-xl text-sm focus:border-primary outline-none transition-colors" placeholder="Enter your
email" />
      </div>
    )}
    <div>
      <label className="label-bold mb-2 block">New Password</label>
      <div className="flex gap-2">
        <div className="bg-white/5 border border-white/10 px-3 py-3 rounded-xl text-sm font-
bold text-white/70"><Lock className="w-4 h-4"/></div>
        <input
          type="password"
          value={newPassword}
          onChange={(e) => setNewPassword(e.target.value)}
          className="flex-1 bg-white/5 border border-white/10 px-4 py-3 rounded-xl text-sm
focus:border-primary outline-none transition-colors"
          placeholder="Enter new password"
        />
      </div>
    </div>
  </div>
) : (
  <div className="space-y-4">
    {mode === 'verify-otp' && (
      <div>
        <label className="label-bold mb-2 block">One Time Password</label>
        <input type="text" className="w-full bg-white/5 border border-white/10 px-4 py-3
rounded-xl text-sm focus:border-primary outline-none transition-colors" placeholder="Enter your
one time password" />
      </div>
    )}
    <div>
      <label className="label-bold mb-2 block">Verify Code</label>
      <div className="flex gap-2">
        <div className="bg-white/5 border border-white/10 px-3 py-3 rounded-xl text-sm font-
bold text-white/70"><Mail className="w-4 h-4"/></div>
        <input
          type="text"
          value={verifyCode}
          onChange={(e) => setVerifyCode(e.target.value)}
          className="flex-1 bg-white/5 border border-white/10 px-4 py-3 rounded-xl text-sm
focus:border-primary outline-none transition-colors"
          placeholder="Enter your verify code"
        />
      </div>
    </div>
  </div>
) : (
  <div className="space-y-4">
    {mode === 'forgot-otp' && (
      <div>
        <label className="label-bold mb-2 block">Email Address</label>
        <input type="text" className="w-full bg-white/5 border border-white/10 px-4 py-3
rounded-xl text-sm focus:border-primary outline-none transition-colors" placeholder="Enter your
email" />
      </div>
    )}
    <div>
      <label className="label-bold mb-2 block">New Password</label>
      <div className="flex gap-2">
        <div className="bg-white/5 border border-white/10 px-3 py-3 rounded-xl text-sm font-
bold text-white/70"><Lock className="w-4 h-4"/></div>
        <input
          type="password"
          value={newPassword}
          onChange={(e) => setNewPassword(e.target.value)}
          className="flex-1 bg-white/5
```

```

</div>
{otpSent && (
  <motion.div
    initial={{ opacity: 0, height: 0 }}
    animate={{ opacity: 1, height: 'auto' }}
    className="space-y-4"
  >
    <div>
      <div className="flex justify-between items-center mb-2">
        <label className="label-bold block">Email OTP</label>
      </div>
      <input
        type="text"
        value={otp}
        onChange={(e) => {
          setOtp(e.target.value);
          if (error) setError('');
        }}
        className={`w-full bg-white/5 border px-4 py-3 rounded-xl text-sm outline-none
transition-all tracking-[0.5em] font-mono text-center ${error ? 'border-red-500/50 bg-red-500/5' :
'border-white/10 focus:border-primary'}}`
        placeholder="XXXXXX"
      />
      {error && (
        <motion.p
          initial={{ opacity: 0, scale: 0.9 }}
          animate={{ opacity: 1, scale: 1 }}
          className="text-[10px] font-bold text-red-500 uppercase tracking-widest mt-2
flex items-center gap-2"
        >
          <AlertTriangle className="w-3 h-3" /> {error}
        </motion.p>
      )}
      <p className="text-[9px] text-white/40 mt-2 uppercase font-bold italic tracking-
wider">Verification sent to your email.</p>
    </div>
  </motion.div>
)}
<Button variant="primary" className="w-full" onClick={handleAction} disabled={loading}>
  {loading ? <Loader2 className="w-5 h-5 animate-spin" /> : (otpSent ? (mode ===
'login' ? 'Verify & Sign In' : 'Verify & Register') : 'Get Verification OTP')}
</Button>
</div>
)}
</div>

<div className="relative mt-6">
  <div className="absolute inset-0 flex items-center">
    <div className="w-full border-t border-white/10"></div>
  </div>
  <div className="relative flex justify-center text-xs">
    <span className="bg-surface px-2 text-white/60 uppercase tracking-widest font-bold">Or
continue with</span>
  </div>
</div>
<button
  onClick={handleGoogleLogin}
  disabled={loading}
  className="mt-6 w-full py-3 bg-white text-black rounded-xl text-sm font-bold uppercase
tracking-widest flex items-center justify-center gap-3 hover:bg-neutral-200 transition-colors
disabled:opacity-50"
>
  <svg viewBox="0 0 24 24" width="24" height="24" xmlns="http://www.w3.org/2000/svg"><path
d="M22.56 12.25c0-.78-.07-1.53-.2-2.25H12v4.26h5.92c-.26 1.37-1.04 2.53-2.21 3.31v2.77h3.57c2.08-1.92
3.28-4.74 3.28-8.09z" fill="#4285F4"/><path d="M12 23c2.97 0 5.46-.98
7.28-2.66l-3.57-2.77c-.98-.66-2.23 1.06-3.71 1.06-2.86 0-5.29-1.93-6.16-4.53H2.18v2.84C3.99 20.53 7.7
23 12 23z" fill="#34A853"/><path d="M5.84
14.09c-.22-.66-.35-1.36-.35-2.09s.13-1.43.35-2.09V7.07H2.18C1.43 8.55 1 10.22 1 12s.43 3.45 1.18
4.93l2.85-2.22.81-.62z" fill="#FBBC05"/><path d="M12 5.38c1.62 0 3.06.56 4.21 1.64l3.15-3.15C17.45
2.09 14.97 1 12 1 7.7 1 3.99 3.47 2.18 7.07l3.66 2.84c.87-2.6 3.3-4.53 6.16-4.53z" fill="#EA4335"/
><path d="M1 1h22v22H1z" fill="none"/></svg>

```

```

        Google
    </button>

    <div className="mt-8 pt-6 border-t border-white/5 text-center flex flex-col gap-3">
        {mode === 'login' ? (
            <p className="text-sm text-white/60">
                New to CAREVIA? <button onClick={() => setMode('signup')} className="text-primary font-
bold hover:underline">Create Account</button>
            </p>
        ) : (
            <p className="text-sm text-white/60">
                Already have an account? <button onClick={() => setMode('login')} className="text-
primary font-bold hover:underline">Sign In</button>
            </p>
        )}
        <div className="flex justify-center gap-4 pt-4">
            <button
                onClick={() => {
                    onClose();
                    onOpenAdmin();
                }}
                className="text-[10px] uppercase tracking-widest font-bold text-white/40 hover:text-
white transition-colors"
            >
                Admin Entrance
            </button>
            <div className="w-px h-4 bg-white/10" />
            <button
                onClick={() => {
                    setMode('sp');
                    setMethod('sp_id');
                }}
                className="text-[10px] uppercase tracking-widest font-bold text-primary opacity-60
hover:opacity-100 transition-all underline decoration-primary/30 underline-offset-4"
            >
                Provider Login
            </button>
        </div>
    </div>
</motion.div>
</div>
    );
};

const SubscriptionPlans = ({ onUpgrade }: any) => {
    const [selectedPlanIdx, setSelectedPlanIdx] = useState<number | null>(null);

    const plans = [
        { name: 'Monthly', price: 159, oldPrice: 299, discount: 99, period: '1 Month', popular: false },
        { name: 'Quarterly', price: 259, oldPrice: 399, discount: 199, period: '3 Months', popular: true },
        { name: 'Half-Yearly', price: 459, oldPrice: 699, discount: 399, period: '6 Months', popular:
false },
        { name: 'Annually', price: 659, oldPrice: 999, discount: 599, period: '12 Months', popular:
false },
    ];

    const benefits = [
        "10% discount on all bookings",
        "Priority assignment of services",
        "Dedicated AI BOT Assistant",
        "Priority Query resolution",
        "24/7 dedicated support",
        "Priority on all refunds",
        " 1500 off on emergency services",
        "Up to 1500 cashback on first 5 bookings",
        "Access to top-rated service providers",
        "Full body checkup at 1499 (6m/12m packs)"
    ];

    return (
        <section id="pricing" className="py-24 max-w-7xl mx-auto px-6 font-sans scroll-mt-24">

```

```

<SectionHeading
  sub="Premium Membership"
  title="Elevate Your Experience."
  desc="Join our subscription plans for exclusive benefits. First 50 customers get their first
month FREE!"
  centered
/>

<div className="flex overflow-x-auto pb-8 gap-6 mb-16 hide-scrollbar snap-x snap-mandatory
lg:grid lg:grid-cols-4">
  {plans.map((plan, i) => (
    <motion.div
      key={i}
      whileHover={{ y: -5 }}
      onClick={() => setSelectedPlanIdx(i)}
      className={`glass-card relative flex flex-col p-8 border cursor-pointer transition-all
flex-shrink-0 snap-start w-[85%] md:w-[45%] lg:w-auto ${selectedPlanIdx === i ? 'border-primary ring-2
ring-primary/40' : plan.popular ? 'border-primary ring-1 ring-primary/20' : 'border-white/5'}`}
      >
        {selectedPlanIdx === i && (
          <motion.div
            layoutId="shine"
            className="absolute inset-0 bg-gradient-to-tr from-transparent via-white/5 to-
transparent pointer-events-none"
            animate={{ x: ['100%', '-100%'] }}
            transition={{ duration: 1.5, repeat: Infinity, ease: 'linear' }}
          />
        )}
        {plan.popular && <span className="bg-primary text-surface text-[10px] font-black uppercase
tracking-widest px-3 py-1 rounded-full w-fit mb-4">Most Popular</span>}
        <div className="flex justify-between items-start mb-2">
          <h3 className="text-xl font-bold">{plan.name}</h3>
          {selectedPlanIdx === i && <CheckCircle2 className="w-5 h-5 text-primary" />}
        </div>
        <div className="flex items-baseline gap-2 mb-6">
          <span className="text-4xl font-bold text-primary"> ${plan.price}</span>
          <span className="text-sm text-white/50 line-through"> ${plan.oldPrice}</span>
        </div>
        <div className="mt-auto space-y-4">
          <div className="flex justify-between text-xs font-bold uppercase tracking-widest text-
white/60">
            <span>Validity</span>
            <span>{plan.period}</span>
          </div>
          <Button
            variant={(selectedPlanIdx === i || plan.popular) ? 'primary' : 'outline'}
            className="w-full py-4 text-xs z-10"
            onClick={(e: any) => {
              e.stopPropagation();
              onUpgrade(plan.name);
            }}
          >
            Select Plan
          </Button>
          <p className="text-[10px] text-center text-white/50 italic">Autopay enabled</p>
        </div>
      </motion.div>
    )}
  )}
</div>

<div className="glass-card p-12 bg-primary-container/20 mb-8">
  <h4 className="text-xl font-bold mb-8 uppercase tracking-widest text-primary flex items-center
gap-3">
    <TrendingUp className="w-5 h-5" /> Membership Benefits
  </h4>
  <div className="grid md:grid-cols-2 lg:grid-cols-3 gap-y-6 gap-x-12">
    {benefits.map((benefit, i) => (
      <div key={i} className="flex items-start gap-3">
        <CheckCircle2 className="w-5 h-5 text-primary flex-shrink-0 mt-0.5" />
        <span className="text-sm text-white/90">{benefit}</span>
      </div>
    )}
  </div>

```

```

        </div>
      )}}
    </div>
  </div>
  <p className="text-center text-xs text-white/50 italic mt-6 px-4 py-3 bg-white/5 rounded-2xl border border-white/5">
    * Please note: Final prices may differ according to your specific requirements, case complexity, and physical service location.
  </p>
</section>
);
};

const HowToUse = () => {
  const steps = [
    { title: "Authentication", desc: "Login or Register your account using mobile OTP or email to secure your profile.", icon: <User /> },
    { title: "Service Discovery", desc: "Browse through our specialized Elderly, Child, or Pet care sections to find what fits.", icon: <Stethoscope /> },
    { title: "Define Requirements", desc: "Submit a requirements form describing your unique problems for precision matching.", icon: <MessageSquare /> },
    { title: "Booking & Lock", icon: <Lock />, desc: "Confirm your selection, pay via CARVIA Wallet, and receive your Start/End OTP." }
  ];

  return (
    <section id="how-to-use" className="py-24 max-w-7xl mx-auto px-6 font-sans scroll-mt-24">
      <SectionHeading
        title="How to use"
        centered
      />
      <div className="grid md:grid-cols-4 gap-8 relative">
        <div className="hidden lg:block absolute top-[60px] left-[10%] right-[10%] h-px bg-gradient-to-r from-transparent via-primary/30 to-transparent" />
        {steps.map((s, i) => (
          <div key={i} className="relative text-center group">
            <div className="w-16 h-16 bg-white/5 border border-white/10 rounded-2xl flex items-center justify-center text-primary mx-auto mb-6 transition-all group-hover:bg-primary group-hover:text-surface group-hover:scale-110">
              {s.icon}
            </div>
            <h4 className="text-sm font-bold uppercase tracking-widest mb-3">{i + 1}. {s.title}</h4>
            <p className="text-xs text-white/60 leading-relaxed px-4">{s.desc}</p>
          </div>
        ))}
      </div>
    </section>
  );
};

const AboutSection = () => (
  <section id="about" className="py-32 bg-surface/50 monolith-grid font-sans overflow-hidden scroll-mt-24">
    <div className="max-w-7xl mx-auto px-6 grid lg:grid-cols-2 gap-20 items-center">
      <div className="relative">
        <div className="absolute -top-20 -left-20 w-64 h-64 bg-primary/20 rounded-full blur-[100px]" />
        <SectionHeading
          sub="The Legacy"
          title="What is CAREVIA?"
          desc="CAREVIA is a trusted care management service. We help families find the best caregivers for their loved ones."
        />
        <div className="space-y-6">
          <p className="text-sm text-white/80 leading-loose">We act as the bridge between caregivers and families. Our focus is on privacy and quality. Whether it's elderly care or child care, we handle everything so you don't have to worry.</p>
          <div className="grid sm:grid-cols-2 gap-4">
            {[
              { t: '10-Step Vetting', d: 'Rigorous vetting for every provider.' },
              { t: '24/7 SOS', d: 'Real-time emergency dispatch hub.' },
            ]}
          </div>
        </div>
      </div>
    </div>
  >

```

```

        { t: 'Global Standards', d: 'HIPAA & GDPR data compliance.' },
        { t: 'Carvia Wallet', d: 'Secure, instant financial management.' }
    ].map((item, i) => (
        <div key={i} className="p-4 bg-white/5 rounded-2xl border border-white/5">
            <p className="text-[10px] font-bold text-primary uppercase tracking-widest
mb-1">{item.t}</p>
            <p className="text-[10px] text-white/50">{item.d}</p>
        </div>
    ))
</div>
</div>
<div className="glass-card aspect-square relative flex items-center justify-center overflow-
hidden">
    
    <div className="absolute inset-0 flex items-center justify-center p-12">
        <div className="text-center">
            <p className="text-4xl font-black mb-4">"Care is not just a service; it's a
commitment."</p>
            <p className="text-primary/70 text-[10px] uppercase font-bold tracking-[0.4em]">Ashvin
Galav // Founder</p>
        </div>
    </div>
</div>
</div>
</section>
);

```

```

const ContactSection = () => {
    const [formData, setFormData] = useState({ name: '', phone: '', query: '' });
    const [isSubmitting, setIsSubmitting] = useState(false);
    const [aiResponse, setAiResponse] = useState('');

```

```

    const handleSubmit = async () => {
        if (!formData.query) return;
        setIsSubmitting(true);
        setAiResponse('');
        try {
            const res = await fetch('/api/inquiries', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({
                    name: formData.name,
                    phone: formData.phone,
                    query: formData.query,
                    category: 'General Inquiry'
                })
            });
            const data = await res.json();
            if (data.success) {
                setAiResponse(data.inquiry.aiResponse);
                setFormData({ name: '', phone: '', query: '' });
            } else {
                setAiResponse("Failed to send inquiry. Please try again.");
            }
        } catch (error) {
            setAiResponse("An error occurred. Please try again.");
        } finally {
            setIsSubmitting(false);
        }
    };

```

```

    return (
        <section id="contact" className="py-24 max-w-7xl mx-auto px-6 font-sans scroll-mt-24">
            <SectionHeading
                title="contact us"
            />

```

```

        centered
    />
    <div className="grid lg:grid-cols-3 gap-8">
        <div className="lg:col-span-1 space-y-4">
            <div className="glass-card p-8 border-white/5">
                <Mail className="text-primary mb-4" />
                <h4 className="text-xs font-bold uppercase tracking-widest mb-2">Email Us</h4>
                <p className="text-lg font-bold">care@carevia.app</p>
            </div>
            <div className="glass-card p-8 border-white/5">
                <Phone className="text-primary mb-4" />
                <h4 className="text-xs font-bold uppercase tracking-widest mb-2">Helpline Number</h4>
                <p className="text-lg font-bold">+91 {import.meta.env.VITE_ADMIN_PHONE || 'XXXXX XXXXX'}</p>
            </div>
        </div>
        <div className="lg:col-span-2 glass-card p-12 bg-primary/5 relative overflow-hidden">
            <div className="absolute top-0 right-0 w-32 h-32 bg-primary/10 rounded-full blur-[60px]" />
            <h4 className="text-xl font-bold mb-8 uppercase tracking-widest text-primary flex items-center gap-3">
                <MessageSquare className="w-5 h-5" /> Inquiry Chat Box
            </h4>
            <div className="space-y-6">
                <div className="grid md:grid-cols-2 gap-6">
                    <div>
                        <label className="label-bold mb-2 block">Full Name</label>
                        <input
                            value={formData.name}
                            onChange={e => setFormData({ ...formData, name: e.target.value })}
                            className="w-full bg-white/5 border border-white/10 px-6 py-4 rounded-2xl text-sm outline-none focus:border-primary transition-colors"
                            placeholder="Your Name"
                        />
                    </div>
                    <div>
                        <label className="label-bold mb-2 block">Mobile No</label>
                        <input
                            value={formData.phone}
                            onChange={e => setFormData({ ...formData, phone: e.target.value })}
                            className="w-full bg-white/5 border border-white/10 px-6 py-4 rounded-2xl text-sm outline-none focus:border-primary transition-colors"
                            placeholder="+91 XXXX..."
                        />
                    </div>
                </div>
                <div>
                    <label className="label-bold mb-2 block">Describe your problem/request</label>
                    <textarea
                        rows={4}
                        value={formData.query}
                        onChange={e => setFormData({ ...formData, query: e.target.value })}
                        className="w-full bg-white/5 border border-white/10 px-6 py-4 rounded-2xl text-sm outline-none focus:border-primary transition-colors resize-none"
                        placeholder="How can we help you today?"
                    />
                </div>
                {aiResponse && (
                    <div className="p-4 bg-primary/10 border border-primary/20 rounded-xl text-sm text-primary-foreground">
                        <p className="font-bold mb-1">AI Assistant Response:</p>
                        <p>{aiResponse}</p>
                    </div>
                )}
                <Button
                    variant="primary"
                    className="w-full uppercase tracking-[0.2em] text-xs py-5"
                    onClick={handleSubmit}
                    disabled={isSubmitting || !formData.query}
                >
                    {isSubmitting ? 'Processing...' : 'Send Inquiry to Coordinator'}
                </Button>
            </div>
        </div>
    </div>

```

```

        </Button>
      </div>
    </div>
  </div>
</section>
);
};
const AIBotChat = ({ isOpen, onClose, isPremium, sessionId, onLog }: any) => {
  const [messages, setMessages] = useState<any[]>([]);
  const [input, setInput] = useState('');
  const [isTyping, setIsTyping] = useState(false);

  useEffect(() => {
    if (isOpen && messages.length === 0 && isPremium) {
      setMessages([{ role: 'model', text: "Hello! I am your CAREVIA Elite Assistant. As a premium member, how can I assist you with your care requirements today?" }]);
    }
  }, [isOpen, isPremium]);

  const handleSend = async () => {
    if (!input.trim() || isTyping) return;

    const userMsg = input.trim();
    setInput('');
    setMessages(prev => [...prev, { role: 'user', text: userMsg }]);
    setIsTyping(true);

    // Initial Log
    onLog?.({
      action: `AI Bot: User Prompt Received`,
      admin: "AI BOT",
      time: new Date().toISOString().replace('T', ' ').substring(0, 16),
      type: 'ai',
      details: `User Prompt: ${userMsg}`,
      sessionId: sessionId,
      userId: "Client_Anonymous",
      status: 'PROCESSING'
    });

    try {
      // internal AI processing step log
      onLog?.({
        action: `AI Bot: System Initialization`,
        admin: "AI BOT",
        time: new Date().toISOString().replace('T', ' ').substring(0, 16),
        type: 'ai',
        details: `Config: Model="gemini-2.5-flash", Context="CAREVIA_Elite", Privacy="Premium_Tier".`,
        sessionId: sessionId,
        userId: "Client_Anonymous",
        status: 'INTERNAL'
      });

      const ai = new GoogleGenAI({ apiKey: process.env.GEMINI_API_KEY });
      const response = await ai.models.generateContent({
        model: "gemini-2.5-flash",
        contents: [
          { role: 'user', parts: [{ text: `Context: You are the CAREVIA AI Bot, a concierge health assistant for a premium healthcare service. Be sophisticated, professional, and helpful. Only answer questions related to healthcare, caregiving, domestic help, or CAREVIA services. User Message: ${userMsg}` } ] }
        ],
        config: {
          systemInstruction: "You are the CAREVIA Elite Assistant. Professionally guide users through healthcare and concierge services provided by CAREVIA."
        }
      });

      const aiResponseText = response.text || "I apologize, I didn't quite catch that. Could you rephrase?";
      setMessages(prev => [...prev, { role: 'model', text: aiResponseText }]);
    }
  };
};

```

```

// Success Log
onLog?.{
  action: `AI Bot: Response Dispatched`,
  admin: "AI BOT",
  time: new Date().toISOString().replace('T', ' ').substring(0, 16),
  type: 'ai',
  details: `Response: ${aiResponseText}`,
  sessionId: sessionId,
  userId: "Client_Anonymous",
  status: 'SUCCESS'
});
} catch (error) {
  const errorMsg = error instanceof Error ? error.message : "Unknown connectivity disruption";
  setMessages(prev => [...prev, { role: 'model', text: `I'm having trouble connecting to the nexus
right now. Details: ${errorMsg}` }]);

  // Error Log
  onLog?.{
    action: `AI Bot: Critical Failure`,
    admin: "AI BOT",
    time: new Date().toISOString().replace('T', ' ').substring(0, 16),
    type: 'ai',
    details: `Fault: ${errorMsg}`,
    sessionId: sessionId,
    userId: "Client_Anonymous",
    status: 'ERROR'
  });
} finally {
  setIsTyping(false);
}
};

if (!isOpen) return null;

return (
  <div className="fixed bottom-24 right-8 z-[160] w-[400px] max-h-[600px] flex flex-col font-sans">
    <motion.div
      initial={{ opacity: 0, scale: 0.95, y: 20 }}
      animate={{ opacity: 1, scale: 1, y: 0 }}
      className="glass-card flex-1 flex flex-col overflow-hidden border-primary/20 shadow-2xl"
    >
      <div className="bg-primary/10 p-6 border-b border-white/5 flex justify-between items-center bg-
gradient-to-r from-primary/20 to-transparent">
        <div className="flex items-center gap-3">
          <div className="w-10 h-10 bg-primary rounded-xl flex items-center justify-center text-
surface shadow-lg shadow-primary/20">
            <Bot className="w-6 h-6" />
          </div>
          <div>
            <h3 className="text-sm font-bold uppercase tracking-widest text-primary">Elite
Assistant</h3>
            <div className="flex items-center gap-1.5">
              <div className="w-1.5 h-1.5 bg-green-500 rounded-full animate-pulse" />
              <span className="text-[10px] text-white/60 uppercase font-black tracking-
widest">Active Member Hub</span>
            </div>
          </div>
          <div>
            <button onClick={onClose} className="p-2 hover:bg-white/10 rounded-full transition-colors
text-white/70"><X /></button>
          </div>
        </div>

      <div className="flex-1 p-10 flex flex-col items-center justify-center text-center bg-
black/40">
        <div className="w-20 h-20 border border-white/10 rounded-3xl flex items-center justify-
center mb-6 text-white/40">
          <Lock className="w-10 h-10" />
        </div>
        <h4 className="text-xl font-bold mb-3 uppercase tracking-tight">Access Restricted</h4>
        <p className="text-sm text-white/60 mb-8 leading-relaxed">

```

```

        The Private AI Concierge is an exclusive benefit for <span className="text-primary font-
bold italic">Premium Subscribers.</span>
    </p>
    <Button variant="primary" className="w-full text-[10px] py-4" onClick={() => {
        onClose();
        document.getElementById('pricing')?.scrollIntoView({ behavior: 'smooth' });
    }}>
        Unlock Premium Now
    </Button>
</div>
) : (
    <>
        <div className="flex-1 overflow-y-auto p-6 space-y-4 custom-scrollbar bg-black/20">
            {messages.map((m, i) => (
                <div key={i} className={`flex ${m.role === 'user' ? 'justify-end' : 'justify-start'}`}>
                    <div className={`max-w-[85%] p-4 rounded-2xl text-sm ${m.role === 'user' ? 'bg-
primary/20 border border-primary/20 text-white rounded-br-none' : 'bg-white/5 border border-white/10
text-white/90 rounded-bl-none'}`}>
                        {m.text}
                    </div>
                </div>
            ))}
            {isTyping && (
                <div className="flex justify-start">
                    <div className="bg-white/5 border border-white/10 p-4 rounded-2xl rounded-bl-none">
                        <Loader2 className="w-4 h-4 text-primary animate-spin" />
                    </div>
                </div>
            )}
        </div>

        <div className="p-4 border-t border-white/5 bg-surface/50">
            <div className="relative flex items-center">
                <input
                    value={input}
                    onChange={(e) => setInput(e.target.value)}
                    onKeyDown={(e) => e.key === 'Enter' && handleSend()}
                    className="w-full bg-white/5 border border-white/10 px-6 py-4 rounded-2xl text-sm
outline-none focus:border-primary transition-colors pr-14"
                    placeholder="Consult with AI Assistant..."
                />
                <button
                    onClick={handleSend}
                    className="absolute right-2 p-3 bg-primary text-surface rounded-xl hover:bg-
primary-hover transition-all active:scale-95"
                >
                    <Send className="w-4 h-4" />
                </button>
            </div>
        </div>
    </>
    </motion.div>
</div>
);
};

const ServiceCatalog = ({ activeTab, setActiveTab, onBook, userLocation, setUserLocation }:
{ activeTab: string, setActiveTab: (id: string) => void, onBook: (name: string) => void, userLocation:
string, setUserLocation: (loc: string) => void }) => {
    const providers = MOCK_PROVIDERS.filter(p => p.category === activeTab && p.location ===
userLocation);
    const activeCategory = CATEGORIES.find(c => c.id === activeTab);

    return (
        <section id="services" className="py-24 bg-surface/50 monolith-grid font-sans scroll-mt-24">
            <div className="max-w-7xl mx-auto px-6">
                <div className="flex flex-col md:flex-row justify-between items-start md:items-center mb-16
gap-8">
                    <div>

```

```

        <SectionHeading
            title="specialized services for every life"
        />
        <p className="text-xs text-white/60 uppercase tracking-widest mt-4 max-w-xl leading-
relaxed">
            {activeCategory?.description}
        </p>
    </div>

    <div className="flex items-center gap-4 bg-white/5 p-4 rounded-3xl border border-white/10 w-
full md:w-auto self-start">
        <MapPin className="text-primary w-5 h-5 flex-shrink-0" />
        <div className="flex flex-col">
            <span className="text-[8px] font-bold text-white/50 uppercase tracking-
[0.2em]">Location</span>
            <select
                value={userLocation}
                onChange={(e) => setUserLocation(e.target.value)}
                className="bg-transparent text-xs font-bold text-white outline-none cursor-pointer
pr-4"
            >
                <option value="" disabled hidden>Select your city</option>
                {LOCATIONS.map(loc => <option key={loc} value={loc} className="bg-surface">{loc}</
option>)}
            </select>
        </div>
    </div>

    <div className="flex overflow-x-auto pb-4 gap-4 mb-12 hide-scrollbar snap-x snap-mandatory
lg:flex-wrap">
        {CATEGORIES.map(cat => (
            <button
                key={cat.id}
                onClick={() => setActiveTab(cat.id)}
                className={`flex-shrink-0 snap-start flex items-center gap-3 px-6 py-4 rounded-2xl
transition-all ${activeTab === cat.id ? 'bg-primary text-surface font-black shadow-
[0_0_20px_rgba(212,175,55,0.3)]' : 'bg-white/10 text-white hover:bg-white/20'}`}
            >
                {cat.icon}
                <span className="uppercase tracking-widest text-xs font-bold">{cat.name}</span>
                {cat.id !== 'elderly' && cat.id !== 'helpers' && !cat.isFull && <span className="text-
[8px] bg-white/10 px-2 py-0.5 rounded-full ml-2 opacity-50">Coming Soon</span>}
            </button>
        ))}
    </div>

    <div className="glass-card min-h-[400px]">
        <div className="grid md:grid-cols-2 lg:grid-cols-3 gap-12">
            {activeCategory?.sections?.map((section, idx) => (
                <div key={idx}>
                    <h5 className="text-primary font-black uppercase tracking-[0.2em] text-[10px] mb-4
pb-2 border-b border-primary/20">{section.name}</h5>
                    <ul className="space-y-4">
                        {section.items.map((item, i) => (
                            <li key={i} className="flex items-center justify-between group">
                                <div className="flex items-center gap-3 text-sm text-white/80 group-hover:text-
white transition-colors">
                                    <div className="w-1.5 h-1.5 bg-primary rounded-full shadow-
[0_0_8px_rgba(45,212,191,0.5)]" />
                                    {item}
                                </div>
                                <button
                                    onClick={() => onBook(item)}
                                    className="px-3 py-1.5 bg-primary/10 hover:bg-primary text-primary hover:text-
surface text-[9px] font-bold uppercase tracking-widest rounded-lg transition-all opacity-0 group-
hover:opacity-100"
                                >
                                    Book
                                </button>
                            </li>
                        ))}
                    </ul>
                </div>
            ))}
        </div>
    </div>

```

```

        </ul>
      </div>
    )}}
    <div className="bg-primary/5 rounded-3xl p-8 flex flex-col justify-between border border-
primary/10">
      <div>
        <h5 className="text-[10px] font-black uppercase tracking-widest text-primary
mb-6">Service Tiers</h5>
        <div className="space-y-6 mb-8">
          {activeCategory?.pricing?.map((p: any, i: number) => (
            <div key={i} className="p-4 bg-white/5 rounded-2xl border border-white/5
hover:border-primary/20 transition-all">
              <div className="flex justify-between items-center mb-2">
                <span className="text-[10px] font-black uppercase tracking-widest text-
white">{p.tier}</span>
                <span className="text-[11px] font-bold text-primary">{p.price}</span>
              </div>
              <div className="flex flex-wrap gap-2">
                {p.features.map((f: string, j: number) => (
                  <span key={j} className="text-[8px] bg-white/5 text-white/60 px-2
py-0.5 rounded tracking-tighter">{f}</span>
                ))}
              </div>
            </div>
          ))}
        </div>
        <Button
          onClick={() => onBook(activeCategory?.name || 'Service')}
          variant="primary"
          className="w-full text-xs"
        >
          Request Consultation
        </Button>
      </div>
    </div>
  </div>

  <div className="mt-20">
    <div className="flex items-center justify-between mb-10">
      <div className="flex items-center gap-4">
        <div className="w-10 h-10 bg-primary/20 rounded-xl flex items-center justify-center
text-primary">
          <ShieldCheck className="w-5 h-5" />
        </div>
        <div>
          <h4 className="text-xs font-black uppercase tracking-[0.3em] text-primary">Vetted
Professionals</h4>
          <p className="text-[10px] text-white/50 uppercase font-bold tracking-widest
mt-1">Direct from {userLocation}</p>
        </div>
      </div>
      <div className="h-px flex-1 mx-8 bg-white/5 hidden md:block" />
      <div className="text-[10px] font-black uppercase text-white/40 tracking-widest">
        {providers.length} Verified Partner{providers.length !== 1 ? 's' : ''} Online
      </div>
    </div>

    <div className="grid md:grid-cols-2 lg:grid-cols-3 gap-8">
      {providers.length > 0 ? providers.map(p => (
        <motion.div
          key={p.id}
          whileHover={{ y: -8, scale: 1.02 }}
          className="glass-card p-0 flex flex-col border border-white/5 relative group
transition-all hover:border-primary/30 overflow-hidden"
        >
          <div className="relative h-48 bg-gradient-to-br from-primary/10 to-transparent">
            <img
              src={(p as any).avatar}
              alt={p.name}
              className="w-full h-full object-cover mix-blend-overlay opacity-60 grayscale

```

```

group-hover:grayscale-0 group-hover:opacity-100 transition-all duration-700"
    />
    <div className="absolute top-4 right-4 flex flex-col items-end gap-2">
      <div className="flex items-center gap-1 bg-surface/90 backdrop-blur-md px-3
py-1.5 rounded-full border-white/10">
        <Star className="w-3 h-3 text-primary fill-primary" />
        <span className="text-[11px] font-black text-white">{p.rating}</span>
      </div>
    </div>
    <div className="absolute bottom-4 left-4">
      <span className="text-[8px] font-black text-white uppercase tracking-widest
bg-primary px-3 py-1 rounded-full shadow-2xl">Verified Expert</span>
    </div>
  </div>

  <div className="p-8">
    <div className="mb-6">
      <h5 className="text-lg font-bold group-hover:text-primary transition-
colors">{p.name}</h5>
      <p className="text-[10px] text-white/50 italic mt-1 line-clamp-2">{(p as
any).bio}</p>
    </div>

    <div className="space-y-4">
      <div className="flex items-center gap-2">
        <GraduationCap className="w-3 h-3 text-primary" />
        <p className="text-[9px] text-white/70 uppercase tracking-widest font-
black">{p.education}</p>
      </div>
      <div className="flex flex-wrap gap-2">
        {(p as any).specialties?.map((s: string, i: number) => (
          <span key={i} className="text-[8px] bg-primary/10 text-primary px-2
py-0.5 rounded font-bold uppercase tracking-normal">#{s}</span>
        ))}
      </div>
    </div>

    <div className="flex items-center justify-between mt-8 pt-6 border-t border-
white/5">
      <div className="flex flex-col">
        <span className="text-[8px] font-bold text-white/40 uppercase tracking-
widest mb-1">Experience</span>
        <span className="text-[10px] font-black text-white/80
uppercase">{p.experience}</span>
      </div>
      <Button variant="outline" className="px-5 py-2.5 text-[9px] uppercase
tracking-widest border-primary/40 text-primary hover:bg-primary hover:text-surface" onClick={() =>
onBook(p.name)}>Hire Now</Button>
    </div>
  </div>
</motion.div>
)) : (
  <div className="col-span-full py-20 text-center glass-card border-dashed border-
white/10 bg-white/[0.01]">
    <div className="w-16 h-16 bg-white/5 rounded-full flex items-center justify-center
mx-auto mb-6">
      <MapPin className="w-8 h-8 text-white/40" />
    </div>
    <h5 className="text-white/70 font-bold uppercase tracking-widest mb-2">Expansion in
Progress</h5>
    <p className="text-xs text-white/40 italic max-w-sm mx-auto">We are currently
vetting elite partners in {userLocation} for this category. Check back shortly or explore our AI
assistant.</p>
  </div>
)}
</div>
</div>
</section>
);
};

```

```

const LegalModal = ({ isOpen, onClose }: any) => {
  if (!isOpen) return null;
  return (
    <div className="fixed inset-0 z-[250] flex items-center justify-center p-6 font-sans">
      <div className="absolute inset-0 bg-black/80 backdrop-blur-md" onClick={onClose} />
      <div className="glass-card w-full max-w-4xl max-h-[80vh] overflow-y-auto relative z-10 p-12">
        <div className="flex justify-between items-center mb-8">
          <SectionHeading sub="Compliance" title="Legal Terms & Policy" desc="Last Updated: April
2025" />
          <button onClick={onClose} className="p-4 bg-white/5 rounded-full hover:bg-white/10"><X /></
button>
        </div>
        <div className="space-y-8 text-white/80 text-sm leading-loose font-light">
          <section>
            <h4 className="text-white font-bold mb-4 uppercase tracking-widest">1. Professional Care
Commitment</h4>
            <p>CAREVIA Group acts as a premium concierge management platform. We vetting each service
provider with a rigorous 10-step protocol, including background checks, credential verification, and
interpersonal skills assessment. However, clinical liability resides with the individual medical
professionals or registered agencies.</p>
          </section>
          <section>
            <h4 className="text-white font-bold mb-4 uppercase tracking-widest">2. Privacy Protocol
(GDPR/HIPAA Compliance)</h4>
            <p>Your data is managed with Tier 1 encryption. Personal identifiable information (PII) is
only shared with assigned care coordinators and service providers during the active service window.</p>
          </section>
          <section>
            <h4 className="text-white font-bold mb-4 uppercase tracking-widest">3. Billing & Refunds</
h4>
            <p>Premium subscriptions are billed monthly/annually. Cancellations take effect at the end
of the billing cycle. Priority refunds are processed within 2 hours to the CARVIA Wallet.</p>
          </section>
          <section>
            <h4 className="text-white font-bold mb-4 uppercase tracking-widest">4. SOS & Emergency
Response</h4>
            <p>The SOS feature is a secondary alert system. Users are advised to contact local
emergency services (102/100) directly for life-critical situations.</p>
          </section>
        </div>
      </div>
    </div>
  );
}

const ActiveServiceTracker = ({ bookings = [] }: { bookings?: any[] }) => {
  const activeBooking = bookings.find(b => b.status === "CONFIRMED" || b.status === "IN_PROGRESS") ||
bookings[0];

  if (!activeBooking) return null;

  return (
    <motion.div
      initial={{ opacity: 0, x: 50 }}
      animate={{ opacity: 1, x: 0 }}
      className="glass p-6 rounded-3xl border-primary/20 bg-primary/5 flex flex-col sm:flex-row items-
center gap-6"
    >
      <div className="flex items-center gap-6 w-full sm:w-auto">
        <div className="w-12 h-12 bg-primary/20 rounded-2xl flex items-center justify-center text-
primary flex-shrink-0">
          <Clock className="animate-spin-slow" />
        </div>
        <div className="max-w-[150px]">
          <p className="text-[10px] font-bold text-primary uppercase tracking-[0.2em]
truncate">Active: {activeBooking.service}</p>
          <p className="text-[10px] font-bold opacity-60">SP: {activeBooking.providerName ||
'Assigning...'}</p>
        </div>
      </div>
    </div>
  );
}

```

```

        <div className="w-full sm:ml-auto flex items-center justify-between sm:justify-end gap-4 bg-
surface/50 p-3 rounded-2xl border border-white/5">
            <div className="flex flex-col">
                <span className="text-[8px] font-bold text-white/50 uppercase tracking-widest">Auth Token</
span>
                <span className="text-sm font-mono font-bold text-primary tracking-
widest">{activeBooking.otp || '****'}</span>
            </div>
            <div className="w-px h-8 bg-white/10" />
            <div className="flex flex-col">
                <span className="text-[8px] font-bold text-white/50 uppercase tracking-widest">Status</
span>
                <span className="text-[10px] font-black uppercase text-white/80 tracking-
tighter">{activeBooking.status}</span>
            </div>
            <Info className="w-4 h-4 text-white/40" />
        </div>
    </motion.div>
    );
};

const ProviderDashboard = ({ providerData, onUpdateProvider, onClose, bookings = [], onAction,
isSpectator = false, masterOtp, onLogout }: any) => {
    const [activeTab, setActiveTab] = useState('profile');
    const [isUploading, setIsUploading] = useState(false);
    const [notification, setNotification] = useState<string | null>(null);
    const [otpInputs, setOtpInputs] = useState<Record<string, string>>({});

    const handleAction = (bookingId: string, action: 'ACCEPT' | 'DECLINE' | 'START_SERVICE' |
'END_SERVICE') => {
        if (action === 'START_SERVICE' || action === 'END_SERVICE') {
            const book = bookings.find((b: any) => b.id === bookingId);
            const enteredOtp = otpInputs[bookingId];
            const isValid = (enteredOtp === (book as any)?.otp || masterOtp === enteredOtp) && enteredOtp !
== undefined;

            if (!isValid && !isSpectator) {
                setNotification('ERR: INVALID AUTHENTICATION TOKEN (OTP)');
                setTimeout(() => setNotification(null), 3000);
                return;
            }
        }

        onAction(bookingId, action);

        const msg = action === 'ACCEPT' ? 'Accepted' : action === 'DECLINE' ? 'Declined' : action ===
'START_SERVICE' ? 'Started' : 'Completed';
        setNotification(`Service ${bookingId} ${msg}`);
        setTimeout(() => setNotification(null), 3000);
    };

    const handleUpload = () => {
        setIsUploading(true);
        setTimeout(() => {
            setIsUploading(false);
            onUpdateProvider({ ...providerData, isDocsUploaded: true });
        }, 2000);
    };

    return (
        <div className="fixed inset-0 z-[300] bg-surface flex flex-col font-sans overflow-hidden">
            </* Header */>
            <div className={`border-b border-white/5 p-6 flex justify-between items-center ${isSpectator ?
'bg-red-500/10 border-red-500/20' : 'bg-card'}`>
                <div className="flex items-center gap-4">
                    {isSpectator ? <Eye className="text-red-500 animate-pulse" /> : <BriefcaseMedical
className="text-primary" />}
                </div>
                <h2 className="text-xl font-bold uppercase tracking-tight">{isSpectator ? `Shadowing:
${providerData.name}` : 'Partner Dashboard'}</h2>
                <div className="flex items-center gap-2">

```

```

        <span className="text-[8px] font-black uppercase text-muted tracking-widest">ID:
{providerData?.spId || 'SP-NEW-001'}</span>
        {isSpectator && (
            <span className="text-[8px] font-black uppercase bg-red-500 text-white px-3 py-1
rounded-full animate-pulse tracking-widest ml-2">Live Surveillance</span>
        )}
        {providerData?.isVerified && !isSpectator && (
            <span className="flex items-center gap-1 text-[8px] font-black uppercase text-
primary tracking-widest bg-primary/10 px-2 py-0.5 rounded-full">
                <ShieldCheck className="w-2.5 h-2.5" /> Verified
            </span>
        )}
    </div>
</div>
</div>
<div className="flex items-center gap-4">
    <button onClick={onClose} className="p-2 hover:bg-white/10 rounded-full text-muted"><X /></
button>
</div>
</div>

<div className="flex flex-1 overflow-hidden">
    {/* Sidebar */}
    <div className="w-64 border-r border-white/5 p-6 flex flex-col gap-2 bg-card/40">
        <button
            onClick={() => setActiveTab('profile')}
            className={`flex items-center gap-3 w-full p-3 rounded-xl text-xs font-bold uppercase
tracking-widest transition-all ${activeTab === 'profile' ? 'bg-primary/10 text-primary' : 'hover:bg-
white/5 text-muted'}`}>
            >
            <User className="w-4 h-4" /> My Profile
        </button>
        <button
            onClick={() => setActiveTab('bookings')}
            className={`flex items-center gap-3 w-full p-3 rounded-xl text-xs font-bold uppercase
tracking-widest transition-all ${activeTab === 'bookings' ? 'bg-primary/10 text-primary' : 'hover:bg-
white/5 text-muted'}`}>
            >
            <Calendar className="w-4 h-4" /> Bookings {bookings.filter(b => b.status ===
"PENDING").length > 0 && <span className="ml-auto w-2 h-2 bg-primary rounded-full animate-pulse" />}
        </button>
        <button
            onClick={() => setActiveTab('earnings')}
            className={`flex items-center gap-3 w-full p-3 rounded-xl text-xs font-bold uppercase
tracking-widest transition-all ${activeTab === 'earnings' ? 'bg-primary/10 text-primary' : 'hover:bg-
white/5 text-muted'}`}>
            >
            <Wallet className="w-4 h-4" /> Earnings
        </button>
        <button
            onClick={() => setActiveTab('bank')}
            className={`flex items-center gap-3 w-full p-3 rounded-xl text-xs font-bold uppercase
tracking-widest transition-all ${activeTab === 'bank' ? 'bg-primary/10 text-primary' : 'hover:bg-
white/5 text-muted'}`}>
            >
            <BriefcaseMedical className="w-4 h-4" /> Bank Details
        </button>

        <div className="mt-auto pt-6 border-t border-white/5">
            <button onClick={onLogout || onClose} className="flex items-center gap-3 w-full p-3 text-
red-500/50 hover:text-red-500 text-xs font-bold uppercase tracking-widest">
                <LogOut className="w-4 h-4" /> Logout
            </button>
        </div>
    </div>

    {/* Content */}
    <div className="flex-1 p-10 overflow-y-auto bg-black/20">
        {notification && (
            <motion.div initial={{ opacity: 0, y: -20 }} animate={{ opacity: 1, y: 0 }}
className="mb-8 p-4 bg-primary/10 border border-primary/20 rounded-2xl text-xs font-bold text-primary

```

```

uppercase tracking-widest">
    {notification}
  </motion.div>
)}

{activeTab === 'profile' && (
  <div className="max-w-4xl space-y-12 animate-in fade-in slide-in-from-bottom-4
duration-500">
    <SectionHeading title="Provider Identity" sub="Credentials & Verification" />

    <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
      <div className="glass-card space-y-6">
        <div className="flex items-center gap-4">
          <div className="w-16 h-16 rounded-2xl bg-white/5 border border-white/10 flex
items-center justify-center">
            <Upload className="w-6 h-6 text-muted" />
          </div>
          <div>
            <h4 className="text-sm font-bold uppercase tracking-widest">Documents
Status</h4>

            <p className="text-[10px] text-muted uppercase mt-1">
              {providerData?.isDocsUploaded ? 'All files received' : 'Action
Required: Upload ID'}
            </p>
          </div>
        </div>

        {!providerData?.isDocsUploaded ? (
          <div className="p-4 bg-primary/5 border border-primary/20 rounded-2xl">
            <p className="text-[10px] text-primary uppercase font-black tracking-widest
mb-4">Verification Steps</p>
            <ul className="space-y-3 mb-6">
              <li className="text-[10px] text-white flex items-center gap-2
opacity-60"><Info className="w-3 h-3" /> Aadhar Card / PAN Card</li>
              <li className="text-[10px] text-white flex items-center gap-2
opacity-60"><Info className="w-3 h-3" /> Professional Certificate</li>
              <li className="text-[10px] text-white flex items-center gap-2
opacity-60"><Info className="w-3 h-3" /> Background Check Consent</li>
            </ul>
            <Button
              onClick={handleUpload}
              disabled={isUploading}
              className="w-full text-[10px] py-4 bg-primary text-surface"
            >
              {isUploading ? <Loader2 className="animate-spin w-4 h-4" /> : 'Begin
Secure Upload'}
            </Button>
          </div>
        ) : (
          <div className="p-6 bg-green-500/5 border border-green-500/20 rounded-2xl">
            <div className="flex items-center gap-3 text-green-500 mb-2">
              <CheckCircle2 className="w-5 h-5" />
              <span className="text-xs font-black uppercase tracking-widest">Files
Synchronized</span>
            </div>
            <p className="text-[10px] text-white/60 uppercase tracking-widest font-
bold">Verification in progress. Usually takes 24-48 hours.</p>
          </div>
        )}
      </div>

      <div className="glass-card">
        <h4 className="text-[10px] font-black uppercase tracking-[0.2em] mb-6 text-
muted">Field Assignments</h4>
        <div className="space-y-4">
          {[ 'Elderly Care', 'Nursing', 'Medical Assistant' ].map(tag => (
            <div key={tag} className="flex items-center justify-between p-4 bg-white/5
border border-white/10 rounded-2xl">
              <span className="text-xs font-bold">{tag}</span>
              <div className="w-2 h-2 bg-primary rounded-full" />
            </div>
          ))}
        </div>
      </div>
    </div>
  </div>
)

```

```

    ))}
  </div>
</div>
</div>
</div>
))}

{activeTab === 'bookings' && (
  <div className="space-y-10 animate-in fade-in slide-in-from-bottom-4 duration-500">
    <div className="flex items-center justify-between">
      <SectionHeading title="Active Requests" sub="Job Pipeline" />
      <div className="flex gap-4">
        <div className="px-6 py-2 bg-white/5 border border-white/10 rounded-xl text-[10px] font-black text-muted uppercase tracking-widest">
          SP ID: {providerData?.spId || 'SP-001'}
        </div>
      </div>
    </div>

    <div className="grid grid-cols-1 gap-6">
      {bookings.map(book => (
        <div key={book.id} className="glass-card flex flex-col md:flex-row gap-8 items-start md:items-center">
          <div className="flex-1 space-y-4">
            <div className="flex items-center gap-3">
              <div className="px-3 py-1 bg-primary/10 border border-primary/20 rounded-full text-[8px] font-black text-primary uppercase tracking-widest">
                {book.status}
              </div>
              <span className="text-muted text-[10px] font-bold">#{book.id}</span>
            </div>
            <h3 className="text-2xl font-bold uppercase tracking-tight">{book.service}</h3>
            <div className="flex flex-wrap gap-6 text-[10px] font-bold uppercase tracking-widest text-muted">
              <div className="flex items-center gap-2"><User className="w-3 h-3" />
                {book.customer}</div>
              <div className="flex items-center gap-2"><MapPin className="w-3 h-3" />
                {book.location}</div>
              <div className="flex items-center gap-2"><Calendar className="w-3 h-3" />
                > {book.date}</div>
              <div className="flex items-center gap-2"><Clock className="w-3 h-3" />
                {book.time}</div>
            </div>
          </div>

          {book.status === 'PENDING' && (
            <div className="flex gap-4 w-full md:w-auto">
              <Button disabled={isSpectator} variant="outline" className="flex-1 md:w-32 py-4 text-[10px]" onClick={() => handleAction(book.id, 'DECLINE')}>Decline</Button>
              <Button disabled={isSpectator} variant="primary" className="flex-1 md:w-32 py-4 text-[10px]" onClick={() => handleAction(book.id, 'ACCEPT')}>Accept</Button>
            </div>
          )}

          {book.status === 'ACCEPTED_BY_SP' && (
            <div className="p-4 bg-primary/10 border border-primary/2 tracking-widest rounded-2xl text-[9px] font-black text-primary uppercase">
              Waiting for Admin Confirmation...
            </div>
          )}

          {book.status === 'CONFIRMED' && (
            <div className="space-y-4 w-full md:w-auto">
              <div className="p-4 bg-primary/10 border border-primary/20 rounded-2xl flex flex-col gap-2">
                <span className="text-[9px] text-primary uppercase font-black">Incoming OTP Check</span>
                <input
                  disabled={isSpectator}
                  type="text"
                />
              </div>
            </div>
          )}
        </div>
      )}
    </div>
  </div>
)

```

```

        value={otpInputs[book.id] || ''}
        onChange={(e) => setOtpInputs(prev => ({ ...prev, [book.id]:
e.target.value })))}

        placeholder="ENTER START OTP"
        className="bg-white/5 border border-white/10 px-4 py-2 rounded-lg
text-xs font-mono tracking-widest w-40 outline-none focus:border-primary"
      />
    </div>
    <Button disabled={isSpectator} variant="primary" className="w-full text-
[10px] py-4" onClick={() => handleAction(book.id, 'START_SERVICE')}>
      Start Service
    </Button>
  </div>
)}

{book.status === 'IN_PROGRESS' && (
  <div className="space-y-4 w-full md:w-auto">
    <div className="p-4 border border-emerald-500/20 bg-emerald-500/5
rounded-2xl flex flex-col gap-2">
      <span className="text-[9px] text-emerald-500 uppercase font-
black">Service Duration</span>
      <span className="text-xl font-mono font-bold">02:44:12</span>
      <input
        disabled={isSpectator}
        type="text"
        value={otpInputs[book.id] || ''}
        onChange={(e) => setOtpInputs(prev => ({ ...prev, [book.id]:
e.target.value })))}
        placeholder="COMPLETION OTP"
        className="bg-white/5 border border-white/10 px-4 py-2 rounded-lg
text-xs font-mono tracking-widest w-40 outline-none focus:border-emerald-500 mt-2"
      />
    </div>
    <Button disabled={isSpectator} variant="outline" className="w-full text-
[10px] py-4 border-emerald-500/50 text-emerald-500 hover:bg-emerald-500 hover:text-surface"
onClick={() => handleAction(book.id, 'END_SERVICE')}>
      Complete Service
    </Button>
  </div>
)}
</div>
)}}
</div>
</div>
)}

{activeTab === 'bank' && (
  <div className="max-w-4xl space-y-12 animate-in fade-in duration-500">
    <SectionHeading title="Financial Setup" sub="Bank Details" />
    <div className="glass-card p-8 border-white/10 space-y-6">
      <p className="text-sm text-white/80 leading-relaxed mb-6">Please provide your bank
account details for automated payouts. These details are securely verified by our finance team.</p>
      <div className="space-y-4">
        <input type="text" placeholder="Account Holder Name" className="w-full bg-
white/5 border border-white/10 px-4 py-3 rounded-xl text-xs outline-none focus:border-primary/50 text-
white" />
        <input type="text" placeholder="Bank Name" className="w-full bg-white/5 border
border-white/10 px-4 py-3 rounded-xl text-xs outline-none focus:border-primary/50 text-white" />
        <input type="text" placeholder="Account Number" className="w-full bg-white/5
border border-white/10 px-4 py-3 rounded-xl text-xs outline-none focus:border-primary/50 text-white" />
        <input type="text" placeholder="IFSC Code" className="w-full bg-white/5 border
border-white/10 px-4 py-3 rounded-xl text-xs outline-none focus:border-primary/50 uppercase text-
white" />
      </div>
      <Button variant="primary" className="w-full py-4 text-xs tracking-[0.2em] mt-6"
onClick={() => alert('Bank details submitted for verification.')}>SAVE BANK DETAILS</Button>
    </div>
  </div>
)}
</div>
</div>

```

```

    </div>
  );
};

const AdminDashboard = ({
  onClose,
  actions: propActions = [],
  bankDetails,
  setBankDetails,
  sharingLevel,
  setSharingLevel,
  teamAccess,
  setTeamAccess,
  bookings = [],
  financialAlerts = [],
  onConfirmBooking,
  onAction,
  onSpectateProvider,
  masterOtp,
  setMasterOtp,
  onLogout
}: any) => {
  const [isLocked, setIsLocked] = useState(true);
  const [isMasterAdmin, setIsMasterAdmin] = useState(false);
  const [secretCode, setSecretCode] = useState('');
  const [step, setStep] = useState(1); // 1: Credentials, 2: OTP
  const [adminNumber, setAdminNumber] = useState('');
  const [pass, setPass] = useState('');
  const [otp, setOtp] = useState('');
  const [activeTab, setActiveTab] = useState('overview');

  // Filtering & Sorting State
  const [filterAdmin, setFilterAdmin] = useState('All');
  const [filterType, setFilterType] = useState('All');
  const [startDate, setStartDate] = useState('');
  const [endDate, setEndDate] = useState('');
  const [sortOrder, setSortOrder] = useState('desc');

  // Bookings Specific State
  const [bookingSearch, setBookingSearch] = useState('');
  const [bookingStatusFilter, setBookingStatusFilter] = useState('All');
  const [selectedBookings, setSelectedBookings] = useState<string[]>([]);
  const [detailBooking, setDetailBooking] = useState<any>(null);

  const [providers, setProviders] = useState<any[]>([]);

  useEffect(() => {
    const fetchProviders = async () => {
      try {
        const q = query(collection(db, 'users'), where('role', '==', 'provider'));
        const unsubscribe = onSnapshot(q, (snap) => {
          setProviders(snap.docs.map(d => ({ id: d.id, ...d.data() })));
        });
        return unsubscribe;
      } catch (err) {
        console.error("Error fetching providers:", err);
      }
    };
    fetchProviders();
  }, []);

  const [inquiries, setInquiries] = useState<any[]>([]);
  const [inquiriesLoading, setInquiriesLoading] = useState(false);

  useEffect(() => {
    if (activeTab === 'inquiries') {
      setInquiriesLoading(true);
      fetch('/api/inquiries')
        .then(res => res.json())
        .then(data => {
          if (Array.isArray(data)) setInquiries(data);
        });
    }
  });

```

```

        setInquiriesLoading(false);
    })
    .catch(err => {
        console.error("Error fetching inquiries:", err);
        setInquiriesLoading(false);
    });
}
}, [activeTab]));

const sosAlerts = propActions.filter((a: any) => a.category === 'SOS');
const securityAlerts = sosAlerts;

const [searchQuery, setSearchQuery] = useState('');
const [isAdminAuthLoading, setIsAdminAuthLoading] = useState(false);

const teamMembers = [
    { id: 1, name: "Ashvin G.", role: "Global Admin", email: "ashvin@carevia.app", level: "Full" },
    { id: 2, name: "Sarah K.", role: "Ops Lead", email: "sarah@carevia.app", level: "Standard" },
    { id: 3, name: "Michael R.", role: "Support Staff", email: "michael@carevia.app", level: "Read-
Only" },
];

const admins = ['All', ...new Set(propActions.map(a => a.admin))];
const types = ['All', 'Admin', 'AI BOT', 'System'];

const filteredAndSortedActions = propActions
    .filter(act => {
        const matchAdmin = filterAdmin === 'All' || act.admin === filterAdmin;
        const matchType = filterType === 'All' || (act.type?.toUpperCase() === filterType.toUpperCase())
        || (filterType === 'Admin' && (!act.type || act.type === 'admin'));
        const matchSearch = !searchQuery ||
            act.action.toLowerCase().includes(searchQuery.toLowerCase()) ||
            (act.details && act.details.toLowerCase().includes(searchQuery.toLowerCase())) ||
            (act.sessionId && act.sessionId.toLowerCase().includes(searchQuery.toLowerCase()));

        const actDate = new Date(act.time);
        const matchStart = !startDate || actDate >= new Date(startDate);
        const matchEnd = !endDate || actDate <= new Date(endDate + 'T23:59:59');
        return matchAdmin && matchType && matchSearch && matchStart && matchEnd;
    })
    .sort((a, b) => {
        const timeA = new Date(a.time).getTime();
        const timeB = new Date(b.time).getTime();
        return sortOrder === 'desc' ? timeB - timeA : timeA - timeB;
    });

const handleDeleteInquiry = async (id: string) => {
    if (!confirm("Are you sure you want to delete this inquiry?")) return;
    try {
        const res = await fetch(`/api/inquiries?id=${id}`, { method: 'DELETE' });
        if (res.ok) {
            setInquiries(inquiries.filter((inq: any) => inq.id !== id));
        } else {
            alert("Failed to delete inquiry");
        }
    } catch (e) {
        console.error(e);
        alert("Error deleting inquiry");
    }
};

const handleVerifySP = async (provider: any) => {
    const adminId = prompt("Enter your Admin ID No. to confirm verification:");
    if (!adminId) return;
    try {
        const generatedSpId = 'SP-' + Math.floor(10000 + Math.random() * 90000);
        await updateDoc(doc(db, 'users', provider.id), {
            isVerified: true,
            status: 'Confirmed',
            verifiedBy: adminId,
            spId: generatedSpId,
        });
    }
};

```

```

        verifiedAt: serverTimestamp()
    });

    // Log to Google Sheets via Netlify Form for now, or just to adminNotifications
    const formData = new URLSearchParams();
    formData.append('form-name', 'sp-verifications');
    formData.append('providerId', provider.id);
    formData.append('providerPhone', provider.phone || '');
    formData.append('adminId', adminId);
    formData.append('spId', generatedSpId);
    formData.append('timestamp', new Date().toISOString());
    fetch('/', {
        method: 'POST',
        headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
        body: formData.toString()
    }).catch(e => console.error(e));

    alert(`Service Provider Verified Successfully.\nGenerated SP ID NO: ${generatedSpId}`);
} catch (err) {
    console.error("Error verifying SP:", err);
    alert("Failed to verify SP.");
}
};

const handleRejectSP = async (provider: any) => {
    const confirmReject = window.confirm("Are you sure you want to reject this application?");
    if (!confirmReject) return;
    try {
        await updateDoc(doc(db, 'users', provider.id), {
            status: 'Rejected',
            isVerified: false,
            rejectedAt: serverTimestamp()
        });
        alert("Application Rejected.");
    } catch (err) {
        console.error("Error rejecting SP:", err);
        alert("Failed to reject application.");
    }
};

const handleVerify = () => {
    if (secretCode === 'BIWIJAWANKARU') {
        setIsLocked(false);
        setIsMasterAdmin(true);
    } else if (secretCode === 'ADMIN_CAREVIA' || secretCode === import.meta.env.VITE_ADMIN_SECRET) {
        setIsLocked(false);
    } else {
        alert("Invalid Secret Code");
    }
};

const generateSystemOtp = () => {
    const code = Math.floor(100000 + Math.random() * 900000).toString();
    setMasterOtp(code);
    setTimeout(() => setMasterOtp(null), 300000); // 5 min expiry
};

const filteredBookings = bookings.filter((b: any) => {
    const matchesSearch = !bookingSearch ||
        b.id.toLowerCase().includes(bookingSearch.toLowerCase()) ||
        b.customer.toLowerCase().includes(bookingSearch.toLowerCase()) ||
        b.service.toLowerCase().includes(bookingSearch.toLowerCase());

    const matchesStatus = bookingStatusFilter === 'All' || b.status === bookingStatusFilter;

    return matchesSearch && matchesStatus;
});

const toggleBookingSelection = (id: string) => {
    setSelectedBookings(prev =>
        prev.includes(id) ? prev.filter(bid => bid !== id) : [...prev, id]
    );
};

```

```

};

const handleBulkAction = (action: string) => {
  if (!onAction) return;
  selectedBookings.forEach(id => {
    onAction(id, action);
  });
  setSelectedBookings([]);
};

if (isLocked) {
  return (
    <div className="fixed inset-0 z-[300] bg-surface/95 backdrop-blur-xl flex items-center justify-center p-6 font-sans">
      <div className="glass-card w-full max-w-md p-10 text-center">
        <div className="w-16 h-16 bg-primary/20 rounded-full flex items-center justify-center mx-auto mb-6 text-primary border border-primary/20">
          <Lock className="w-8 h-8" />
        </div>
        <h2 className="text-2xl font-bold uppercase tracking-tight mb-2">Admin Command Center</h2>
        <p className="text-xs text-white/60 uppercase tracking-[0.2em] mb-8 font-bold">Protocol: Secure 2-Step Gate</p>

        <div className="space-y-6">
          <div>
            <label className="label-bold mb-2 block text-left text-primary">Master Secret Code</label>
            <input
              type="password"
              value={secretCode || ''}
              onChange={(e) => setSecretCode(e.target.value)}
              className="w-full bg-white/5 border border-white/10 px-6 py-4 rounded-xl text-center text-sm outline-none focus:border-primary tracking-[0.5em] font-mono"
              placeholder="....."
            />
          </div>
          <div className="flex gap-4">
            <Button variant="outline" className="flex-1 py-4 text-[10px]" onClick={onClose}>Cancel</Button>
            <Button variant="primary" className="flex-1 py-4 text-[10px]" onClick={handleVerify}>Access Secure Hub</Button>
          </div>
        </div>
      </div>
    </div>
  );
}

return (
  <div className="fixed inset-0 z-[300] bg-surface flex flex-col font-sans">
    <div className="border-b border-white/5 p-6 flex justify-between items-center bg-primary-container/10">
      <div className="flex items-center gap-4">
        
        <h2 className="text-xl font-bold uppercase tracking-tight">
          {isMasterAdmin ? 'Master Control Console' : 'Admin Console v1.0'}
        </h2>
        {isMasterAdmin ? (
          <div className="px-3 py-1 bg-red-500/20 border border-red-500/50 rounded-full text-[8px] font-black uppercase text-red-500 tracking-widest animate-pulse">MASTER CONTROL</div>
        ) : (
          <div className="px-3 py-1 bg-primary/10 border border-primary/20 rounded-full text-[8px] font-black uppercase text-primary tracking-widest">Team Access On</div>
        )}
      </div>
    </div>
    <div className="flex items-center gap-4">

```

```

        <div className="flex flex-col text-right">
            <span className="text-[10px] font-bold text-white/60 uppercase tracking-widest">Active
Head</span>
            <span className="text-xs font-bold text-primary">ID: ADMIN_001</span>
        </div>
        <button onClick={onClose} className="p-2 hover:bg-white/10 rounded-full"><X /></button>
    </div>
</div>

<div className="flex flex-1 overflow-hidden">
    <div className="w-64 border-r border-white/5 p-6 flex flex-col gap-2 bg-neutral-900/50">
        <button
            onClick={() => setActiveTab('overview')}
            className={`flex items-center gap-3 w-full p-3 rounded-xl text-xs font-bold uppercase tracking-
widest transition-all ${activeTab === 'overview' ? 'bg-primary/10 text-primary' : 'hover:bg-white/5
text-white/60'}`}
        >
            <LayoutDashboard className="w-4 h-4" /> Overview
        </button>
        <button
            onClick={() => setActiveTab('bookings')}
            className={`flex items-center gap-3 w-full p-3 rounded-xl text-xs font-bold uppercase tracking-
widest transition-all ${activeTab === 'bookings' ? 'bg-primary/10 text-primary' : 'hover:bg-white/5
text-white/60'}`}
        >
            <Calendar className="w-4 h-4" /> Booking Ops {bookings.filter(b => b.status ===
"ACCEPTED_BY_SP").length > 0 && <span className="ml-auto w-2 h-2 bg-primary rounded-full animate-
pulse" />}
        </button>
        <button
            onClick={() => setActiveTab('intel')}
            className={`flex items-center gap-3 w-full p-3 rounded-xl text-xs font-bold uppercase tracking-
widest transition-all ${activeTab === 'intel' ? 'bg-primary/10 text-primary' : 'hover:bg-white/5 text-
white/60'}`}
        >
            <Eye className="w-4 h-4" /> Partner Intel
        </button>
        <button
            onClick={() => setActiveTab('sp-requests')}
            className={`flex items-center gap-3 w-full p-3 rounded-xl text-xs font-bold uppercase tracking-
widest transition-all ${activeTab === 'sp-requests' ? 'bg-primary/10 text-primary' : 'hover:bg-white/5
text-white/60'}`}
        >
            <BriefcaseMedical className="w-4 h-4" /> SP Requests
        </button>
        <button
            onClick={() => setActiveTab('data')}
            className={`flex items-center gap-3 w-full p-3 rounded-xl text-xs font-bold uppercase tracking-
widest transition-all ${activeTab === 'data' ? 'bg-primary/10 text-primary' : 'hover:bg-white/5 text-
white/60'}`}
        >
            <Database className="w-4 h-4" /> Data & Export
        </button>
        <button
            onClick={() => setActiveTab('inquiries')}
            className={`flex items-center gap-3 w-full p-3 rounded-xl text-xs font-bold uppercase tracking-
widest transition-all ${activeTab === 'inquiries' ? 'bg-primary/10 text-primary' : 'hover:bg-white/5
text-white/60'}`}
        >
            <MessageSquare className="w-4 h-4" /> AI Inquiries
        </button>
        <button
            onClick={() => setActiveTab('financials')}
            className={`flex items-center gap-3 w-full
p-3 rounded-xl text-xs font-bold uppercase tracking-widest transition-all ${activeTab ===
'financials' ? 'bg-primary/10 text-primary' : 'hover:bg-white/5 text-white/60'}`}
        >
            <TrendingUp className="w-4 h-4" /> Financials
        </button>
        {isMasterAdmin && (
            <button
                onClick={() => setActiveTab('bank')}

```

```

        className={`flex items-center gap-3 w-full p-3 rounded-xl text-xs font-bold uppercase
tracking-widest transition-all ${activeTab === 'bank' ? 'bg-primary/10 text-primary' : 'hover:bg-
white/5 text-white/60'}`}
    >
        <BriefcaseMedical className="w-4 h-4" /> Bank Details
    </button>
)}
{isMasterAdmin && (
    <button
        onClick={() => setActiveTab('surveillance')}
        className={`flex items-center gap-3 w-full p-3 rounded-xl text-xs font-bold uppercase tracking-
widest transition-all ${activeTab === 'surveillance' ? 'bg-primary/10 text-primary' :
'hover:bg-white/5 text-white/60'}`}
    >
        <Eye className="w-4 h-4" /> Admin Surveillance
    </button>
)}
<button
    onClick={() => setActiveTab('security')}
    className={`flex items-center gap-3 w-full p-3 rounded-xl text-xs font-bold uppercase tracking-
widest transition-all ${activeTab === 'security' ? 'bg-red-500/10 text-red-500' : 'hover:bg-white/5
text-white/60'}`}
    >
        <ShieldAlert className="w-4 h-4" /> Security Protocols
    </button>

<div className="mt-4 pt-4 border-t border-white/5 space-y-2">
    <p className="text-[8px] font-black text-white/40 uppercase tracking-widest px-3 mb-2">Cloud
Synced Tools</p>
    <a
        href="https://drive.google.com"
        target="_blank"
        rel="noopener noreferrer"
        className="flex items-center gap-3 w-full p-3 hover:bg-white/5 text-white/60 rounded-xl text-
xs font-bold uppercase tracking-widest transition-all"
    >
        <Database className="w-4 h-4 text-amber-500" /> Google Drive
    </a>
    <a
        href="https://mail.google.com"
        target="_blank"
        rel="noopener noreferrer"
        className="flex items-center gap-3 w-full p-3 hover:bg-white/5 text-white/60 rounded-xl text-
xs font-bold uppercase tracking-widest transition-all"
    >
        <Mail className="w-4 h-4 text-emerald-500" /> Administrative Gmail
    </a>
</div>

<div className="mt-auto pt-6 border-t border-white/5">
    <div className="px-3 mb-4">
        <p className="text-[8px] font-black text-primary uppercase tracking-widest
mb-2">Master OTP Generator</p>
        <div className="flex gap-2">
            <div className="flex-1 bg-white/5 border border-white/10 rounded-lg flex items-
center justify-center font-mono text-xs font-bold">
                {masterOtp || '-----'}
            </div>
            <button
                onClick={generateSystemOtp}
                className="p-2 bg-primary/10 hover:bg-primary/20 text-primary rounded-lg
transition-colors"
                title="Generate Token"
            >
                <Sparkles className="w-4 h-4" />
            </button>
        </div>
        {masterOtp && <p className="text-[7px] text-white/50 uppercase mt-1">Expiring in
300s...</p>}
    </div>
    <button className="flex items-center gap-3 w-full p-3 hover:bg-white/5 text-white/60

```

```

rounded-xl text-xs font-bold uppercase tracking-widest mb-2"><Navigation className="w-4 h-4" /> Server
Status</button>
    <button className="flex items-center gap-3 w-full p-3 text-red-500/50 hover:text-red-500
text-xs font-bold uppercase tracking-widest" onClick={onLogout || onClose}><LogOut className="w-4
h-4" /> Exit Panel</button>
    </div>
</div>
<div className="flex-1 p-10 overflow-y-auto bg-black/20">
    {activeTab === 'overview' && (
        <div className="animate-in fade-in duration-500">
            <div className="grid grid-cols-1 lg:grid-cols-4 gap-6 mb-12">
                <div className="glass-card p-6 border-white/10 text-center">
                    <p className="text-[10px] font-bold text-white/50 uppercase tracking-widest
mb-2">Total Users</p>
                    <p className="text-3xl font-bold">0</p>
                </div>
                <div className="glass-card p-6 border-white/10 text-center">
                    <p className="text-[10px] font-bold text-white/50 uppercase tracking-widest
mb-2">Active Care</p>
                    <p className="text-3xl font-bold">0</p>
                </div>
                <div className="glass-card p-6 border-white/10 text-center">
                    <p className="text-[10px] font-bold text-white/50 uppercase tracking-widest mb-2
text-primary">Revenue</p>
                    <p className="text-3xl font-bold"> 10</p>
                </div>
                <div className="glass-card p-6 border-red-500/20 text-center bg-red-500/5">
                    <p className="text-[10px] font-bold text-red-500 uppercase tracking-widest
mb-2">SOS Alerts</p>
                    <p className="text-3xl font-bold text-red-500 animate-pulse">{sosAlerts.length}</
p>
                </div>
            </div>
            <div>
                <div className="grid grid-cols-1 lg:grid-cols-2 gap-6 mb-12">
                    <div className="glass-card p-8 border-white/5 text-left">
                        <div className="flex items-center justify-between mb-8">
                            <h3 className="text-lg font-bold uppercase tracking-tight">Active Operation
Stream</h3>
                            <span className="flex items-center gap-2 text-[8px] font-bold text-
emerald-500 uppercase tracking-widest">
                                <div className="w-1.5 h-1.5 bg-emerald-500 rounded-full animate-pulse" />
                                Live Feed
                            </span>
                        </div>
                        <div className="space-y-4">
                            {bookings.filter((b: any) => b.status === "ACCEPTED_BY_SP").map((book: any) => (
                                <div key={book.id} className="p-4 bg-primary/5 border border-primary/20
rounded-2xl flex items-center justify-between">
                                    <div className="flex items-center gap-4">
                                        <Activity className="w-4 h-4 text-primary" />
                                        <div>
                                            <p className="text-xs font-bold">{book.service} accepted by
{book.providerName || 'Partner'}</p>
                                            <p className="text-[10px] text-white/60 uppercase tracking-
widest">Awaiting Admin Confirmation</p>
                                        </div>
                                    </div>
                                    <Button variant="primary" className="text-[8px] py-1.5 px-4" onClick={() =>
setActiveTab('bookings')}>Handle</Button>
                                </div>
                            ))}
                            {bookings.filter((b: any) => b.status === "ACCEPTED_BY_SP").length === 0 && (
                                <p className="text-[10px] text-center text-white/40 uppercase font-bold
py-10">No immediate actions required</p>
                            )}
                        </div>
                    </div>
                </div>
                <div className="glass-card p-8 border-red-500/10 bg-red-500/[0.02] text-left">
                    <div className="flex items-center justify-between mb-8">

```

```

<h3 className="text-lg font-bold uppercase tracking-tight text-red-500">Critical
Access Alerts</h3>
    <button onClick={() => setActiveTab('security')} className="text-[8px] font-bold
text-red-500 uppercase tracking-widest hover:underline">View All</button>
</div>
<div className="space-y-3">
    {sosAlerts.slice(0, 3).map((alert: any) => (
        <div key={alert.id} className="p-4 bg-white/5 border border-white/5 rounded-2xl
text-left">
            <div className="flex justify-between items-start mb-2">
                <span className="text-[8px] font-black text-red-500 uppercase tracking-
widest px-2 py-0.5 bg-red-500/10 rounded">SOS: {alert.type}</span>
                <span className="text-[8px] font-bold text-white/40 uppercase">
                    {alert.createdAt?.seconds ? new Date(alert.createdAt.seconds *
1000).toLocaleString() : 'Just now'}
                </span>
            </div>
            <div className="flex justify-between items-end">
                <div>
                    <p className="text-xs font-bold text-red-400">{alert.userEmail}</p>
                    <p className="text-[8px] text-white/60 uppercase tracking-widest
mt-1">UID: {alert.userId}</p>
                </div>
                <div className="text-right">
                    <p className="text-[10px] font-mono text-white">{alert.message}</p>
                </div>
            </div>
        </div>
    )))
    {sosAlerts.length === 0 && <p className="text-[10px] text-center text-white/40
uppercase py-10 font-black">No SOS Alerts Active</p>}
</div>
</div>
</div>
)}

{activeTab === 'bookings' && (
    <div className="space-y-8 animate-in fade-in slide-in-from-bottom-4 duration-500 pb-20">
        <div className="flex flex-col lg:flex-row lg:items-center justify-between gap-6">
            <div>
                <h3 className="text-xl font-bold uppercase tracking-tight">Booking Operations</
h3>
                <p className="text-[10px] text-white/60 uppercase tracking-widest
mt-1">Lifecycle Hub & Command Control</p>
            </div>

            <div className="flex flex-wrap items-center gap-4">
                </* Search & History Filters */>
                <div className="relative">
                    <Search className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-
white/40" />
                    <input
                        type="text"
                        placeholder="Search bookings..."
                        value={bookingSearch}
                        onChange={(e) => setBookingSearch(e.target.value)}
                        className="bg-white/5 border border-white/10 rounded-xl pl-10 pr-4 py-2 text-
xs outline-none focus:border-primary transition-all w-64"
                    />
                </div>

                <div className="flex items-center gap-2 bg-white/5 border border-white/10
rounded-xl p-1">
                    {[ 'All', 'PENDING', 'CONFIRMED', 'IN_PROGRESS', 'COMPLETED' ].map(st => (
                        <button
                            key={st}
                            onClick={() => setBookingStatusFilter(st)}
                            className={`px-3 py-1.5 rounded-lg text-[9px] font-bold transition-all
uppercase tracking-widest ${bookingStatusFilter === st ? 'bg-primary text-surface' : 'text-white/60
hover:text-white'}``

```

```

        >
        {st === 'All' ? 'History' : st.replace('_', ' ')}
      </button>
    )}}
  </div>
</div>
</div>

{/* Bulk Action Bar */}
<AnimatePresence>
  {selectedBookings.length > 0 && (
    <motion.div
      initial={{ opacity: 0, y: 20 }}
      animate={{ opacity: 1, y: 0 }}
      exit={{ opacity: 0, y: 20 }}
      className="fixed bottom-10 left-1/2 -translate-x-1/2 glass-card border-
primary/40 bg-primary/10 px-8 py-4 flex items-center gap-8 z-[400] shadow-2xl backdrop-blur-2xl"
    >
      <div className="flex items-center gap-3">
        <div className="w-8 h-8 rounded-full bg-primary flex items-center justify-
center text-surface font-bold text-xs">
          {selectedBookings.length}
        </div>
        <p className="text-[10px] font-black uppercase tracking-widest text-
primary">Records Selected</p>
      </div>
      <div className="h-8 w-px bg-primary/20" />
      <div className="flex gap-3">
        <button onClick={() => handleBulkAction('START_SERVICE')} className="px-4
py-2 bg-amber-500 text-surface rounded-lg text-[9px] font-black uppercase tracking-widest hover:bg-
amber-600">Bulk Start</button>
        <button onClick={() => handleBulkAction('END_SERVICE')} className="px-4
py-2 bg-emerald-500 text-surface rounded-lg text-[9px] font-black uppercase tracking-widest hover:bg-
emerald-600">Bulk Complete</button>
        <button onClick={() => setSelectedBookings([])} className="px-4 py-2
hover:bg-white/10 rounded-lg text-[9px] font-black uppercase tracking-widest">Clear Selection</button>
      </div>
    </motion.div>
  )}
</AnimatePresence>

<div className="grid grid-cols-1 gap-4">
  {filteredBookings.length === 0 && (
    <div className="p-20 text-center border-2 border-dashed border-white/5
rounded-3xl opacity-20">
      <Search className="w-12 h-12 mx-auto mb-4" />
      <p className="text-sm font-bold uppercase tracking-[0.2em]">Zero results in
target scope</p>
    </div>
  )}

  {filteredBookings.map((book: any) => {
    const isSelected = selectedBookings.includes(book.id);
    return (
      <div
        key={book.id}
        className={`glass-card p-6 border-white/10 flex items-center justify-between
transition-all hover:bg-white/[0.02] cursor-pointer group ${isSelected ? 'border-primary/40 bg-primary/
[0.02]' : ''}`}
        onClick={() => setDetailBooking(book)}
      >
        <div className="flex items-center gap-6">
          <button
            onClick={(e) => {
              e.stopPropagation();
              toggleBookingSelection(book.id);
            }}
            className={`w-5 h-5 rounded border transition-all flex items-center
justify-center ${isSelected ? 'border-primary bg-primary text-surface' : 'border-white/20 hover:border-
primary'}`}
          >

```

```

        {isSelected && <CheckCircle2 className="w-3 h-3" />}
      </button>

      <div className="w-12 h-12 bg-white/5 rounded-full flex items-center
justify-center relative flex-shrink-0">
        <div className={`absolute -top-1 -right-1 w-3 h-3 rounded-full
border-2 border-surface ${
          book.status === 'IN_PROGRESS' ? 'bg-amber-500 animate-pulse' :
          book.status === 'CONFIRMED' ? 'bg-emerald-500' :
          book.status === 'PENDING' ? 'bg-blue-500' : 'bg-white/20'
        }}` />
          {book.status === 'IN_PROGRESS' ? <Activity className="w-6 h-6 text-
amber-500" /> :
            book.status === 'PENDING' ? <Clock className="w-6 h-6 text-
blue-400" /> :
              <ShieldCheck className="w-6 h-6 text-emerald-500" />}
        </div>
      <div>
        <div className="flex items-center gap-2 mb-1">
          <span className={`text-[9px] font-black uppercase tracking-[0.1em]
px-2 py-0.5 rounded ${
            book.status === 'IN_PROGRESS' ? 'bg-amber-500/10 text-
amber-500' :
            book.status === 'CONFIRMED' ? 'bg-emerald-500/10 text-
emerald-500' :
            book.status === 'PENDING' ? 'bg-blue-500/10 text-blue-400' : 'bg-
white/5 text-white/60'
          }}`>
            {book.status.replace('_', ' ')}
          </span>
          <span className="text-xs font-bold text-white/60">#{book.id}</span>
        </div>
        <h4 className="text-lg font-bold flex items-center gap-2">
          {book.service}
          <div className="opacity-0 group-hover:opacity-100 transition-
opacity">
            <div className="text-[8px] font-bold uppercase py-0.5 px-2 bg-
white/10 rounded">Details</div>
          </div>
        </h4>
        <p className="text-xs text-white/80">Partner: <span className="text-
white font-bold">{book.providerName || 'N/A'}</span> • Customer: {book.customer}</p>
      </div>
    </div>
    <div className="flex gap-3" onClick={e => e.stopPropagation()}>
      {book.status === 'ACCEPTED_BY_SP' && (
        <Button variant="primary" className="text-[9px] py-2 px-6" onClick={()
=> onConfirmBooking(book.id)}>Confirm Allocation</Button>
      )}
      {book.status === 'CONFIRMED' && (
        <Button
          variant="primary"
          className="text-[9px] py-2 px-6 bg-amber-500 text-surface border-
none"
          onClick={() => onAction && onAction(book.id, 'START_SERVICE')}
        >
          Start Service
        </Button>
      )}
      {book.status === 'IN_PROGRESS' && (
        <Button
          variant="primary"
          className="text-[9px] py-2 px-6 bg-emerald-500 text-surface border-
none"
          onClick={() => onAction && onAction(book.id, 'END_SERVICE')}
        >
          Complete Service
        </Button>
      )}
    </div>
  </div>

```

```

    });
  }}
</div>
</div>
))

{/* Booking Details Modal */}
<AnimatePresence>
  {detailBooking && (
    <div className="fixed inset-0 z-[500] flex items-center justify-center p-6 sm:p-20">
      <motion.div
        initial={{ opacity: 0 }}
        animate={{ opacity: 1 }}
        exit={{ opacity: 0 }}
        onClick={() => setDetailBooking(null)}
        className="absolute inset-0 bg-black/80 backdrop-blur-md"
      />
      <motion.div
        initial={{ opacity: 0, scale: 0.95, y: 20 }}
        animate={{ opacity: 1, scale: 1, y: 0 }}
        exit={{ opacity: 0, scale: 0.95, y: 20 }}
        className="glass-card w-full max-w-2xl overflow-hidden relative z-10"
      >
        <div className="absolute top-0 left-0 w-full h-2 bg-gradient-to-r from-primary
via-emerald-500 to-amber-500" />

        <div className="p-8">
          <div className="flex justify-between items-start mb-8">
            <div>
              <div className="flex items-center gap-3 mb-2">
                <span className="text-[10px] font-black text-primary uppercase
tracking-[0.2em] px-3 py-1 bg-primary/10 rounded-full">Booking Manifest</span>
                <span className="text-sm font-bold text-white/50">ID:
{detailBooking.id}</span>
              </div>
              <h3 className="text-3xl font-black uppercase tracking-
tight">{detailBooking.service}</h3>
            </div>
            <button onClick={() => setDetailBooking(null)} className="p-2 hover:bg-
white/5 rounded-full text-white/60"><X /></button>
          </div>

          <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
            <div className="space-y-6">
              <div className="p-6 bg-white/5 rounded-2xl border border-white/5">
                <p className="text-[10px] font-bold text-white/40 uppercase tracking-
widest mb-4">Customer Intelligence</p>
                <div className="flex items-center gap-4 mb-4">
                  <div className="w-12 h-12 rounded-full bg-primary/20 flex items-
center justify-center text-primary">
                    <User className="w-6 h-6" />
                  </div>
                  <div>
                    <p className="text-lg font-bold">{detailBooking.customer}</p>
                    <p className="text-[10px] text-white/60 uppercase tracking-
widest">{detailBooking.location}</p>
                  </div>
                </div>
                <div className="space-y-2 pt-2 border-t border-white/5">
                  <div className="flex justify-between text-[10px]">
                    <span className="text-white/50 uppercase font-bold tracking-
widest">Protocol</span>
                    <span className="font-bold text-primary">ELITE_SERVICE</span>
                  </div>
                  <div className="flex justify-between text-[10px]">
                    <span className="text-white/50 uppercase font-bold tracking-
widest">Priority</span>
                    <span className="font-bold text-amber-500">HIGH</span>
                  </div>
                </div>
              </div>
            </div>
          </div>
        </div>
      </div>
    )
  )
}

```



```

        <button
          onClick={() => setDetailBooking(null)}
          className="flex-1 py-4 bg-white/5 hover:bg-white/10 text-[10px] uppercase font-black tracking-widest rounded-xl transition-all"
        >
          Close Manifest
        </button>
      {detailBooking.status === 'CONFIRMED' && (
        <button
          onClick={() => {
            onAction && onAction(detailBooking.id, 'START_SERVICE');
            setDetailBooking(null);
          }}
          className="flex-1 py-4 bg-amber-500 text-surface text-[10px] uppercase font-black tracking-widest rounded-xl transition-all shadow-xl shadow-amber-500/20"
        >
          Execute Service Start
        </button>
      )}
    </div>
  </div>
</motion.div>
</div>
)}
</AnimatePresence>

{activeTab === 'surveillance' && isMasterAdmin && (
  <div className="space-y-6 animate-in fade-in slide-in-from-bottom-5 duration-500">
    <div className="flex justify-between items-center mb-6 border-b border-white/5 pb-4">
      <div>
        <h3 className="text-xl font-bold uppercase tracking-tight text-primary flex items-center gap-3">
          <Eye className="w-6 h-6" /> Admin Surveillance
        </h3>
        <p className="text-[10px] uppercase font-bold text-white/60 tracking-widest mt-1">Live monitoring of all admin activities</p>
      </div>
    </div>
    <div className="glass-card p-6">
      <div className="space-y-4">
        {propActions.length > 0 ? (
          propActions.map((act: any, i: number) => (
            <div key={i} className="p-4 bg-white/5 border border-white/10 rounded-2xl flex justify-between items-center hover:bg-white/10 transition-colors">
              <div className="flex items-center gap-4">
                <div className="w-10 h-10 bg-primary/20 rounded-full flex items-center justify-center border border-primary/30">
                  <ShieldCheck className="w-5 h-5 text-primary" />
                </div>
                <div>
                  <p className="text-sm font-bold text-white">{act.action}</p>
                  <p className="text-[10px] text-white/70 uppercase tracking-widest">{act.admin || 'Unknown Admin'} • {act.time}</p>
                </div>
              </div>
              <div>
                <div className="flex items-center gap-4">
                  <span className={`${px-3 py-1 rounded-full text-[8px] font-black uppercase tracking-widest ${act.status === 'Reverted' ? 'bg-red-500/20 text-red-500 border border-red-500/50' : 'bg-emerald-500/20 text-emerald-500 border border-emerald-500/50'}}`}>
                    {act.status || 'LOG'}
                  </span>
                  <button onClick={() => {
                    alert(`Master Control: Overriding Action [${act.action}]... Reverted.`);
                  }} className="px-4 py-2 bg-red-500/10 text-red-500 rounded-xl text-[10px] font-bold uppercase tracking-widest hover:bg-red-500 hover:text-white transition-all border border-red-500/20">
                    Undo Action
                  </button>
                </div>
              </div>
            </div>
          ))
        ) : null}
      </div>
    </div>
  </div>
)

```

```

    ))
  ) : (
    <div className="text-center p-12 bg-white/5 border border-white/5 rounded-3xl">
      <EyeOff className="w-12 h-12 mx-auto text-white/40 mb-4" />
      <p className="text-white/60 uppercase text-xs font-bold tracking-widest">No
Admin Activities Recorded</p>
    </div>
  )}
</div>
</div>
</div>
)}

{activeTab === 'security' && (
slide-in-from-bottom-5 duration-500">
  <div className="flex items-center justify-between">
    <div>
      <h3 className="text-2xl font-black uppercase tracking-tight text-
red-500">Security Protocols</h3>
      <p className="text-[10px] text-white/60 uppercase tracking-widest mt-1">Master
Admin Oversight Panel</p>
    </div>
    <div className="px-4 py-2 bg-red-500/10 border border-red-500/20 rounded-xl flex
items-center gap-3">
      <div className="w-2 h-2 bg-red-500 rounded-full animate-pulse" />
      <span className="text-[10px] font-bold text-red-500 uppercase tracking-
widest">Real-time Intrusion Monitoring</span>
    </div>
  </div>

  <div className="grid grid-cols-1 md:grid-cols-2 gap-6 mb-8 mt-8">
    <div className="glass-card p-8 border-red-500/20 bg-red-500/5">
      <ShieldAlert className="w-8 h-8 text-red-500 mb-4" />
      <h4 className="text-sm font-bold uppercase tracking-widest mb-2">Generate
Master Bypass</h4>
      <p className="text-[10px] text-white/60 leading-relaxed mb-6">Generates a high-
security temporary OTP for emergency system access or manual overrides.</p>
      <button
        onClick={generateSystemOtp}
        className="w-full py-4 bg-red-500 text-surface text-[10px] font-black
uppercase tracking-widest rounded-xl hover:bg-red-600 transition-all font-mono"
        >
        {masterOtp ? `Active OTP: ${masterOtp}` : 'Trigger Emergency OTP'}
      </button>
    </div>
    <div className="glass-card p-8 border-amber-500/20 bg-amber-500/5">
      <Zap className="w-8 h-8 text-amber-500 mb-4" />
      <h4 className="text-sm font-bold uppercase tracking-widest mb-2">Website Risk
Parameter</h4>
      <p className="text-[10px] text-white/60 leading-relaxed mb-6">Current threat
level: LOW. All regional nodes are operating within normal latency parameters.</p>
      <div className="flex items-center gap-4">
        <span className="text-[10px] font-bold text-amber-500 uppercase tracking-
widest">Auto-Mitigation:</span>
        <div className="flex-1 bg-white/10 h-2 rounded-full overflow-hidden">
          <div className="bg-amber-500 h-full w-full" />
        </div>
        <span className="text-[10px] font-bold text-amber-500">ACTIVE</span>
      </div>
    </div>
  </div>

  <div className="glass-card p-0 overflow-hidden border-red-500/10">
    <div className="bg-white/5 p-4 flex justify-between items-center border-b border-
white/5">
      <p className="text-[10px] font-black uppercase tracking-widest
opacity-40">Unauthorized Login Intercepts</p>
      <p className="text-[8px] font-bold text-white/40 uppercase">Source: Network
Gatekeeper</p>
    </div>
    <div className="overflow-x-auto">

```

```

        <table className="w-full text-left">
          <thead className="bg-white/[0.02] border-b border-white/5">
            <tr className="text-[8px] font-black text-white/60 uppercase tracking-
widest">
              <th className="px-6 py-4">Timestamp</th>
              <th className="px-6 py-4">Auth ID</th>
              <th className="px-6 py-4">Target Number</th>
              <th className="px-6 py-4">Handshake (OTP)</th>
              <th className="px-6 py-4">Encryption (PW)</th>
              <th className="px-6 py-4 text-right">Status</th>
            </tr>
          </thead>
          <tbody className="divide-y divide-white/5">
            {securityAlerts.map((alert) => (
              <tr key={alert.id} className="hover:bg-white/[0.01] transition-colors
group">
                <td className="px-6 py-4 text-[10px] font-bold text-
white/60">{alert.date}</td>
                <td className="px-6 py-4 text-[10px] font-mono font-bold text-
primary">{alert.detailId}</td>
                <td className="px-6 py-4 text-[10px] font-bold text-
white">{alert.number}</td>
                <td className="px-6 py-4 text-[10px] font-mono text-
amber-500">{alert.otp}</td>
                <td className="px-6 py-4 text-[10px] font-mono text-
white/60">{alert.pass}</td>
                <td className="px-6 py-4 text-right">
                  <span className="text-[8px] font-black uppercase px-2 py-0.5
rounded bg-red-500/10 text-red-500 border border-red-500/20 tracking-widest">
                    {alert.status}
                  </span>
                </td>
              </tr>
            )
          )
        </tbody>
      </table>
    </div>
  </div>
</div>
)}

{activeTab === 'inquiries' && (
  <div className="space-y-8 animate-in fade-in slide-in-from-bottom-5 duration-500">
    <div className="flex items-center justify-between mb-8">
      <div>
        <h3 className="text-2xl font-bold uppercase tracking-tight">AI Generated Inquiries</h3>
        <p className="text-xs text-white/60 uppercase tracking-widest mt-1">Customer
questions handled by AI Agent</p>
      </div>
      <button
        onClick={() => {
          setInquiriesLoading(true);
          fetch('/api/inquiries').then(r => r.json()).then(d => { setInquiries(d);
setInquiriesLoading(false); });
        }}
        className="btn-outline p-3 text-[10px] font-bold uppercase tracking-widest text-
primary border-primary/20 flex items-center gap-2 hover:bg-primary/10"
      >
        {inquiriesLoading ? <Loader2 className="w-4 h-4 animate-spin" /> : <Activity
className="w-4 h-4" />} Refresh
      </button>
    </div>

    <div className="glass-card border-white/5 overflow-hidden">
      {inquiriesLoading && inquiries.length === 0 ? (
        <div className="p-12 text-center text-white/60 flex flex-col items-center gap-4">
          <Loader2 className="w-8 h-8 animate-spin text-primary" />
          <p className="text-xs font-bold uppercase tracking-widest">Loading Inquiries...</p>
        </div>
      )
    }
  </div>
)}

```

```

    ) : inquiries.length === 0 ? (
      <div className="p-12 text-center text-white/60">
        <p className="text-xs font-bold uppercase tracking-widest">No inquiries found.</p>
      </div>
    ) : (
      <div className="divide-y divide-white/5">
        {inquiries.map((inq: any) => (
          <div key={inq.id} className="p-6 hover:bg-white/5 transition-colors">
            <div className="flex justify-between items-start mb-4">
              <div>
                <p className="text-sm font-bold">{inq.name} <span className="text-white/60 text-xs ml-2">{inq.email || inq.phone}</span></p>
                <p className="text-[10px] font-bold text-primary uppercase tracking-widest mt-1">{inq.category}</p>
              </div>
              <div className="flex items-center gap-4">
                <span className="text-[10px] font-bold text-white/50 uppercase tracking-widest">{new Date(inq.timestamp).toLocaleString()}</span>
                <button onClick={() => handleDeleteInquiry(inq.id)} className="text-red-500 hover:text-red-400 text-xs font-bold uppercase tracking-widest transition-colors"><Trash2 className="w-3 h-3" /></button>
              </div>
            </div>
            <div>
              <div className="space-y-4">
                <div className="bg-white/5 p-4 rounded-xl border border-white/10">
                  <p className="text-xs font-bold text-white/60 uppercase tracking-widest mb-2 flex items-center gap-2"><User className="w-3 h-3" /> Customer Query</p>
                  <p className="text-sm text-white leading-relaxed">{inq.query}</p>
                </div>
                <div className="bg-primary/5 p-4 rounded-xl border border-primary/20">
                  <p className="text-xs font-bold text-primary uppercase tracking-widest mb-2 flex items-center gap-2"><Bot className="w-3 h-3" /> AI Agent Response</p>
                  <p className="text-sm text-primary-foreground leading-relaxed">{inq.aiResponse}</p>
                </div>
              </div>
            </div>
          </div>
        ))}
      </div>
    )}
  </div>
</div>
)}

{activeTab === 'financials' && (
  <div className="space-y-8 animate-in fade-in slide-in-from-bottom-5 duration-500">
    <div className="flex items-center justify-between mb-8">
      <div>
        <h3 className="text-2xl font-bold uppercase tracking-tight">AI Financial
          Controller</h3>
        <p className="text-xs text-white/60 uppercase tracking-widest mt-1">Autonomous
          Transaction Management</p>
      </div>
      <div className="bg-emerald-500/10 border border-emerald-500/20 px-4 py-2 rounded-xl flex items-center gap-3">
        <div className="w-2 h-2 bg-emerald-500 rounded-full animate-pulse" />
        <span className="text-[10px] font-bold text-emerald-500 uppercase tracking-widest">System Online: Auto-Pay Active</span>
      </div>
    </div>

    <div className="grid lg:grid-cols-3 gap-6">
      <div className="glass-card p-8 border-primary/20 bg-primary/5">
        <Bot className="w-8 h-8 text-primary mb-4" />
        <h4 className="text-xs font-bold uppercase tracking-widest mb-2">AI Bookkeeper</h4>
        <p className="text-[10px] text-white/60 leading-relaxed mb-6">Our integrated AI
          handles all invoice generation, tax estimation, and provider payouts automatically.</p>
        <div className="space-y-3">
          <div className="flex justify-between text-[8px] font-bold uppercase tracking-widest">
            <span className="text-white/40">Efficiency</span>

```

```

        <span className="text-primary">99.9%</span>
      </div>
      <div className="w-full bg-white/5 h-1 rounded-full overflow-hidden">
        <div className="bg-primary h-full w-[99.9%]" />
      </div>
    </div>
  </div>

  <div className="col-span-2 glass-card p-8">
    <h4 className="text-xs font-bold uppercase tracking-widest mb-6">Recent Automated
Payouts</h4>
    <div className="space-y-4">
      {[
        { id: 'TXN-901', to: 'Dr. Ramesh S.', amt: ' ₹12,400', status: 'COMPLETED', time:
'2 mins ago' },
        { id: 'TXN-902', to: 'Care Home A', amt: ' ₹45,000', status: 'PROCESSING', time:
'14 mins ago' },
        { id: 'TXN-903', to: 'Sarah Nurse', amt: ' ₹8,900', status: 'COMPLETED', time: '1
hour ago' },
      ]}.map(txn => (
        <div key={txn.id} className="flex justify-between items-center p-4 bg-white/5
rounded-2xl border border-white/5">
          <div className="flex items-center gap-4">
            <div className="w-10 h-10 rounded-xl bg-white/5 flex items-center justify-
center text-[10px] font-bold text-white/60">{txn.id.split('-')[1]}</div>
            <div>
              <p className="text-xs font-bold">{txn.to}</p>
              <p className="text-[8px] text-white/50 uppercase tracking-widest">{txn.id}
</p>
            </div>
          </div>
          <div>
            <div className="text-right">
              <p className="text-sm font-bold text-primary">{txn.amt}</p>
              <p className={`text-[8px] font-black uppercase tracking-widest ${txn.status
=== 'COMPLETED' ? 'text-emerald-500' : 'text-amber-500'}`}>{txn.status}</p>
            </div>
          </div>
        </div>
      )]}
    </div>
  </div>
</div>
</div>
))}
</div>
</div>
</div>
</div>
))}

{isMasterAdmin && activeTab === 'bank' && (
  <div className="max-w-4xl animate-in fade-in slide-in-from-bottom-5 duration-500">
    <div className="mb-10">
      <h3 className="text-2xl font-bold uppercase tracking-tight">System Payout
Configuration</h3>
      <p className="text-xs text-white/60 uppercase tracking-widest mt-1">Bank details
presented to customers for transfers</p>
    </div>

    <div className="glass-card p-10 flex flex-col md:flex-row gap-10">
      <div className="flex-1 space-y-8">
        <div className="grid md:grid-cols-2 gap-8">
          <div>
            <label className="label-bold mb-3 block">Account Holder Name</label>
            <input
              type="text"
              value={bankDetails.accountHolder}
              onChange={(e) => setBankDetails({...bankDetails, accountHolder:
e.target.value})}
              className="w-full bg-white/5 border border-white/10 px-6 py-4
rounded-2xl text-sm outline-none focus:border-primary"
            />
          </div>
          <div>
            <label className="label-bold mb-3 block">Account Number</label>
            <input
              type="text"

```

```

        value={bankDetails.accountNumber}
        onChange={(e) => setBankDetails({...bankDetails, accountNumber:
e.target.value})}}
        className="w-full bg-white/5 border border-white/10 px-6 py-4
rounded-2xl font-mono text-sm outline-none focus:border-primary tracking-widest"
      />
    </div>
    <div>
      <label className="label-bold mb-3 block">IFSC Code</label>
      <input
        type="text"
        value={bankDetails.ifsc}
        onChange={(e) => setBankDetails({...bankDetails, ifsc: e.target.value})}
        className="w-full bg-white/5 border border-white/10 px-6 py-4
rounded-2xl font-mono text-sm uppercase outline-none focus:border-primary tracking-widest"
      />
    </div>
    <div>
      <label className="label-bold mb-3 block">Bank Name</label>
      <input
        type="text"
        value={bankDetails.bankName}
        onChange={(e) => setBankDetails({...bankDetails, bankName:
e.target.value})}}
        className="w-full bg-white/5 border border-white/10 px-6 py-4
rounded-2xl text-sm outline-none focus:border-primary"
      />
    </div>
  </div>

  <div className="pt-6 border-t border-white/5 flex gap-4">
    <Button variant="primary" className="flex-1 py-5 uppercase tracking-widest
text-xs">Update Linked Account</Button>
    <Button variant="outline" className="px-8 py-5 flex items-center gap-2
uppercase tracking-widest text-[10px]">
      <CheckCircle2 className="w-4 h-4" /> Verified
    </Button>
  </div>
</div>

<div className="w-full md:w-64 flex flex-col items-center justify-center p-6 bg-
white/5 rounded-3xl border border-white/5 text-center">
  <p className="text-[10px] font-bold text-white/50 uppercase tracking-widest
mb-4">Active Payment QR</p>
  <div className="bg-white p-3 rounded-2xl mb-4">
    
  </div>
  <p className="text-[9px] text-white/40 uppercase font-black tracking-
tighter">Protocol: UPI_INTERCEPT_V2</p>
</div>
</div>

<div className="mt-8 flex items-center gap-4 p-6 bg-primary/5 border border-
primary/20 rounded-3xl">
  <div className="w-12 h-12 bg-primary/20 rounded-2xl flex items-center justify-
center text-primary">
    <Shield className="w-6 h-6" />
  </div>
  <div>
    <p className="text-[10px] font-bold text-primary uppercase tracking-
widest">Secure Integration</p>
    <p className="text-xs text-white/80 leading-relaxed italic">Changes here are
verified by the Head Admin and synced across the AI payout network.</p>
  </div>
</div>

```

```

    </div>
  })

  {activeTab === 'sp-requests' && (
    <div className="space-y-10 animate-in fade-in slide-in-from-bottom-4 duration-500">
      <div className="mb-10">
        <h3 className="text-2xl font-bold uppercase tracking-tight">Service Provider
Requests</h3>
        <p className="text-xs text-white/60 uppercase tracking-widest mt-1">Verify new
providers and confirm registration.</p>
      </div>
      <div className="grid grid-cols-1 gap-6 pb-20">
        {providers.map(p => (
          <div key={p.id} className="glass-card p-8 border-white/5 flex flex-col md:flex-
row items-center justify-between gap-4">
            <div>
              <h4 className="font-bold flex items-center gap-2">
                {p.name || 'Anonymous'}
                {p.status && <span className={`text-[8px] px-2 py-0.5 rounded-full
uppercase tracking-widest ${p.status === 'Confirmed' ? 'bg-green-500/20 text-green-500' : p.status ===
'Rejected' ? 'bg-red-500/20 text-red-500' : 'bg-amber-500/20 text-amber-500'}`}>{p.status}</span>}
              </h4>
              <p className="text-[10px] text-white/60 font-bold uppercase tracking-
widest">Phone: {p.phone} • DB ID: {p.id}</p>
              <p className="text-[10px] text-primary font-bold uppercase tracking-widest
mt-1">SP ID NO: {p.spId || 'N/A'} • Education: {p.education || 'N/A'}</p>
            </div>
            <div className="flex gap-2">
              {p.isVerified ? (
                <span className="bg-primary/20 text-primary px-4 py-2 rounded-xl text-
[10px] font-bold uppercase tracking-widest">Verified by {p.verifiedBy}</span>
              ) : p.status === 'Rejected' ? (
                <span className="bg-red-500/20 text-red-500 px-4 py-2 rounded-xl text-
[10px] font-bold uppercase tracking-widest">Rejected</span>
              ) : (
                <>
                  <button
                    onClick={() => handleRejectSP(p)}
                    className="bg-red-500/10 text-red-500 border border-red-500/30 px-6
py-3 rounded-xl text-xs font-bold uppercase tracking-widest hover:bg-red-500 hover:text-white
transition-colors"
                  >
                    Reject
                  </button>
                  <button
                    onClick={() => handleVerifySP(p)}
                    className="bg-primary text-primary-foreground px-6 py-3 rounded-xl
text-xs font-bold uppercase tracking-widest hover:bg-primary/90 transition-colors"
                  >
                    Verify & Confirm
                  </button>
                </>
              )
            </div>
          </div>
        ))}
        {providers.length === 0 && (
          <p className="text-sm text-white/60">No providers found.</p>
        )}
      </div>
    </div>
  )}

  {activeTab === 'intel' && (
    <div className="space-y-10 animate-in fade-in slide-in-from-bottom-4 duration-500">
      <div className="flex justify-between items-center">
        <div>
          <h3 className="text-xl font-bold uppercase tracking-tight text-white">Partner
Intelligence</h3>
          <p className="text-[10px] text-white/60 uppercase tracking-widest mt-1">Silent
Surveillance & Performance Monitoring</p>

```

```

        </div>
        <div className="flex items-center gap-2 bg-red-500/10 px-4 py-2 rounded-full
border border-red-500/20">
            <Eye className="w-4 h-4 text-red-500 animate-pulse" />
            <span className="text-[10px] font-black text-red-500 uppercase tracking-widest
font-sans">Shadow Protocols Active</span>
        </div>
    </div>

    <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6 pb-20">
        {MOCK_PROVIDERS.map(p => (
            <div key={p.id} className="glass-card p-8 border-white/5 hover:border-primary/20
transition-all group relative">
                <div className="flex justify-between items-start mb-6">
                    <div className="flex items-center gap-4">
                        <div className="w-12 h-12 bg-white/5 rounded-2xl flex items-center
justify-center border border-white/10 group-hover:border-primary/30 transition-all overflow-hidden
relative">
                            <img src={(p as any).avatar} alt={p.name} className="w-full h-full
object-cover opacity-50 contrast-125" />
                            <div className="absolute inset-0 bg-red-500/10 opacity-0 group-
hover:opacity-100 transition-opacity" />
                        </div>
                        <div>
                            <h4 className="font-bold">{p.name}</h4>
                            <p className="text-[8px] text-white/50 uppercase font-black tracking-
widest">{p.category} • {p.location}</p>
                        </div>
                    </div>
                    <div className="flex items-center gap-1 bg-white/5 px-2 py-1 rounded-lg">
                        <Star className="w-2 h-2 text-primary fill-primary" />
                        <span className="text-[10px] font-bold">{p.rating}</span>
                    </div>
                </div>

                <div className="grid grid-cols-2 gap-4 mb-8">
                    <div className="p-3 bg-white/[0.02] rounded-xl border border-white/5">
                        <p className="text-[8px] font-black text-white/40 uppercase
mb-1">Status</p>

                        <div className="flex items-center gap-2">
                            <div className="w-1.5 h-1.5 bg-green-500 rounded-full animate-
pulse" />
                            <span className="text-[10px] font-bold uppercase transition-colors
group-hover:text-green-500">Online</span>
                        </div>
                    </div>
                    <div className="p-3 bg-white/[0.02] rounded-xl border border-white/5">
                        <p className="text-[8px] font-black text-white/40 uppercase
mb-1">Active jobs</p>

                        <span className="text-[10px] font-bold uppercase group-hover:text-
primary transition-colors">{bookings.filter((b: any) => b.providerName === p.name && b.status ===
"IN_PROGRESS").length}</span>
                    </div>
                </div>

                <Button
                    variant="primary"
                    className="w-full py-4 text-[9px] uppercase tracking-[0.2em] bg-white/5
border border-white/10 hover:bg-red-500 hover:text-white hover:border-red-500 transition-all font-
black flex items-center justify-center gap-3"
                    onClick={() => onSpectateProvider(p)}
                >
                    <Eye className="w-3.5 h-3.5" />
                    Spectate Dashboard
                </Button>
            </div>
        ))}
    </div>
</div>
})

```

```

{activeTab === 'data' && (
  <div className="space-y-12 animate-in fade-in slide-in-from-bottom-5 duration-500">
    <div className="flex flex-col md:flex-row gap-6 mb-8">
      <div className="flex-1 glass-card p-8 border-emerald-500/20 bg-emerald-500/5
group hover:bg-emerald-500/10 cursor-pointer transition-all" onClick={async () => {
        try {
          const headers = ['Action', 'Admin', 'Time', 'Type', 'Status'];
          const rows = propActions.map((a: any) => [a.action, a.admin, a.time,
a.type, a.status || 'LOG']);
          await exportToSheets(`CareVia_Logs_${new Date().toLocaleDateString()}`,
headers, rows);

          alert('Exported to Google Sheets successfully!');
        } catch (err: any) {
          alert(err.message);
        }
      }}>
      <div className="flex items-center justify-between mb-4">
        <FileSpreadsheet className="w-8 h-8 text-emerald-500 group-hover:scale-110
transition-transform" />
        <span className="text-[10px] font-black text-emerald-500 uppercase tracking-
widest bg-emerald-500/10 px-3 py-1 rounded-full">Excel Sync</span>
      </div>
      <h4 className="text-xl font-bold uppercase tracking-tight">Export Audit
Records</h4>
      <p className="text-[10px] text-white/60 uppercase tracking-widest mt-2 leading-
relaxed">Direct export of all system logs and financial transactions to Google Sheets.</p>
    </div>

    <div className="flex-1 glass-card p-8 border-primary/20 bg-primary/5 group
hover:bg-primary/10 cursor-pointer transition-all" onClick={async () => {
        try {
          const content = JSON.stringify(propActions, null, 2);
          await uploadToDrive(`CareVia_Backup_${Date.now()}.json`, content,
'application/json');

          alert('Backup uploaded to Google Drive successfully!');
        } catch (err: any) {
          alert(err.message);
        }
      }}>
      <div className="flex items-center justify-between mb-4">
        <Cloud className="w-8 h-8 text-primary group-hover:scale-110 transition-
transform" />
        <span className="text-[10px] font-black text-primary uppercase tracking-
widest bg-primary/10 px-3 py-1 rounded-full">Secure Backup</span>
      </div>
      <h4 className="text-xl font-bold uppercase tracking-tight">Cloud Snapshot</h4>
      <p className="text-[10px] text-white/60 uppercase tracking-widest mt-2 leading-
relaxed">Secure JSON encrypted backup of existing system state to your Cloud Drive.</p>
    </div>

    <div className="flex-1 glass-card p-8 border-blue-500/20 bg-blue-500/5 group
hover:bg-blue-500/10 cursor-pointer transition-all" onClick={async () => {
        try {
          await addDoc(collection(db, 'adminNotifications'), {
            type: 'SOURCE_CODE_DOWNLOAD',
            message: `An admin has downloaded the application source code.`,
            adminId: auth.currentUser?.uid || 'Unknown',
            timestamp: Date.now()
          });
          await addDoc(collection(db, 'queries'), {
            type: 'system',
            action: `Source Code Downloaded`,
            details: `Admin ${auth.currentUser?.email || 'Unknown'} downloaded the
application source code.`,
            admin: auth.currentUser?.email || 'Unknown',
            createdAt: serverTimestamp(),
            time: new Date().toLocaleTimeString(),
            status: 'SUCCESS'
          });
          const link = document.createElement('a');
          link.href = '/Website_Source_Code.pdf';

```

```

        link.download = 'CAREVIA_Source_Code.pdf';
        link.click();
    } catch (err: any) {
        console.error(err);
    }
  })
  <div className="flex items-center justify-between mb-4">
    <Download className="w-8 h-8 text-blue-500 group-hover:scale-110 transition-
transform" />
    <span className="text-[10px] font-black text-blue-500 uppercase tracking-
widest bg-blue-500/10 px-3 py-1 rounded-full">Secure File</span>
  </div>
  <h4 className="text-xl font-bold uppercase tracking-tight">Source Code (PDF)</
h4>
  <p className="text-[10px] text-white/60 uppercase tracking-widest mt-2 leading-
relaxed">Download the encrypted project architecture and source files as PDF.</p>
</div>

  <div className="flex-1 glass-card p-8 border-purple-500/20 bg-purple-500/5 group
hover:bg-purple-500/10 cursor-pointer transition-all" onClick={async () => {
    try {
      // 1. Sync Bookings to Google Sheets
      const headers = ['ID', 'Customer', 'Service', 'Status', 'Date', 'Time',
'Location', 'Amount'];
      const rows = bookings.map((b: any) => [b.id, b.customer, b.service,
b.status, b.date, b.time, b.location, b.amount || 0]);
      await exportToSheets(`CareVia_Bookings_Sync_${new
Date().toLocaleDateString()}`, headers, rows);

      // 2. Add to Google Cloud (Drive)
      const content = JSON.stringify(bookings, null, 2);
      await uploadToDrive(`CareVia_Bookings_Cloud_Sync_${Date.now()}.json`,
content, 'application/json');

      // 3. Log to Google Cloud Firestore
      await addDoc(collection(db, 'adminNotifications'), {
        type: 'SYNC_AND_ADD',
        message: `Admin synced data with Google Sheets and Google Cloud.`,
        adminId: auth.currentUser?.uid || 'Unknown',
        timestamp: Date.now()
      });
      await addDoc(collection(db, 'queries'), {
        type: 'system',
        action: `Data Synced`,
        details: `Admin ${auth.currentUser?.email || 'Unknown'} synced data with
Google Sheets.`,
        admin: auth.currentUser?.email || 'Unknown',
        createdAt: serverTimestamp(),
        time: new Date().toLocaleTimeString(),
        status: 'SUCCESS'
      });

      alert('Sync and Add with Google Sheets and Google Cloud successfully
completed!');
    } catch (err: any) {
      alert(err.message);
    }
  })
  <div className="flex items-center justify-between mb-4">
    <Database className="w-8 h-8 text-purple-500 group-hover:scale-110
transition-transform" />
    <span className="text-[10px] font-black text-purple-500 uppercase tracking-
widest bg-purple-500/10 px-3 py-1 rounded-full">Google Cloud Sync</span>
  </div>
  <h4 className="text-xl font-bold uppercase tracking-tight">Sync & Add</h4>
  <p className="text-[10px] text-white/60 uppercase tracking-widest mt-2 leading-
relaxed">Direct integration: Syncs system bookings with Google Sheets and backs up to Google Cloud
Drive.</p>
</div>
</div>

```

```

        <div className="glass-card p-8">
            <div className="flex flex-col md:flex-row justify-between items-start md:items-
center mb-8 gap-4">
                <div>
                    <h4 className="text-xs font-bold uppercase tracking-widest text-primary
mb-1">Recent Action History</h4>
                    <p className="text-[8px] text-white/50 uppercase tracking-widest">Protocol
Audit Logs</p>
                </div>
                <div className="flex flex-wrap items-center gap-4 bg-white/5 p-3 rounded-2xl
border border-white/5">
                    <div className="flex items-center gap-2">
                        <User className="w-3 h-3 text-white/40" />
                        <select
                            value={filterAdmin}
                            onChange={(e) => setFilterAdmin(e.target.value)}
                            className="bg-transparent text-[9px] font-bold text-white/80 outline-
none cursor-pointer uppercase tracking-widest"
                        >
                            {admins.map(adm => <option key={adm} value={adm} className="bg-
surface">{adm}</option>)}
                        </select>
                    </div>
                    <div className="w-px h-4 bg-white/10 hidden md:block" />
                    <div className="flex items-center gap-2">
                        <Shield className="w-3 h-3 text-white/40" />
                        <select
                            value={filterType}
                            onChange={(e) => setFilterType(e.target.value)}
                            className="bg-transparent text-[9px] font-bold text-white/80 outline-
none cursor-pointer uppercase tracking-widest"
                        >
                            {types.map(t => <option key={t} value={t} className="bg-surface">{t}</
option>)}
                        </select>
                    </div>
                    <div className="w-px h-4 bg-white/10 hidden md:block" />
                    <div className="flex items-center gap-2 flex-1 min-w-[120px]">
                        <Search className="w-3 h-3 text-white/40" />
                        <input
                            type="text"
                            placeholder="SEARCH LOGS / SESSIONS..."
                            value={searchQuery}
                            onChange={(e) => setSearchQuery(e.target.value)}
                            className="bg-transparent text-[9px] font-bold text-white/80 outline-
none uppercase tracking-widest w-full"
                        />
                    </div>
                    <div className="w-px h-4 bg-white/10 hidden md:block" />
                    <div className="flex items-center gap-3">
                        <div className="flex items-center gap-2">
                            <Calendar className="w-3 h-3 text-white/40" />
                            <input
                                type="date"
                                value={startDate}
                                onChange={(e) => setStartDate(e.target.value)}
                                className="bg-transparent text-[9px] font-bold text-white/80
outline-none cursor-pointer uppercase invert opacity-80"
                            />
                        </div>
                        <span className="text-white/40 text-[9px]">—</span>
                        <input
                            type="date"
                            value={endDate}
                            onChange={(e) => setEndDate(e.target.value)}
                            className="bg-transparent text-[9px] font-bold text-white/80 outline-
none cursor-pointer uppercase invert opacity-80"
                        />
                    </div>
                    <div className="w-px h-4 bg-white/10 hidden md:block" />

```

```

        <button
            onClick={() => setSortOrder(prev => prev === 'desc' ? 'asc' : 'desc')}
            className="flex items-center gap-2 text-[9px] font-bold text-primary
uppercase tracking-widest hover:text-white transition-colors"
        >
            {sortOrder === 'desc' ? 'Newest' : 'Oldest'}
            <ArrowUpDown className="w-3 h-3" />
        </button>
    </div>
</div>

<div className="space-y-4 max-h-[400px] overflow-y-auto pr-2 custom-scrollbar">
    {filteredAndSortedActions.length > 0 ? filteredAndSortedActions.map(act => (
        <motion.div
            initial={{ opacity: 0, x: -10 }}
            animate={{ opacity: 1, x: 0 }}
            key={act.id}
            className="flex justify-between items-center p-4 bg-white/5 rounded-2xl
border border-white/5 hover:bg-white/10 transition-all group"
        >
            <div className="flex items-center gap-4">
                <div className={`w-10 h-10 rounded-xl flex items-center justify-center
font-bold text-[10px] ${
                    act.type === 'ai' ? 'bg-primary/20 text-primary border border-
primary/20' :
                    act.type === 'system' ? 'bg-amber-500/20 text-amber-500 border
border-amber-500/20' :
                    'bg-white/5 text-white/60 border border-white/5'
                }`}>
                    {act.type === 'ai' ? <Brain className="w-5 h-5" /> : (act.admin ?
act.admin.substring(0, 2).toUpperCase() : '??')}
                </div>
                <div className="flex flex-col">
                    <div className="flex items-center gap-2">
                        <span className="text-sm font-bold">{act.action}</span>
                        {act.status && (
                            <span className={`text-[8px] px-1 rounded border ${
                                act.status === 'ERROR' ? 'border-red-500/40 text-red-400 bg-
red-400/10' :
                                act.status === 'SUCCESS' ? 'border-emerald-500/40 text-
emerald-400 bg-emerald-400/10' :
                                'border-primary/40 text-primary bg-primary/10'
                            }`}>
                                {act.status}
                            </span>
                        )}
                    </div>
                    <div className="flex items-center gap-2 mt-0.5">
                        <span className="text-[10px] text-white/50 uppercase tracking-
widest">{act.admin}</span>
                        {act.sessionId && (
                            <span className="text-[8px] text-primary/60 font-mono tracking-
tighter bg-primary/5 px-2 py-0.5 rounded border border-primary/10">ID: {act.sessionId.substring(0, 8)}
                        </span>
                        )}
                    </div>
                    {act.details && (
                        <p className="text-[9px] text-white/60 mt-3 font-mono bg-
black/20 p-3 rounded-xl border border-white/5 leading-relaxed max-w-[600px] break-words">
                            {act.details}
                        </p>
                    )}
                </div>
            </div>
            <div className="flex flex-col items-end gap-2 text-right">
                <span className="block text-[10px] text-white/40 font-mono tracking-
tighter">{act.time}</span>
                <span className="text-[8px] text-primary/60 uppercase font-black
tracking-widest">{act.type?.toUpperCase() || 'LOG'}</span>
                {isMasterAdmin && (
                    <button

```

```

                                onClick={() => alert(`Action "${act.action}" has been forcefully
stopped/undone by Master Admin.`)}
                                className="bg-red-500/20 text-red-500 hover:bg-red-500/40 px-3
py-1 rounded text-[8px] font-bold uppercase tracking-widest transition-colors mt-2"
                                >
                                    UNDO
                                </button>
                            )}
                        </div>
                    </motion.div>
                )) : (
                    <div className="py-20 text-center opacity-30">
                        <Search className="w-10 h-10 mx-auto mb-4" />
                        <p className="text-[10px] uppercase font-bold tracking-widest">No
matching records found</p>
                    </div>
                )}
            </div>
            <div className="glass-card p-8">
                <h4 className="text-xs font-bold uppercase tracking-widest text-primary
mb-6">Data Monitoring</h4>
                <div className="h-48 flex items-end justify-between gap-4">
                    {[40, 70, 45, 90, 100, 60, 80].map((h, i) => (
                        <div key={i} className="flex-1 bg-primary/20 rounded-t-lg relative group
transition-all hover:bg-primary/40" style={{ height: `${h}%` }}>
                            <div className="absolute -top-8 left-1/2 -translate-x-1/2 bg-primary text-
surface p-1 rounded text-[8px] font-bold opacity-0 group-hover:opacity-100 transition-opacity">v{h}</
div>
                        </div>
                    )]}
                </div>
                <div className="flex justify-between mt-4 text-[8px] font-bold text-white/40
uppercase tracking-widest">
                    <span>Mon</span><span>Tue</span><span>Wed</span><span>Thu</span><span>Fri</
span><span>Sat</span><span>Sun</span>
                </div>
            </div>

            <div className="grid lg:grid-cols-2 gap-12">
                <div className="glass-card p-8 border-primary/20 bg-primary/5">
                    <div className="flex items-center gap-4 mb-6">
                        <div className="w-10 h-10 bg-primary/20 rounded-xl flex items-center justify-
center text-primary">
                            <Shield className="w-5 h-5" />
                        </div>
                        <div>
                            <h4 className="text-xs font-bold uppercase tracking-widest text-
primary">Head Admin Protocol</h4>
                            <p className="text-[10px] text-white/60 mt-1 uppercase tracking-
widest">Data Sharing Control</p>
                        </div>
                    </div>
                    <p className="text-sm text-white/90 mb-8 leading-relaxed">
                        Establish the global protocol for data extraction and external sharing.
Minimal setting restricts all AI-powered data mining.
                    </p>
                    <div className="grid grid-cols-3 gap-3">
                        [['Full', 'Restricted', 'Minimal'].map(level => (
                            <button
                                key={level}
                                onClick={() => setSharingLevel(level)}
                                className={`py-4 rounded-xl text-[10px] font-black uppercase tracking-
widest border transition-all ${sharingLevel === level ? 'bg-primary text-surface border-primary' : 'bg-
white/5 border-white/10 text-white/50 hover:bg-white/10'}`]
                                >
                                    {level}
                                </button>
                            )
                        )]}
                    </div>
                </div>
            </div>

```

```

<div className="glass-card p-8">
  <div className="flex items-center gap-4 mb-6">
    <div className="w-10 h-10 bg-blue-500/20 rounded-xl flex items-center justify-
center text-blue-500">
      <Activity className="w-5 h-5" />
    </div>
    <div>
      <h4 className="text-xs font-bold uppercase tracking-widest text-
blue-500">Live Data Feed</h4>
      <p className="text-[10px] text-white/60 mt-1 uppercase tracking-
widest">System Health & API Usage</p>
    </div>
  </div>
  <div className="space-y-4">
    <div className="flex justify-between items-center text-[10px] uppercase font-
bold tracking-widest">
      <span className="text-white/60">API Requests</span>
      <span className="text-primary">Healthy</span>
    </div>
    <div className="w-full bg-white/5 h-2 rounded-full overflow-hidden">
      <motion.div initial={{ width: 0 }} animate={{ width: '65%' }}
className="bg-primary h-full" />
    </div>
    <div className="flex justify-between items-center text-[10px] uppercase font-
bold tracking-widest">
      <span className="text-white/60">Data Encryption</span>
      <span className="text-green-500">AES-256 Active</span>
    </div>
    <div className="w-full bg-white/5 h-2 rounded-full overflow-hidden">
      <motion.div initial={{ width: 0 }} animate={{ width: '100%' }}
className="bg-green-500 h-full" />
    </div>
  </div>
</div>

<div className="glass-card overflow-hidden">
  <div className="p-8 border-b border-white/5 bg-white/[0.02] flex justify-between
items-center">
    <div>
      <h4 className="text-xs font-bold uppercase tracking-widest text-primary
mb-1">Team Data Access Management</h4>
      <p className="text-[10px] text-white/50 uppercase tracking-widest">Define
granular access levels for administrative staff</p>
    </div>
    <div className="flex gap-4">
      <span className="text-[10px] font-bold text-white/60 uppercase tracking-
widest">Global Override:</span>
      <button
        onClick={() => setTeamAccess(prev => prev === 'Full' ? 'Read-Only' :
'Full')}
        className={`px-3 py-1 rounded text-[8px] font-black uppercase tracking-
tighter ${teamAccess === 'Full' ? 'bg-green-500/20 text-green-500' : 'bg-red-500/20 text-red-500'}`}
      >
        {teamAccess === 'Full' ? 'Open' : 'Restricted'}
      </button>
    </div>
  </div>
  <div className="overflow-x-auto">
    <table className="w-full text-left">
      <thead>
        <tr className="border-b border-white/5 text-[10px] font-bold uppercase
tracking-widest text-white/50">
          <th className="p-6">Member</th>
          <th className="p-6">Position</th>
          <th className="p-6 text-center">Protocol Level</th>
          <th className="p-6 text-right">Action</th>
        </tr>
      </thead>
      <tbody className="divide-y divide-white/5">

```

```

        {teamMembers.map(member => (
            <tr key={member.id} className="hover:bg-white/[0.01] transition-colors">
                <td className="p-6">
                    <div className="flex flex-col">
                        <span className="text-sm font-bold">{member.name}</span>
                        <span className="text-[11px] text-white/50">{member.email}</span>
                    </div>
                </td>
                <td className="p-6 text-xs text-white/80 font-medium">{member.role}</td>
                <td className="p-6 text-center">
                    <span className={`px-3 py-1 rounded-full text-[9px] font-black uppercase tracking-widest ${member.level === 'Full' ? 'bg-primary/20 text-primary' : member.level === 'Standard' ? 'bg-blue-500/20 text-blue-500' : 'bg-white/5 text-white/50'}`}>
                        {member.level}
                    </span>
                </td>
                <td className="p-6 text-right">
                    <button className="text-primary hover:underline text-[10px] font-bold uppercase tracking-widest">Permissions</button>
                </td>
            </tr>
        ))}
    </tbody>
</table>
</div>
</div>

<div className="glass-card p-8 border-red-500/10 bg-red-500/[0.02] text-left">
    <div className="flex items-center justify-between mb-8">
        <h3 className="text-lg font-bold uppercase tracking-tight text-red-500">Critical Access Alerts</h3>
        <button onClick={() => setActiveTab('security')} className="text-[8px] font-bold text-red-500 uppercase tracking-widest hover:underline">View All</button>
    </div>
    <div className="space-y-3">
        {securityAlerts.slice(0, 3).map((alert) => (
            <div key={alert.id} className="p-4 bg-white/5 border border-white/5 rounded-2xl text-left">
                <div className="flex justify-between items-start mb-2">
                    <span className="text-[8px] font-black text-red-500 uppercase tracking-widest px-2 py-0.5 bg-red-500/10 rounded">Unauthorized Attempt</span>
                    <span className="text-[8px] font-bold text-white/40 uppercase">{alert.date}</span>
                </div>
                <div className="flex justify-between items-end">
                    <div>
                        <p className="text-xs font-bold">{alert.number}</p>
                        <p className="text-[8px] text-white/60 uppercase tracking-widest mt-1">ID: {alert.detailId}</p>
                    </div>
                    <div className="text-right">
                        <p className="text-[10px] font-mono text-white">OTP: {alert.otp}</p>
                        <p className="text-[8px] font-mono text-white/60">PW: {alert.pass}</p>
                    </div>
                </div>
            </div>
        ))}
    </div>
</div>

<div className="glass-card p-8 border-red-500/10 bg-red-500/[0.02] text-left">
    <div className="flex items-center justify-between mb-8">
        <h3 className="text-lg font-bold uppercase tracking-tight text-red-500">Critical Access Alerts</h3>
        <button onClick={() => setActiveTab('security')} className="text-[8px] font-bold text-red-500 uppercase tracking-widest hover:underline">View All</button>
    </div>
    <div className="space-y-3">

```

```

        {securityAlerts.slice(0, 3).map((alert) => (
          <div key={alert.id} className="p-4 bg-white/5 border border-white/5
rounded-2xl text-left">
            <div className="flex justify-between items-start mb-2">
              <span className="text-[8px] font-black text-red-500 uppercase tracking-
widest px-2 py-0.5 bg-red-500/10 rounded">Unauthorized Attempt</span>
              <span className="text-[8px] font-bold text-white/40
uppercase">{alert.date}</span>
            </div>
            <div className="flex justify-between items-end">
              <div>
                <p className="text-xs font-bold">{alert.number}</p>
                <p className="text-[8px] text-white/60 uppercase tracking-widest
mt-1">ID: {alert.detailId}</p>
              </div>
              <div className="text-right">
                <p className="text-[10px] font-mono text-white">OTP: {alert.otp}</p>
                <p className="text-[8px] font-mono text-white/60">PW: {alert.pass}</p>
              </div>
            </div>
          </div>
        ))}
      </div>
    </div>
  )}
</div>
</div>
</div>
);
};

const WalletSection = ({ onManage }: { onManage?: () => void }) => (
  <div className="glass p-8 rounded-[2.5rem] flex flex-col sm:flex-row items-center justify-between
font-sans border-white/5 gap-8">
    <div className="flex items-center gap-6 w-full sm:w-auto">
      <div className="w-16 h-16 bg-primary/20 rounded-2xl flex items-center justify-center text-
primary flex-shrink-0">
        <Wallet className="w-8 h-8" />
      </div>
      <div>
        <h4 className="text-sm font-black uppercase tracking-widest opacity-40">CARVIA Wallet</h4>
        <p className="text-3xl font-bold mt-1">¹⁰.⁰⁰</p>
      </div>
    </div>
    <div className="text-center sm:text-right w-full sm:w-auto">
      <p className="text-[10px] uppercase font-bold text-white/50 tracking-widest mb-2">Rewards
Active</p>
      <Button variant="outline" className="w-full sm:w-auto px-6 py-3 text-[10px] uppercase tracking-
widest" onClick={onManage}>Manage Balance</Button>
    </div>
  </div>
);

const LiveAlarm = ({ bookings, financialAlerts = [], isAdmin, isSP, onAcknowledge }: any) => {
  const [lastCount, setLastCount] = useState(bookings.length);
  const [lastFinancialCount, setLastFinancialCount] = useState(financialAlerts.length);

```

```

const [show, setShow] = useState(false);
const [currentBooking, setCurrentBooking] = useState<any>(null);
const [currentFinancial, setCurrentFinancial] = useState<any>(null);

useEffect(() => {
  if (financialAlerts.length > lastFinancialCount && isAdmin) {
    const latest = financialAlerts[0];
    setCurrentFinancial(latest);
    setCurrentBooking(null);
    setShow(true);
    const audio = new Audio('https://assets.mixkit.co/active_storage/sfx/2869/2869-preview.mp3');
    audio.play().catch(e => console.log("Sound blocked by browser:", e));
  }
  setLastFinancialCount(financialAlerts.length);
}, [financialAlerts.length, isAdmin]);

useEffect(() => {
  if (bookings.length > lastCount) {
    const latest = bookings[0];
    if ((isAdmin && latest.status === 'PENDING') || (isSP && latest.status === 'PENDING')) {
      setCurrentFinancial(null);
      setCurrentBooking(latest);
      setShow(true);
      const audio = new Audio('https://assets.mixkit.co/active_storage/sfx/2869/2869-preview.mp3');
      audio.play().catch(e => console.log("Sound blocked by browser:", e));
    }
  }
  setLastCount(bookings.length);
}, [bookings.length, isAdmin, isSP]);

// Check for status changes (e.g. SP Accepted)
useEffect(() => {
  if (isAdmin) {
    const accepted = bookings.find((b: any) => b.status === 'ACCEPTED_BY_SP' && !b.adminNotified);
    if (accepted) {
      setCurrentFinancial(null);
      setCurrentBooking(accepted);
      setShow(true);
      const audio = new Audio('https://assets.mixkit.co/active_storage/sfx/2869/2869-preview.mp3');
      audio.play().catch(e => console.log("Sound blocked by browser:", e));
      // In a real app, we'd mark it as notified in DB
    }
  }
}, [bookings, isAdmin]);

if (!show) return null;

const isFinancial = !!currentFinancial;

return (
  <motion.div
    initial={{ y: 100, opacity: 0 }}
    animate={{ y: 0, opacity: 1 }}
    className={`fixed bottom-10 right-10 z-[500] w-96 glass-card p-6 ${isFinancial ? 'border-amber-500 bg-amber-500/10 shadow-[0_0_50px_rgba(245,158,11,0.5)]' : 'border-primary bg-primary/10 shadow-[0_0_50px_rgba(20,184,166,0.5)]'}` `}
  >
    <div className="flex items-center gap-4 mb-4">
      <div className={`w-12 h-12 rounded-full flex items-center justify-center text-surface animate-bounce ${isFinancial ? 'bg-amber-500' : 'bg-primary'}`} `>
        {isFinancial ? <Wallet className="w-6 h-6" /> : <AlertTriangle className="w-6 h-6" />}
      </div>
      <div>
        <h4 className={`text-sm font-black uppercase tracking-widest ${isFinancial ? 'text-amber-500' : 'text-primary'}`} `>
          {isFinancial ? 'Financial Intercept' : 'Critical Alert'}
        </h4>
        <p className="text-xs font-bold text-white uppercase tracking-tight">
          {isFinancial ? currentFinancial.type : currentBooking?.service}
        </p>
      </div>
    </div>
  </div>
)

```

```

    </div>
    <p className="text-[10px] text-white/80 mb-6 font-bold uppercase tracking-widest leading-relaxed">
      {isFinancial ? currentFinancial.details : (isAdmin ? 'A partner has accepted a contract. Approval required.' : 'New service request detected in your sector.')}
    </p>
    <div className="flex gap-3">
      <Button
        variant={isFinancial ? "outline" : "primary"}
        className={`flex-1 py-3 text-[9px] ${isFinancial ? 'border-amber-500/40 text-amber-500' : ''}}`
        onClick={() => { setShow(false); onAcknowledge(); }}
      >
        {isFinancial ? 'Audit Transaction' : 'View Assignment'}
      </Button>
      <Button variant="outline" className="py-3 text-[9px] px-4" onClick={() => setShow(false)}><X
        className="w-4 h-4" /></Button>
    </div>
  </motion.div>
);
};

// --- Main App ---

const ReviewsSection = () => {
  const reviews: any[] = [];

  return (
    <section className="py-24 max-w-7xl mx-auto px-6 font-sans">
      <SectionHeading
        sub="Testimonials"
        title="Community Feedback."
        desc="Hear from our community of satisfied families and dedicated service providers."
        centered
      />
      <div className="grid md:grid-cols-3 gap-8">
        {reviews.map((rev, i) => (
          <div key={i} className="glass-card p-8 group">
            <div className="flex gap-1 mb-4 text-primary">
              {[...Array(rev.rating)].map((_, i) => <Star key={i} className="w-3 h-3 fill-current" />)}
            </div>
            <p className="text-sm text-white/90 italic mb-6 leading-loose">{rev.text}</p>
            <div className="flex justify-between items-center">
              <div>
                <p className="text-sm font-bold">{rev.name}</p>
                <p className="text-[10px] uppercase font-bold text-primary/80 tracking-widest">{rev.role}</p>
              </div>
              <span className="text-[10px] text-white/40 font-bold">{rev.date}</span>
            </div>
          </div>
        ))}
      </div>
      <div className="mt-12 text-center">
        <Button variant="outline" className="text-xs">Submit Your Review</Button>
      </div>
    </section>
  );
};

const ProviderRegistrationModal = ({ isOpen, onClose }: any) => {
  const [step, setStep] = useState(1);
  const [form, setForm] = useState({
    name: '',
    phone: '',
    address: '',
    education: '',
    aadhaar: null as File | null,
    pan: null as File | null,
  });
};

```

```

const handleFileChange = (field: 'aadhaar' | 'pan', file: File | null) => {
  setForm(prev => ({ ...prev, [field]: file }));
};

const handleRegister = async () => {
  if (!form.name || !form.phone) return alert("Please provide at least Name and Phone Number");
  try {
    await addDoc(collection(db, 'users'), {
      role: 'provider',
      name: form.name,
      phone: form.phone,
      address: form.address,
      education: form.education,
      isVerified: false,
      status: 'Pending Review',
      createdAt: serverTimestamp()
    });
    setStep(2);
  } catch(e) {
    console.error(e);
    alert("Registration failed");
  }
};

if (!isOpen) return null;

return (
  <div className="fixed inset-0 z-[250] flex items-center justify-center p-6 font-sans">
    <motion.div
      initial={{ opacity: 0 }}
      animate={{ opacity: 1 }}
      onClick={onClose}
      className="absolute inset-0 bg-black/80 backdrop-blur-md"
    />
    <motion.div
      initial={{ opacity: 0, scale: 0.9, y: 30 }}
      animate={{ opacity: 1, scale: 1, y: 0 }}
      className="glass-card w-full max-w-2xl relative z-10 overflow-hidden max-h-[90vh] flex flex-
col"
    >
      <div className="p-8 border-b border-white/5 bg-primary/5 flex justify-between items-center">
        <div className="flex items-center gap-4">
          <div className="w-12 h-12 bg-primary/20 rounded-2xl flex items-center justify-center
text-primary">
            <Handshake className="w-6 h-6" />
          </div>
          <div>
            <p className="text-[10px] font-bold text-primary uppercase tracking-[0.2em]
mb-1">Partner with us</p>
            <h3 className="text-xl font-bold">Join as Service Provider</h3>
          </div>
        </div>
        <button onClick={onClose} className="p-2 hover:bg-white/10 rounded-full"><X className="w-5
h-5 text-white/60" /></button>
      </div>

      <div className="flex-1 overflow-y-auto p-8">
        {step === 1 ? (
          <div className="space-y-8">
            <div className="grid md:grid-cols-2 gap-6">
              <div>
                <label className="label-bold mb-2 block">Full Name</label>
                <input
                  type="text"
                  value={form.name}
                  onChange={(e) => setForm({...form, name: e.target.value})}
                  className="w-full bg-white/5 border border-white/10 px-4 py-3 rounded-xl text-
sm focus:border-primary outline-none transition-colors"
                  placeholder="John Doe"
                />
              </div>
            </div>

```

```

<div>
  <label className="label-bold mb-2 block">Phone Number</label>
  <input
    type="tel"
    value={form.phone}
    onChange={(e) => setForm({...form, phone: e.target.value})}
    className="w-full bg-white/5 border border-white/10 px-4 py-3 rounded-xl text-
sm focus:border-primary outline-none transition-colors"
    placeholder="9999999999"
  />
</div>
<div className="md:col-span-2">
  <label className="label-bold mb-2 block">Educational Qualification</label>
  <div className="relative">
    <GraduationCap className="absolute left-4 top-1/2 -translate-y-1/2 w-4 h-4
text-white/50" />
    <input
      type="text"
      value={form.education}
      onChange={(e) => setForm({...form, education: e.target.value})}
      className="w-full bg-white/5 border border-white/10 px-10 py-3 rounded-xl
text-sm focus:border-primary outline-none transition-colors"
      placeholder="B.Sc Nursing / MBA"
    />
  </div>
</div>
</div>

<div>
  <label className="label-bold mb-2 block">Complete Residential Address</label>
  <textarea
    value={form.address}
    onChange={(e) => setForm({...form, address: e.target.value})}
    className="w-full bg-white/5 border border-white/10 px-4 py-3 rounded-xl text-sm
focus:border-primary outline-none transition-colors min-h-[100px]"
    placeholder="Enter your full address for verification"
  />
</div>

<div className="grid md:grid-cols-2 gap-8">
  <div className="space-y-3">
    <label className="label-bold block">Aadhaar Card (Front/Back)</label>
    <div
      className={`border-2 border-dashed rounded-2xl p-6 text-center transition-all
${form.aadhaar ? 'border-primary bg-primary/5' : 'border-white/10 hover:border-primary/50'}}
      onDragOver={(e) => e.preventDefault()}
      onDrop={(e) => {
        e.preventDefault();
        handleFileChange('aadhaar', e.dataTransfer.files[0]);
      }}
    >
      <Upload className={`w-8 h-8 mx-auto mb-2 ${form.aadhaar ? 'text-primary' :
'text-white/40'}} />
      <p className="text-[10px] text-white/60 uppercase font-black">
        {form.aadhaar ? form.aadhaar.name : 'Drop Aadhaar Image or Click'}
      </p>
      <input
        type="file"
        className="hidden"
        id="aadhaar-upload"
        onChange={(e) => handleFileChange('aadhaar', e.target.files?.[0] || null)}
      />
      <label htmlFor="aadhaar-upload" className="mt-4 inline-block px-4 py-2 bg-
white/5 rounded-lg text-[9px] font-bold uppercase tracking-widest cursor-pointer hover:bg-
white/10">Browse Files</label>
    </div>
  </div>

  <div className="space-y-3">
    <label className="label-bold block">PAN Card</label>
    <div

```

```

        className={`border-2 border-dashed rounded-2xl p-6 text-center transition-all
        ${form.pan ? 'border-primary bg-primary/5' : 'border-white/10 hover:border-primary/50'}`}
        onDragOver={(e) => e.preventDefault()}
        onDrop={(e) => {
            e.preventDefault();
            handleFileChange('pan', e.dataTransfer.files[0]);
        }}
    >
        <FileText className={`w-8 h-8 mx-auto mb-2 ${form.pan ? 'text-primary' :
'text-white/40'}`} />

        <p className="text-[10px] text-white/60 uppercase font-black">
            {form.pan ? form.pan.name : 'Drop PAN Image or Click'}
        </p>
        <input
            type="file"
            className="hidden"
            id="pan-upload"
            onChange={(e) => handleFileChange('pan', e.target.files?.[0] || null)}
        />
        <label htmlFor="pan-upload" className="mt-4 inline-block px-4 py-2 bg-white/5
rounded-lg text-[9px] font-bold uppercase tracking-widest cursor-pointer hover:bg-white/10">Browse
Files</label>

        </div>
    </div>
</div>

    <div className="bg-primary/5 p-4 rounded-xl flex items-start gap-4">
        <ShieldCheck className="w-5 h-5 text-primary flex-shrink-0" />
        <p className="text-[10px] text-white/70 leading-relaxed uppercase font-bold
tracking-widest">
            Your identity documents are used strictly for background verification purposes
            and encrypted with Tier-1 security protocols.
        </p>
    </div>
    </div>
    ) : (
        <div className="text-center py-12 px-8 space-y-6">
            <div className="w-24 h-24 bg-green-500/20 rounded-full flex items-center justify-
center mx-auto mb-8">
                <ShieldCheck className="w-12 h-12 text-green-500 animate-pulse" />
            </div>
            <h4 className="text-2xl font-bold uppercase tracking-tight">Application Received!</h4>
            <p className="text-sm text-white/70 leading-relaxed max-w-md mx-auto">
                Our vetting team will verify your credentials and documents within <span
                className="text-primary font-bold">24-48 hours</span>. We will contact you at your registered mobile
                number for a quick orientation.
            </p>
            <div className="grid grid-cols-2 gap-4 max-w-xs mx-auto pt-6">
                <div className="p-4 bg-white/5 rounded-xl border border-white/10">
                    <span className="block text-[8px] uppercase font-bold text-white/50
mb-1">Status</span>
                    <span className="text-[10px] font-bold text-primary uppercase">Pending Review</
span>
                </div>
                <div className="p-4 bg-white/5 rounded-xl border border-white/10">
                    <span className="block text-[8px] uppercase font-bold text-white/50
mb-1">Vetting ID</span>
                    <span className="text-[10px] font-bold text-white uppercase">#PV-9201</span>
                </div>
            </div>
        </div>
    )}
</div>

    <div className="p-8 border-t border-white/5 bg-black/20">
        {step === 1 ? (
            <Button
                variant="primary"
                className="w-full py-4 uppercase tracking-[0.2em] text-xs"
                onClick={handleRegister}
            >

```

```

        Submit Application
      </Button>
    ) : (
      <Button
        variant="outline"
        className="w-full py-4 uppercase tracking-[0.2em] text-xs"
        onClick={onClose}
      >
        Close Portal
      </Button>
    )}
  </div>
</motion.div>
</div>
);
};

const BookingModal = ({ isOpen, onClose, serviceName }: any) => {
  const [step, setStep] = useState(1);
  const [date, setDate] = useState('');
  const [address, setAddress] = useState('');

  if (!isOpen) return null;

  return (
    <div className="fixed inset-0 z-[250] flex items-center justify-center p-6 font-sans">
      <motion.div
        initial={{ opacity: 0 }}
        animate={{ opacity: 1 }}
        onClick={onClose}
        className="absolute inset-0 bg-black/80 backdrop-blur-md"
      />
      <motion.div
        initial={{ opacity: 0, scale: 0.9, y: 30 }}
        animate={{ opacity: 1, scale: 1, y: 0 }}
        className="glass-card w-full max-w-lg relative z-10 overflow-hidden"
      >
        <div className="p-8 border-b border-white/5 bg-primary/5 flex justify-between items-center">
          <div>
            <p className="text-[10px] font-bold text-primary uppercase tracking-[0.2em] mb-1">Service Booking</p>
            <h3 className="text-xl font-bold">{serviceName}</h3>
          </div>
          <button onClick={onClose} className="p-2 hover:bg-white/10 rounded-full"><X className="w-5 h-5 text-white/60" /></button>
        </div>

        <div className="p-8 space-y-6">
          {step === 1 ? (
            <motion.div initial={{ opacity: 0, x: 20 }} animate={{ opacity: 1, x: 0 }}
              className="space-y-6">
              <div>
                <label className="label-bold mb-3 block">Service Schedule</label>
                <div className="relative">
                  <Calendar className="absolute left-4 top-1/2 -translate-y-1/2 w-4 h-4 text-white/50" />
                  <input
                    type="date"
                    value={date}
                    onChange={(e) => setDate(e.target.value)}
                    className="w-full bg-white/5 border border-white/10 px-12 py-4 rounded-xl text-sm focus:border-primary outline-none transition-colors"
                  />
                </div>
              </div>
              <div>
                <label className="label-bold mb-3 block">Service Location</label>
                <div className="relative">
                  <MapPin className="absolute left-4 top-1/2 -translate-y-1/2 w-4 h-4 text-white/50" />
                  <input

```

```

        type="text"
        value={address}
        placeholder="Enter full service address"
        onChange={(e) => setAddress(e.target.value)}
        className="w-full bg-white/5 border border-white/10 px-12 py-4 rounded-xl text-
sm focus:border-primary outline-none transition-colors"
      />
    </div>
  </div>
  <Button variant="primary" className="w-full py-4 uppercase tracking-widest text-xs"
onClick={() => setStep(2)}>
    Confirm Details
  </Button>
</motion.div>
) : (
  <motion.div initial={{ opacity: 0, x: 20 }} animate={{ opacity: 1, x: 0 }}
className="space-y-6 text-center py-4">
    <div className="w-20 h-20 bg-primary/20 rounded-full flex items-center justify-center
mx-auto mb-6">
      <ShieldCheck className="w-10 h-10 text-primary" />
    </div>
    <h4 className="text-2xl font-bold">Booking Confirmed!</h4>
    <p className="text-sm text-white/70 px-8 leading-relaxed">
      Your request for <span className="text-primary font-bold">{serviceName}</span> is
being processed. A dedicated care coordinator will contact you shortly.
    </p>
    <div className="p-4 bg-white/5 rounded-2xl text-left border border-white/10">
      <p className="text-[10px] uppercase font-bold text-white/50 tracking-widest
mb-2">Summary</p>
      <div className="flex justify-between text-xs mb-1">
        <span className="text-white/70">Date:</span>
        <span className="font-bold">{date || 'ASAP'}</span>
      </div>
      <div className="flex justify-between text-xs">
        <span className="text-white/70">Mode:</span>
        <span className="font-bold">Home Visit</span>
      </div>
    </div>
    <Button variant="outline" className="w-full py-4" onClick={onClose}>Close Portal</
Button>
  </motion.div>
)}
</div>
</motion.div>
</div>
);
};

const ProfileModal = ({ isOpen, onClose }: any) => {
  const [profile, setProfile] = useState(() => {
    const saved = localStorage.getItem('user_profile');
    return saved ? JSON.parse(saved) : { name: '', email: '', phone: '', preferredServices: [] };
  });

  const toggleService = (serviceId: string) => {
    setProfile((prev: any) => ({
      ...prev,
      preferredServices: prev.preferredServices.includes(serviceId)
        ? prev.preferredServices.filter((s: string) => s !== serviceId)
        : [...prev.preferredServices, serviceId]
    }));
  };

  const handleSave = () => {
    localStorage.setItem('user_profile', JSON.stringify(profile));
    onClose();
  };

  if (!isOpen) return null;

  return (

```

```

<div className="fixed inset-0 z-[200] flex items-center justify-center p-6 font-sans">
  <motion.div
    initial={{ opacity: 0 }}
    animate={{ opacity: 1 }}
    onClick={onClose}
    className="absolute inset-0 bg-black/80 backdrop-blur-sm"
  />
  <motion.div
    initial={{ opacity: 0, scale: 0.9, y: 30 }}
    animate={{ opacity: 1, scale: 1, y: 0 }}
    className="glass-card w-full max-w-xl relative z-10 overflow-hidden max-h-[90vh] flex flex-col"
  >
    <div className="flex justify-between items-center mb-8 p-8 border-b border-white/5">
      <div className="flex items-center gap-4">
        <div className="w-12 h-12 bg-primary/20 rounded-2xl flex items-center justify-center text-
primary">
          <Settings className="w-6 h-6" />
        </div>
        <h2 className="text-2xl font-bold uppercase tracking-tight">User Profile</h2>
      </div>
      <button onClick={onClose} className="p-2 hover:bg-white/10 rounded-full transition-colors">
        <X className="w-5 h-5 text-white/70" />
      </button>
    </div>

    <div className="flex-1 overflow-y-auto p-8 space-y-8">
      <div className="grid md:grid-cols-2 gap-8">
        <div>
          <label className="label-bold mb-2 block">Full Name</label>
          <input
            type="text"
            value={profile.name}
            onChange={(e) => setProfile({...profile, name: e.target.value})}
            className="w-full bg-white/5 border border-white/10 px-4 py-3 rounded-xl text-sm
focus:border-primary outline-none transition-colors"
            placeholder="Enter your name"
          />
        </div>
        <div>
          <label className="label-bold mb-2 block">Email Address</label>
          <input
            type="email"
            value={profile.email}
            onChange={(e) => setProfile({...profile, email: e.target.value})}
            className="w-full bg-white/5 border border-white/10 px-4 py-3 rounded-xl text-sm
focus:border-primary outline-none transition-colors"
            placeholder="name@example.com"
          />
        </div>
        <div>
          <label className="label-bold mb-2 block">Phone Number</label>
          <input
            type="tel"
            value={profile.phone}
            onChange={(e) => setProfile({...profile, phone: e.target.value})}
            className="w-full bg-white/5 border border-white/10 px-4 py-3 rounded-xl text-sm
focus:border-primary outline-none transition-colors"
            placeholder="+91 XXXXX XXXXX"
          />
        </div>
      </div>

      <div>
        <label className="label-bold mb-4 block underline decoration-primary decoration-4
underline-offset-8">Preferred Services</label>
        <div className="grid grid-cols-2 md:grid-cols-4 gap-4 mt-8">
          {CATEGORIES.map(cat => (
            <button
              key={cat.id}
              onClick={() => toggleService(cat.id)}
              className="flex flex-col items-center gap-3 p-4 rounded-2xl border transition-all

```

```

    ${profile.preferredServices.includes(cat.id) ? 'bg-primary/20 border-primary text-primary' : 'bg-
    white/5 border-white/10 text-white/50 hover:bg-white/10'}}
    >
    {cat.icon}
    <span className="text-[10px] uppercase font-bold tracking-widest">{cat.name}</span>
    {profile.preferredServices.includes(cat.id) && <CheckCircle2 className="w-3 h-3
absolute top-2 right-2" />}
    </button>
  )}}
</div>
</div>
</div>

    <div className="p-8 border-t border-white/5 bg-primary/5">
      <Button variant="primary" className="w-full py-4 text-xs tracking-[0.2em] uppercase"
onClick={handleSave}>
        Save Profile Changes
      </Button>
    </div>
  </motion.div>
</div>
);
};

const AuthLandingPage = ({ onEnter, onProviderRegister }: { onEnter: () => void; onProviderRegister:
() => void }) => {
  return (
    <div className="min-h-screen bg-surface flex items-center justify-center p-6 relative overflow-
hidden monolith-grid font-sans">
      <div className="absolute top-6 right-6 z-50">
        <LanguageSelector />
      </div>

      <div className="absolute inset-0 pointer-events-none">
        <div className="absolute top-1/4 -left-20 w-96 h-96 bg-primary/10 blur-[120px] rounded-full" />
        <div className="absolute bottom-1/4 -right-20 w-96 h-96 bg-primary/5 blur-[120px] rounded-
full" />
      </div>

      <motion.div
        initial={{ opacity: 0, y: 30 }}
        animate={{ opacity: 1, y: 0 }}
        className="glass-card max-w-4xl w-full p-12 md:p-20 relative z-10 flex flex-col md:flex-row
gap-16 items-center"
      >
        <div className="flex-1 space-y-8">
          <div className="flex items-center gap-3">
            <div className="w-10 h-10 bg-primary/20 rounded-xl flex items-center justify-center
overflow-hidden">
              
            </div>
            <span className="text-xl font-black uppercase tracking-tighter">CareVia.<span
className="text-primary">Elite</span></span>
          </div>

          <h1 className="text-5xl md:text-6xl font-black leading-tight uppercase tracking-tighter">
            Premium Care. <span className="text-primary">Unlocked.</span>
          </h1>

          <p className="text-lg text-white/60 leading-relaxed max-w-sm">
            The best care network in India. Please log in to view our caregivers.
          </p>

          <div className="flex items-center gap-6 pt-4">
            <div className="flex -space-x-4">
              {[1,2,3,4].map(i => (
                <div key={i} className="w-10 h-10 rounded-full border-2 border-surface overflow-

```

```

hidden bg-primary/20">
    <img src={`https://i.pravatar.cc/100?img=${i+10}`} alt="User" />
  </div>
  )})
</div>
  <p className="text-[10px] font-bold uppercase tracking-widest text-white/80">Joined by
<span className="text-primary">12k</span> active families</p>
</div>
</div>

  <div className="w-full md:w-[350px] space-y-6">
    <div className="glass-card p-8 bg-primary/20 border-primary/30 backdrop-blur-xl shadow-
[0_0_50px_rgba(85,107,92,0.15)] relative overflow-hidden">
      <div className="absolute inset-0 bg-gradient-to-br from-primary/10 to-transparent"></div>
      <h3 className="relative text-xs font-black uppercase tracking-[0.2em] mb-6 text-center
text-primary drop-shadow-sm">Care Elite Entrance</h3>
      <div className="relative space-y-4">
        <Button variant="primary" className="w-full py-5 uppercase tracking-widest text-xs
font-black shadow-[0_0_30px_rgba(85,107,92,0.4)]" onClick={onEnter}>
          Enter Priority Care Portal
        </Button>
        <div className="relative">
          <div className="absolute inset-0 flex items-center"><div className="w-full border-
t border-white/5"></div></div>
          <div className="relative flex justify-center text-[8px] uppercase font-
black"><span className="bg-surface px-4 text-white/40 tracking-[0.2em]">Partner Network</span></div>
        </div>
        <Button
          variant="outline"
          className="w-full py-5 uppercase tracking-widest text-[10px] opacity-60
hover:opacity-100 transition-all"
          onClick={onProviderRegister}
        >
          Join as Healthcare Partner
        </Button>
      </div>
      <p className="mt-8 text-center text-[9px] text-white/40 uppercase tracking-widest font-
bold">
        Secured by Google Cloud & CareVia Nexus
      </p>
    </div>
  </div>
</motion.div>
</div>
);
};

export default function App() {
  useEffect(() => {
    applyHindiCorrections();
  }, []);

  const [isAuthOpen, setIsAuthOpen] = useState(false);
  const [isLegalOpen, setIsLegalOpen] = useState(false);
  const [isPremium, setIsPremium] = useState(false);
  const [isAdminDashboardOpen, setIsAdminDashboardOpen] = useState(false);
  const [spectatedProvider, setSpectatedProvider] = useState<any>(null);
  const [masterOtp, setMasterOtp] = useState<string | null>(null);
  const [isBotOpen, setIsBotOpen] = useState(false);
  const [isLoggedIn, setIsLoggedIn] = useState(false);
  const [isProfileOpen, setIsProfileOpen] = useState(false);
  const [isBookingOpen, setIsBookingOpen] = useState(false);
  const [isProviderModalOpen, setIsProviderModalOpen] = useState(false);
  const [selectedBookingService, setSelectedBookingService] = useState('');
  const [isLandingPage, setIsLandingPage] = useState(true);
  const [isPaymentOpen, setIsPaymentOpen] = useState(false);
  const [isWalletOpen, setIsWalletOpen] = useState(false);
  const [selectedPlanName, setSelectedPlanName] = useState('');
  const [isProviderDashboardOpen, setIsProviderDashboardOpen] = useState(false);
  const [financialAlerts, setFinancialAlerts] = useState<any[]>([]);
  const [providerProfile, setProviderProfile] = useState({

```

```

    name: 'Aarav Sharma',
    spId: 'SP-9921',
    isVerified: true,
    isDocsUploaded: true
  });

const [activeBookings, setActiveBookings] = useState<any[]>([]);

const [user, setUser] = useState<any>(null);
const [profile, setProfile] = useState<any>(null);

// 1. Auth Listener
useEffect(() => {
  const unsubscribe = onAuthStateChanged(auth, async (u) => {
    setUser(u);
    if (u) {
      const snap = await getDoc(doc(db, 'users', u.uid));
      if (snap.exists()) {
        setProfile(snap.data());
        setIsLoggedIn(true);
        setIsLandingPage(false);
        if (snap.data().role === 'admin') setIsAdminDashboardOpen(true);
      } else {
        // New user logic handled in AuthModal
      }
    } else {
      setIsLoggedIn(false);
      setIsLandingPage(true);
      setProfile(null);
    }
  });
  return () => unsubscribe();
}, []);

// 2. Real-time Firestore Sync
useEffect(() => {
  if (!user || !profile) return;

  let bQuery;
  if (profile.role === 'admin') {
    bQuery = query(collection(db, 'bookings'), orderBy('bookingDate', 'desc'));
  } else if (profile.role === 'provider') {
    bQuery = query(collection(db, 'bookings'), where('providerId', '==', user.uid),
    orderBy('bookingDate', 'desc'));
  } else {
    bQuery = query(collection(db, 'bookings'), where('customerId', '==', user.uid),
    orderBy('bookingDate', 'desc'));
  }

  const unsubscribeBookings = onSnapshot(bQuery, (snap) => {
    const data = snap.docs.map(d => ({ id: d.id, ...d.data() }));
    setActiveBookings(data);
  }, (err) => handleFirestoreError(err, 'list', 'bookings'));

  let lQuery;
  if (profile.role === 'admin') {
    lQuery = query(collection(db, 'queries'), orderBy('createdAt', 'desc'));
  } else {
    lQuery = query(collection(db, 'queries'), where('userId', '==', user.uid), orderBy('createdAt',
    'desc'));
  }

  const unsubscribeLogs = onSnapshot(lQuery, (snap) => {
    const data = snap.docs.map(d => ({ id: d.id, ...d.data() }));
    setLogs(data);
  }, (err) => handleFirestoreError(err, 'list', 'queries'));

  return () => {
    unsubscribeBookings();
    unsubscribeLogs();
  };
}, []);

```

```

}, [user, profile]);

const [notificationSound] = useState(new Audio('https://assets.mixkit.co/active_storage/sfx/2869/2869-preview.mp3'));

const [providerSound] = useState(new Audio('https://assets.mixkit.co/active_storage/sfx/2869/2869-preview.mp3'));
const [customerSound] = useState(new Audio('https://assets.mixkit.co/active_storage/sfx/2870/2870-preview.mp3'));
const [premiumSound] = useState(new Audio('https://assets.mixkit.co/active_storage/sfx/2871/2871-preview.mp3'));

useEffect(() => {
  if (!profile || profile.role !== 'admin') return;

  // Listen for new users
  const unsubUsers = onSnapshot(query(collection(db, 'users'), orderBy('updatedAt', 'desc'), limit(5)), (snap) => {
    snap.docChanges().forEach(change => {
      if (change.type === 'added') {
        const data = change.doc.data();
        // Only play if it's very recent (last 2 mins)
        if (data.updatedAt && (Date.now() - data.updatedAt.toMillis() < 120000)) {
          if (data.role === 'provider') providerSound.play().catch(()=>console.log('Audio error'));
          else customerSound.play().catch(()=>console.log('Audio error'));
        }
      }
    });
  });

  // Listen for admin notifications (like source code download)
  const unsubAdminNotifs = onSnapshot(query(collection(db, 'adminNotifications'), orderBy('timestamp', 'desc'), limit(1)), (snap) => {
    snap.docChanges().forEach(change => {
      if (change.type === 'added') {
        const data = change.doc.data();
        if (Date.now() - data.timestamp < 120000 && data.adminId !== user?.uid) {
          notificationSound.play().catch(()=>console.log('Audio error'));
          const newAlert = {
            id: `SYS-${Date.now()}-${Math.random().toString(36).substring(2, 9)}`,
            type: 'SYSTEM',
            details: data.message,
            timestamp: new Date().toLocaleTimeString(),
            acknowledged: false
          };
          setFinancialAlerts(prev => [newAlert, ...prev]);
        }
      }
    });
  });

  return () => {
    unsubUsers();
    unsubAdminNotifs();
  };
}, [profile, user, notificationSound]);

const handleLogout = async () => {
  try {
    await signOut(auth);
    setIsLoggedIn(false);
    setIsLandingPage(true);
    setIsAdminDashboardOpen(false);
    setIsProviderDashboardOpen(false);
    setIsProfileOpen(false);
    setProfile(null);
    setUser(null);
  } catch (error) {
    console.error("Logout failed:", error);
  }
}

```

```

    }
  };

  const playNotification = () => {
    notificationSound.currentTime = 0;
    notificationSound.play().catch(e => console.log("Sound blocked by browser:", e));
  };

  const handleSPAccept = (bookingId: string) => {
    setActiveBookings(prev => prev.map(b => b.id === bookingId ? { ...b, status: 'ACCEPTED_BY_SP' } :
b));
    playNotification(); // Notify Admin
    addLog({ action: `SP Accepted Booking ${bookingId}. Awaiting Admin Finalization.`, admin:
"System", time: new Date().toLocaleString(), type: 'system' });
  };

  const triggerFinancialAlert = (type: 'PAYMENT' | 'WALLET', details: string) => {
    const newAlert = {
      id: `FIN-${Date.now()}-${Math.random().toString(36).substring(2, 9)}`,
      type,
      details,
      timestamp: new Date().toLocaleTimeString(),
      acknowledged: false
    };
    setFinancialAlerts(prev => [newAlert, ...prev]);
    playNotification();
    addLog({ action: `Financial Event Intercepted: ${type} - ${details}`, admin: "Finance Bot", time:
new Date().toLocaleString(), type: 'financial' });
  };

  const handleProviderAction = (bookingId: string, action: string) => {
    setActiveBookings(prev => prev.map(b => {
      if (b.id === bookingId) {
        if (action === 'ACCEPT') return { ...b, status: 'ACCEPTED_BY_SP' };
        if (action === 'DECLINE') return { ...b, status: 'DECLINED' };
        if (action === 'START_SERVICE') return { ...b, status: 'IN_PROGRESS' };
        if (action === 'END_SERVICE') return { ...b, status: 'COMPLETED' };
      }
      return b;
    }));

    addLog({
      action: `Provider ${action}ed Booking ${bookingId}`,
      admin: "System",
      time: new Date().toLocaleString(),
      type: 'system',
      details: `Action: ${action} triggered by Partner ID ${providerProfile.spId}`
    });

    if (action === 'ACCEPT' || action === 'START_SERVICE' || action === 'END_SERVICE') {
      playNotification();
    }
  };

  const handleConfirmBooking = (bookingId: string) => {
    setActiveBookings(prev => prev.map(b => b.id === bookingId ? { ...b, status: 'CONFIRMED', otp:
Math.floor(1000 + Math.random() * 9000).toString() } : b));
    addLog({ action: `Booking ${bookingId} finalized by Admin`, admin: "Ashvin", time: new
Date().toLocaleString(), type: 'admin' });
  };

  const [bankDetails, setBankDetails] = useState({
    accountHolder: 'CAREVIA SOLUTIONS PVT LTD',
    accountNumber: '923010045566778',
    ifsc: 'UTIB0000001',
    bankName: 'AXIS BANK',
    branch: 'MUMBAI MAIN'
  });

  const [logs, setLogs] = useState<any[]>([]);

```

```

const addLog = (log: any) => {
  setLogs(prev => [{ ...log, id: `${Date.now()}-${Math.random().toString(36).substring(2, 9)}` }, ...prev]);
};

const [sharingLevel, setSharingLevel] = useState('Restricted');
const [teamAccess, setTeamAccess] = useState('Read-Only');
const aiSessionId = useMemo(() => `SID-${Math.random().toString(36).substring(2, 9).toUpperCase()}`,
[]);

// AI Service Matching States
const [userLocation, setUserLocation] = useState('');
const [activeServiceTab, setActiveServiceTab] = useState('elderly');
const [isAnalyzing, setIsAnalyzing] = useState(false);
const [analysisStep, setAnalysisStep] = useState(0);
const [selectedCategory, setSelectedCategory] = useState('');

const analysisSteps = [
  "Analyzing nature of concern...",
  "Cross-referencing provider database...",
  "Applying precision matching logic...",
  "Optimizing care coordinator assignment...",
  "Finalizing recommendations..."
];

const handleRequirementSubmit = () => {
  setIsAnalyzing(true);
  setAnalysisStep(0);

  // Simulate AI analysis steps
  const interval = setInterval(() => {
    setAnalysisStep(prev => {
      if (prev >= analysisSteps.length - 1) {
        clearInterval(interval);
        setTimeout(() => {
          setIsAnalyzing(false);
          // Map category to tab
          const mapping: Record<string, string> = {
            'Elderly Assistance': 'elderly',
            'Medical Support': 'elderly',
            'Child Development': 'child',
            'Pet Care': 'pet',
            'Domestic Help': 'helpers'
          };
          if (mapping[selectedCategory]) {
            setActiveServiceTab(mapping[selectedCategory]);
          }
          document.getElementById('services')?.scrollIntoView({ behavior: 'smooth' });
        }, 800);
        return prev;
      }
      return prev + 1;
    });
  }, 600);
};

return (
  <div className="min-h-screen overflow-x-hidden">
    {isLandingPage ? (
      <AuthLandingPage
        onEnter={() => setIsLandingPage(false)}
        onProviderRegister={() => {
          setIsLandingPage(false);
          setIsProviderModalOpen(true);
        }}
      />
    ) : (
      <>
        <Navbar
          onAuthClick={() => setIsAuthOpen(true)}

```

```

onBotClick={() => setIsBotOpen(true)}
onProviderClick={() => setIsProviderModalOpen(true)}
isPremium={isPremium}
isLoggedIn={isLoggedIn}
onProfileClick={() => setIsProfileOpen(true)}
setActiveTab={setActiveServiceTab}
onLogout={handleLogout}
/>

<MobileBottomNav
  onAuthClick={() => setIsAuthOpen(true)}
  onBotClick={() => setIsBotOpen(true)}
  isLoggedIn={isLoggedIn}
  onProfileClick={() => setIsProfileOpen(true)}
/>

<main className="pb-24 lg:pb-0">
  {/* Hero Overhaul */}
  <section id="home" className="relative min-h-screen flex items-center pt-20 overflow-
hidden font-sans">
    <div className="absolute inset-0 z-0">
      <div className="absolute top-1/2 left-1/2 -translate-x-1/2 -translate-y-1/2 text-[450px]
font-black leading-none uppercase opacity-[0.03] select-none pointer-events-none tracking-tighter">
        HEALTH
      </div>
      
      <div className="absolute inset-0 bg-gradient-to-t from-surface via-surface/40 to-
transparent" />
    </div>

    <div className="max-w-7xl mx-auto px-6 relative z-10 grid lg:grid-cols-2 gap-16 items-
center">
      <motion.div
        initial={{ opacity: 0, x: -50 }}
        animate={{ opacity: 1, x: 0 }}
        transition={{ duration: 0.8 }}
      >
        <h1 className="text-[72px] md:text-[110px] leading-[0.85] font-black tracking-tighter
mb-8 uppercase">
          HEALTH. <br />
          <span className="text-primary">Wellness.</span><br />
          Compassion.
        </h1>
        <p className="text-lg text-white/70 mb-12 max-w-lg font-light leading-relaxed">
          Elite-tier healthcare and assistance management for your loved ones. Experience the
future of home-based support.
        </p>
        <div className="flex flex-wrap gap-6">
          <Button variant="primary" className="px-10 py-5 text-xs tracking-widest uppercase"
onClick={() => setIsAuthOpen(true)}>
            Get Started
          </Button>
          <Button variant="outline" className="px-10 py-5 text-xs tracking-widest uppercase"
onClick={() => document.getElementById('services')?.scrollIntoView({ behavior: 'smooth' })}>
            Our Packages
          </Button>
        </div>
      </motion.div>

      <div className="relative">
        <div className="glass-card p-4 rounded-[3rem] border-white/10">
          <div className="bg-neutral-900 rounded-[2.5rem] overflow-hidden aspect-video relative
group">
            
    <div className="absolute inset-0 bg-gradient-to-t from-black/80 to-transparent
flex items-end p-8">
        <div className="flex items-center gap-4">
            <div className="w-12 h-12 rounded-full bg-primary flex items-center justify-
center text-surface">
                <Phone className="w-5 h-5" />
            </div>
            <div>
                <p className="text-xs font-bold text-primary uppercase tracking-
widest">Connect Instantly</p>
                <p className="text-xl font-bold">Priority Medical Dispatch</p>
            </div>
        </div>
        <div>
            <div className="mt-4 flex flex-col gap-4">
                <ActiveServiceTracker bookings={activeBookings} />
                <WalletSection onManage={() => setIsWalletOpen(true)} />
            </div>
        </div>
    </div>
</div>
</div>
</section>

<HowToUse />

<AboutSection />

{/* Requirements Form Section - Moved here before Services */}
<section id="requirements" className="relative py-24 bg-surface monolith-grid font-sans
overflow-hidden scroll-mt-24">
    {isAnalyzing && (
        <motion.div
            initial={{ opacity: 0 }}
            animate={{ opacity: 1 }}
            className="absolute inset-0 z-50 bg-surface/90 backdrop-blur-xl flex flex-col items-
center justify-center text-center p-6"
        >
            <div className="w-24 h-24 mb-8 relative">
                <motion.div
                    animate={{ rotate: 360 }}
                    transition={{ duration: 4, repeat: Infinity, ease: "linear" }}
                    className="absolute inset-0 rounded-full border-t-2 border-primary"
                />
                <motion.div
                    animate={{ rotate: -360 }}
                    transition={{ duration: 6, repeat: Infinity, ease: "linear" }}
                    className="absolute inset-2 rounded-full border-b-2 border-primary/30"
                />
                <div className="absolute inset-0 flex items-center justify-center">
                    <Bot className="w-8 h-8 text-primary animate-pulse" />
                </div>
            </div>
            <h3 className="text-xl font-black uppercase tracking-widest mb-2">Analyzing
Requirements</h3>
            <p className="text-primary font-mono text-[10px] uppercase tracking-[0.3em] h-4">
                {analysisSteps[analysisStep]}
            </p>
            <div className="mt-8 w-64 h-1 bg-white/5 rounded-full overflow-hidden">
                <motion.div
                    initial={{ width: 0 }}
                    animate={{ width: `${(analysisStep + 1) / analysisSteps.length * 100}%` }}
                    className="h-full bg-primary"
                />
            </div>
        </motion.div>
    )}

```

```

    })

    <div className="max-w-5xl mx-auto px-6">
      <SectionHeading
        sub="Tailored For You"
        title="Describe Your Requirements."
        desc="Tell us about the problems you're facing. Our AI evaluates your needs to recommend
the ideal care strategy."
        centered
      />
      <motion.div
        whileHover={{ scale: 1.01 }}
        className="glass-card p-12 bg-white/[0.02]"
      >
        <div className="grid md:grid-cols-2 gap-8 mb-8">
          <div>
            <label className="label-bold mb-4 block">Nature of Concern</label>
            <select
              value={selectedCategory}
              onChange={(e) => setSelectedCategory(e.target.value)}
              className="w-full bg-white/5 border border-white/10 px-6 py-4 rounded-2xl text-sm
focus:border-primary outline-none transition-colors appearance-none text-white"
            >
              <option value="" className="bg-surface">Select Category</option>
              <option value="Elderly Assistance" className="bg-surface">Elderly Assistance</
option>
              <option value="Medical Support" className="bg-surface">Medical Support</option>
              <option value="Child Development" className="bg-surface">Child Development</option>
              <option value="Pet Care" className="bg-surface">Pet Care</option>
              <option value="Domestic Help" className="bg-surface">Domestic Help</option>
            </select>
          </div>
          <div>
            <label className="label-bold mb-4 block">Urgency Level</label>
            <div className="flex flex-wrap sm:flex-nowrap gap-4">
              {[ 'Standard', 'Priority', 'Emergency' ].map(level => (
                <button key={level} className="flex-1 min-w-[100px] py-4 border border-white/10
rounded-2xl text-xs font-bold uppercase tracking-widest hover:bg-primary/10 hover:border-primary/40
transition-all">
                  {level}
                </button>
              ))}
            </div>
          </div>
        </div>
        <div className="mb-8">
          <label className="label-bold mb-4 block">Brief Description</label>
          <textarea rows={4} className="w-full bg-white/5 border border-white/10 px-6 py-4
rounded-2xl text-sm focus:border-primary outline-none transition-colors resize-none"
placeholder="Provide details for our AI to analyze..." />
        </div>
        <Button
          variant="primary"
          className="w-full py-5 text-xs tracking-widest uppercase"
          onClick={handleRequestSubmit}
          disabled={!selectedCategory || isAnalyzing}
        >
          {isAnalyzing ? 'Studying Requirements...' : 'Analyze & Show Best Services'}
        </Button>
      </motion.div>
    </div>
  </section>

  <ServiceCatalog
    activeTab={activeServiceTab}
    setActiveTab={setActiveServiceTab}
    userLocation={userLocation}
    setUserLocation={setUserLocation}
    onBook={(name) => {
      setSelectedBookingService(name);
      setIsBookingOpen(true);
    }}
  />

```

```

    }}
  />

  <SubscriptionPlans onUpgrade={({planName: string}) => {
    setSelectedPlanName(planName);
    setIsPaymentOpen(true);
  }} />

  <section id="join-provider" className="py-24 bg-surface relative overflow-hidden font-sans">
    <div className="absolute inset-0 bg-primary/5 -skew-y-3 translate-y-12" />
    <div className="max-w-7xl mx-auto px-6 relative z-10">
      <div className="glass-card p-12 border-primary/20 bg-primary-container/10 flex flex-col lg:flex-row items-center justify-between gap-12">
        <div className="lg:max-w-2xl">
          <div className="flex items-center gap-4 mb-6 text-primary">
            <Handshake className="w-10 h-10" />
            <h2 className="text-3xl font-black uppercase tracking-tight">Empower Lives With
Us.</h2>
          </div>
          <p className="text-lg text-white/80 leading-relaxed mb-8 font-light">
            Are you a specialized care provider? Join the CAREVIA ecosystem to connect with
families seeking elite healthcare, child care, and domestic assistance. We offer fair commissions,
flexible slots, and a platform built on trust.
          </p>
          <div className="grid grid-cols-2 gap-6 mb-10">
            <div className="flex items-center gap-3">
              <CheckCircle2 className="text-primary w-5 h-5 flex-shrink-0" />
              <span className="text-xs font-bold uppercase tracking-widest text-
white/60">Verified Onboarding</span>
            </div>
            <div className="flex items-center gap-3">
              <CheckCircle2 className="text-primary w-5 h-5 flex-shrink-0" />
              <span className="text-xs font-bold uppercase tracking-widest text-
white/60">Flexible Schedules</span>
            </div>
            <div className="flex items-center gap-3">
              <CheckCircle2 className="text-primary w-5 h-5 flex-shrink-0" />
              <span className="text-xs font-bold uppercase tracking-widest text-
white/60">Direct Payouts</span>
            </div>
            <div className="flex items-center gap-3">
              <CheckCircle2 className="text-primary w-5 h-5 flex-shrink-0" />
              <span className="text-xs font-bold uppercase tracking-widest text-
white/60">Premium Client Base</span>
            </div>
          </div>
          <Button
            variant="primary"
            className="px-10 py-5 uppercase tracking-[0.2em] text-xs"
            onClick={() => setIsProviderModalOpen(true)}
          >
            Join as Service Provider
          </Button>
        </div>
        <div className="w-full lg:w-96 flex flex-col gap-4">
          <div className="glass-card p-6 border-white/5 bg-white/[0.02]">
            <p className="text-[10px] font-bold text-primary uppercase tracking-widest mb-4">Required Documents</p>
            <ul className="space-y-3">
              <li className="flex items-center gap-3 text-xs text-white/80">
                <FileText className="w-4 h-4 text-white/40" /> Aadhaar Card
              </li>
              <li className="flex items-center gap-3 text-xs text-white/80">
                <FileText className="w-4 h-4 text-white/40" /> PAN Card
              </li>
              <li className="flex items-center gap-3 text-xs text-white/80">
                <GraduationCap className="w-4 h-4 text-white/40" /> Academic Certificates
              </li>
            </ul>
          </div>
          <div className="glass-card p-6 border-white/5 bg-white/[0.02] text-center">

```

```

                <p className="text-3xl font-bold text-primary mb-1">2.4k+</p>
                <p className="text-[10px] font-bold text-white/50 uppercase tracking-
widest">Active Partners</p>
            </div>
        </div>
    </div>
</div>
</section>

<ReviewsSection />

<ContactSection />
</main>

<Footer onLegalClick={() => setIsLegalOpen(true)} />
<SOSButton user={user} />
<AuthModal
    isOpen={isOpen}
    onOpenAdmin={() => setIsAdminDashboardOpen(true)}
    onClose={() => setIsAuthOpen(false)}
    onLogin={() => setIsLoggedIn(true)}
    onProviderLogin={() => setIsProviderDashboardOpen(true)}
/>
{isAdminDashboardOpen && (
    <AdminDashboard
        onClose={() => setIsAdminDashboardOpen(false)}
        actions={logs}
        bankDetails={bankDetails}
        setBankDetails={setBankDetails}
        sharingLevel={sharingLevel}
        setSharingLevel={setSharingLevel}
        teamAccess={teamAccess}
        setTeamAccess={setTeamAccess}
        bookings={activeBookings}
        financialAlerts={financialAlerts}
        onConfirmBooking={handleConfirmBooking}
        onAction={handleProviderAction}
        onSpectateProvider={(p: any) => {
            setSpectatedProvider(p);
            setIsAdminDashboardOpen(false);
        }}
        masterOtp={masterOtp}
        setMasterOtp={setMasterOtp}
        onLogout={handleLogout}
    />
)}
{spectatedProvider && (
    <ProviderDashboard
        providerData={spectatedProvider}
        isSpectator={true}
        onUpdateProvider={() => {}}
        onClose={() => setSpectatedProvider(null)}
        bookings={activeBookings.filter((b: any) => b.providerName === spectatedProvider.name)}
        onAction={() => {}}
        masterOtp={masterOtp}
    />
)}
{isProviderDashboardOpen && (
    <ProviderDashboard
        providerData={providerProfile}
        onUpdateProvider={(data: any) => setProviderProfile(data)}
        onClose={() => setIsProviderDashboardOpen(false)}
        bookings={activeBookings}
        onAction={handleProviderAction}
        masterOtp={masterOtp}
        onLogout={handleLogout}
    />
)}
<BookingModal
    isOpen={isOpen}
    serviceName={selectedBookingService}

```

```

        onClose={() => setIsBookingOpen(false)}
      />
      <ProviderRegistrationModal
        isOpen={isProviderModalOpen}
        onClose={() => setIsProviderModalOpen(false)}
      />
      <ProfileModal isOpen={isProfileOpen} onClose={() => setIsProfileOpen(false)} />
      <PaymentModal
        isOpen={isPaymentOpen}
        planName={selectedPlanName}
        onClose={() => setIsPaymentOpen(false)}
        onPaymentComplete={() => {
          setIsPaymentOpen(false);
          setIsPremium(true);
          triggerFinancialAlert('PAYMENT', `Customer completed payment for ${selectedPlanName} Pack`);
        }}
      />
      <WalletModal isOpen={isWalletOpen} onClose={() => setIsWalletOpen(false)} />
      <LegalModal isOpen={isLegalOpen} onClose={() => setIsLegalOpen(false)} />
      <AIBotChat isOpen={isBotOpen} isPremium={isPremium} onClose={() => setIsBotOpen(false)}
    sessionId={aiSessionId} onLog={addLog} />
    <LiveAlarm
      bookings={activeBookings}
      financialAlerts={financialAlerts}
      isAdmin={isAdminDashboardOpen}
      isSP={isProviderDashboardOpen}
      onAcknowledge={() => {}}
    />
  </>
)}
</div>
);
}

```

```

const MobileBottomNav = ({ onAuthClick, onBotClick, isLoggedIn, onProfileClick }: any) => {
  return (
    <div className="lg:hidden fixed bottom-0 left-0 right-0 z-[150] bg-surface/90 backdrop-blur-xl
border-t border-white/5 pb-safe font-sans">
      <div className="flex items-center justify-around p-4">
        <a href="#home" className="flex flex-col items-center gap-1 text-white/70 hover:text-primary
transition-colors">
          <Home className="w-5 h-5" />
          <span className="text-[8px] font-bold uppercase tracking-widest">Home</span>
        </a>
        <a href="#services" className="flex flex-col items-center gap-1 text-white/70 hover:text-
primary transition-colors">
          <Stethoscope className="w-5 h-5" />
          <span className="text-[8px] font-bold uppercase tracking-widest">Services</span>
        </a>
        <button onClick={onBotClick} className="flex flex-col items-center gap-1 text-primary -mt-6
relative">
          <div className="w-12 h-12 rounded-full bg-primary flex items-center justify-center shadow-
[0_0_20px_rgba(212,175,55,0.4)]">
            <Bot className="w-6 h-6 text-surface" />
          </div>
          <span className="text-[8px] font-bold uppercase tracking-widest mt-1">AI Concierge</span>
        </button>
        <a href="#pricing" className="flex flex-col items-center gap-1 text-white/70 hover:text-
primary transition-colors">
          <Star className="w-5 h-5" />
          <span className="text-[8px] font-bold uppercase tracking-widest">Premium</span>
        </a>
        <button onClick={isLoggedIn ? onProfileClick : onAuthClick} className="flex flex-col items-
center gap-1 text-white/70 hover:text-primary transition-colors">
          <User className="w-5 h-5" />
          <span className="text-[8px] font-bold uppercase tracking-widest">{isLoggedIn ? 'Profile' :
'Login'}</span>
        </button>
      </div>
    </div>
  );
};

```

```

};

const Navbar = ({ onAuthClick, onBotClick, isPremium, isLoggedIn, onProfileClick, setActiveTab,
onProviderClick, onLogout }: any) => {
  const [isScrolled, setIsScrolled] = useState(false);
  const [searchQuery, setSearchQuery] = useState('');
  const [searchResults, setSearchResults] = useState<any[]>([]);
  const [isMobileMenuOpen, setIsMobileMenuOpen] = useState(false);

  useEffect(() => {
    const handleScroll = () => setIsScrolled(window.scrollY > 20);
    window.addEventListener('scroll', handleScroll);
    return () => window.removeEventListener('scroll', handleScroll);
  }, []);

  useEffect(() => {
    if (!searchQuery.trim()) {
      setSearchResults([]);
      return;
    }

    const query = searchQuery.toLowerCase();
    const results: any[] = [];

    CATEGORIES.forEach(cat => {
      cat.sections.forEach(sec => {
        sec.items.forEach(item => {
          if (item.toLowerCase().includes(query) || sec.name.toLowerCase().includes(query) ||
cat.name.toLowerCase().includes(query)) {
            // Avoid duplicates of the same item if it matches multiple things, though unlikely here
            if (!results.find(r => r.item === item)) {
              results.push({
                catId: cat.id,
                catName: cat.name,
                item: item,
                section: sec.name
              });
            }
          }
        });
      });
    });

    setSearchResults(results.slice(0, 6)); // Limit results
  }, [searchQuery]);

  return (
    <nav className={`fixed top-0 left-0 right-0 z-[160] transition-all duration-300 font-sans border-b
border-primary/20 ${isScrolled ? 'bg-surface py-3 shadow-[0_4px_30px_rgba(0,0,0,0.5)]' : 'bg-surface
py-5'}`}>
      <div className="max-w-7xl mx-auto px-6 flex justify-between items-center">
        <div className="flex items-center gap-4 flex-shrink-0">
          <Logo />
        </div>

        </* Search Bar - Desktop */>
        <div className="hidden lg:flex flex-1 max-w-sm mx-12 relative">
          <div className="relative w-full group">
            <Search className="absolute left-4 top-1/2 -translate-y-1/2 w-4 h-4 text-white/50 group-
focus-within:text-primary transition-colors" />
            <input
              type="text"
              value={searchQuery}
              onChange={(e) => setSearchQuery(e.target.value)}
              placeholder="Search services..."
              className="w-full bg-white/5 border border-white/10 px-10 py-3 rounded-2xl text-xs
outline-none focus:border-primary/50 focus:bg-white/10 transition-all font-bold tracking-widest
placeholder:text-white/40"
            />
            <AnimatePresence>
              {searchResults.length > 0 && (

```

```

<motion.div
  initial={{ opacity: 0, y: 10, scale: 0.95 }}
  animate={{ opacity: 1, y: 0, scale: 1 }}
  exit={{ opacity: 0, y: 10, scale: 0.95 }}
  className="absolute top-full left-0 right-0 mt-4 glass-card p-2 border-primary/20
shadow-2xl overflow-hidden"
>
  {searchResults.map((res, i) => (
    <button
      key={i}
      onClick={() => {
        setActiveTab(res.catId);
        setSearchQuery('');
        setSearchResults([]);
        document.getElementById('services')?.scrollIntoView({ behavior: 'smooth' });
      }}
      className="w-full text-left p-3 hover:bg-primary/10 rounded-xl transition-all
group"
    >
      <p className="text-[10px] font-bold text-primary uppercase tracking-widest
mb-0.5">{res.catName}</p>
      <p className="text-xs font-bold text-white group-hover:text-white">{res.item}</p>
      <p className="text-[8px] text-white/50 uppercase tracking-widest
mt-1">{res.section}</p>
    </button>
  ))}
</motion.div>
)}
</AnimatePresence>
</div>
</div>

<div className="flex items-center gap-3">
  <LanguageSelector className="hidden md:flex" />
  {/* Join as Provider - Desktop */}
  <div className="hidden xl:flex gap-6 mr-4">
    {NAV_LINKS.map(link => (
      <a
        key={link.name}
        href={link.href}
        className="text-[9px] font-bold tracking-[0.2em] uppercase text-white/70 hover:text-
primary transition-all"
      >
        {link.name}
      </a>
    ))}
  </div>

  <button
    onClick={onBotClick}
    className={`p-3 rounded-xl border transition-all flex items-center gap-2 group relative
${isPremium ? 'bg-primary/20 border-primary/40 text-primary' : 'bg-white/5 border-white/10 text-
white/80 hover:bg-white/10'}`}
  >
    <Bot className="w-4 h-4" />
    <span className="text-[9px] uppercase font-bold tracking-widest hidden md:block">Elite AI</
span>
    {!isPremium && <Lock className="w-2.5 h-2.5 opacity-40" />}
    {isPremium && <Sparkles className="w-2.5 h-2.5 animate-pulse" />}
  </button>

  <button
    onClick={isLoggedIn ? onProfileClick : onAuthClick}
    className="btn-outline p-3 xs:px-5 xs:py-2.5 text-[9px] uppercase tracking-widest flex
items-center gap-2 bg-primary text-surface font-black border-transparent hover:opacity-90 transition-
all shadow-[0_0_20px_rgba(45,212,191,0.2)]"
  >
    <User className="w-3.5 h-3.5" />
    <span className="hidden xs:block">{isLoggedIn ? 'Logged In' : 'Login'}</span>
  </button>

```

```

        {isLoggedIn && (
            <button
                onClick={onLogout}
                className="p-3 xs:px-5 xs:py-2.5 bg-red-500/10 border border-red-500/20 text-red-500
rounded-xl hover:bg-red-500 hover:text-white transition-all flex items-center gap-2"
                title="Logout"
            >
                <LogOut className="w-3.5 h-3.5" />
                <span className="hidden xs:block text-[9px] uppercase tracking-widest font-black">Log
Out</span>
            </button>
        )}

        <button
            onClick={() => setIsMobileMenuOpen(!isMobileMenuOpen)}
            className="xl:hidden p-3 bg-white/5 rounded-xl text-white/70 hover:text-white transition-
all ml-1"
        >
            {isMobileMenuOpen ? <X className="w-5 h-5" /> : <Menu className="w-5 h-5" />}
        </button>
    </div>
</div>

{/* Mobile Menu Overlay */}
<AnimatePresence>
    {isMobileMenuOpen && (
        <motion.div
            initial={{ opacity: 0, height: 0 }}
            animate={{ opacity: 1, height: 'auto' }}
            exit={{ opacity: 0, height: 0 }}
            className="xl:hidden bg-surface border-t border-white/5 overflow-hidden"
        >
            <div className="p-6 space-y-6">
                <div className="grid grid-cols-2 gap-4">
                    {NAV_LINKS.map(link => (
                        <a
                            key={link.name}
                            href={link.href}
                            onClick={() => setIsMobileMenuOpen(false)}
                            className="block text-[10px] font-black uppercase tracking-[0.2em] text-white/80
hover:text-primary transition-all p-4 bg-white/5 rounded-2xl border border-white/5 hover:border-
primary/20"
                        >
                            {link.name}
                        </a>
                    ))}
                </div>
                <div className="pt-4 space-y-3">
                    <button
                        onClick={() => { onProviderClick(); setIsMobileMenuOpen(false); }}
                        className="w-full py-4 bg-primary/10 border border-primary/20 rounded-2xl text-
[10px] font-bold uppercase tracking-widest text-primary flex items-center justify-center gap-3"
                    >
                        <Handshake className="w-4 h-4" /> Join as Provider
                    </button>
                    <button
                        onClick={() => { isLoggedIn ? onProfileClick() : onAuthClick();
setIsMobileMenuOpen(false); }}
                        className="w-full py-5 bg-primary text-surface rounded-2xl text-[10px] font-black
uppercase tracking-widest flex items-center justify-center gap-3"
                    >
                        <User className="w-4 h-4" /> {isLoggedIn ? 'Logged In' : 'Login'}
                    </button>
                    {isLoggedIn && (
                        <button
                            onClick={() => { onLogout(); setIsMobileMenuOpen(false); }}
                            className="w-full py-4 bg-red-500/10 border border-red-500/20 rounded-2xl text-
[10px] font-bold uppercase tracking-widest text-red-500 flex items-center justify-center gap-3"
                        >
                            <LogOut className="w-4 h-4" /> Sign Out
                        </button>
                    )}
                </div>
            </div>
        </motion.div>
    )}

```

```

    })
  </div>
</div>
</motion.div>
  })
</AnimatePresence>
</nav>
);
};

const Footer = ({ onLegalClick }: any) => {
  return (
    <footer className="py-24 border-t border-white/5 font-sans">
      <div className="max-w-7xl mx-auto px-6 grid md:grid-cols-2 lg:grid-cols-4 gap-16 mb-20">
        <div className="col-span-2">
          <Logo className="mb-8" />
          <p className="text-white/50 text-sm font-light leading-loose max-w-sm">
            The definitive destination for elite concierge healthcare and domestic management.
            Discretion, quality, and commitment.
          </p>
        </div>

        <div>
          <h5 className="text-[10px] uppercase font-bold tracking-[0.3em] text-primary
mb-8">Governance</h5>
          <ul className="space-y-4 text-xs font-bold text-white/60 uppercase tracking-widest">
            <li><button onClick={onLegalClick} className="hover:text-white transition-colors">Legal
Terms</button></li>
            <li><button onClick={onLegalClick} className="hover:text-white transition-colors">Privacy
Policy</button></li>
            <li><a href="#" className="hover:text-white transition-colors">User Agreement</a></li>
            <li><a href="#" className="hover:text-white transition-colors">Provider Vetting</a></li>
          </ul>
        </div>
      </div>

      <div className="max-w-7xl mx-auto px-6 pt-12 border-t border-white/5 flex flex-col sm:flex-row
justify-between items-center gap-6">
        <div className="flex items-center gap-4 text-[10px] font-bold uppercase tracking-widest text-
white/40">
          <span>© 2025 CAREVIA™</span>
          <span className="w-1 h-1 bg-white/20 rounded-full" />
          <span>Berlin // New York // Dubai</span>
        </div>
        <div className="flex gap-8 text-[10px] font-bold uppercase tracking-widest text-white/40">
          <a href="#" className="hover:text-white">Refunds</a>
          <a href="#" className="hover:text-white">SOS Feedback</a>
          <a href="#" className="hover:text-white">Investor Relations</a>
        </div>
      </div>
    </footer>
  );
};

```

File: src/LanguageSelector.tsx

```
import React, { useEffect, useState } from 'react';
import { Globe } from 'lucide-react';

export const LanguageSelector = ({ className = "" }: { className?: string }) => {
  const [lang, setLang] = useState('en');

  useEffect(() => {
    // Try to get current lang from google translate cookie
    const match = document.cookie.match(/googtrans=\//en\/(en|hi)/);
    if (match && match[1]) {
      setLang(match[1]);
    }
  }, []);

  const handleLanguageChange = (e: React.ChangeEvent<HTMLSelectElement>) => {
    const selectedLang = e.target.value;
    setLang(selectedLang);

    // Set google translate cookie manually just in case
    document.cookie = `googtrans=/en/${selectedLang}; path=/`;
    document.cookie = `googtrans=/en/${selectedLang}; path=/; domain=${window.location.hostname};`;

    // Reload the page to apply the google translate cookie immediately to the whole site
    window.location.reload();
  };

  useEffect(() => {
    // Hide the default Google Translate element and top banner
    const style = document.createElement('style');
    style.innerHTML = `
      #google_translate_element { display: none !important; }
      .goog-te-banner-frame { display: none !important; }
      body { top: 0 !important; }
      .skiptranslate { display: none !important; }
    `;
    document.head.appendChild(style);
  }, []);

  return (
    <div className={`flex items-center gap-2 bg-white/5 border border-white/10 rounded-xl px-3 py-2
text-white hover:bg-white/10 transition-colors ${className}`}>
      <Globe className="w-4 h-4 text-primary" />
      <select
        value={lang}
        onChange={handleLanguageChange}
        className="bg-transparent text-[10px] font-bold uppercase tracking-widest outline-none cursor-
pointer appearance-none text-white [>option]:bg-neutral-900"
      >
        <option value="en">English</option>
        <option value="hi">हिन्दी (Hindi)</option>
      </select>
    </div>
  );
};
```

File: src/hindiCorrector.ts

```
export function applyHindiCorrections() {
  const isHindi = document.cookie.includes('googtrans=/en/hi');
  if (!isHindi) return;

  const corrections: Record<string, string> = {
    'Domestic Help': '%0"y2"' 8">'ù ',
    'Child Care': 'É "ù "₁ 'Y@ 'iG'i-"é2',
    'Elderly Care': 'ÉA'ÉA" M'yK•" " &"y 'ù>"rÀ
    'Pet Care': '@>")$"" "é("Y0"₁ 'Y@ 'iG'i-"é2',
    'Helpers': "%9"é/'RrÀ
    'Elite': '•™$ "ù "97"ù ',
    'Compassion': 'Y0" #"ârÀ
    'Get Started': '"iA" B 'Y0"y ',
    'Our Packages': '"™."é0"r *"% "y ',
    'Priority Medical Dispatch': '@M" >'Y."ù 'I> '@?'Y?'IM"> '©M" G"y#',
    'Active Partners': '"% "ù0"ù/ 'ù>'y@'i>" p
  };

  const walkAndReplace = (node: Node) => {
    if (node.nodeType === Node.TEXT_NODE) {
      let text = node.nodeValue;
      if (text && text.trim() !== '') {
        let updated = text;
        for (const [en, hi] of Object.entries(corrections)) {
          // We use simple string replacement to avoid regex issues
          if (updated.includes(en)) {
            updated = updated.split(en).join(hi);
          }
        }
        if (updated !== text) {
          node.nodeValue = updated;
        }
      }
    } else if (node.nodeType === Node.ELEMENT_NODE) {
      const el = node as HTMLElement;
      if (el.tagName !== 'SCRIPT' && el.tagName !== 'STYLE') {
        node.childNodes.forEach(walkAndReplace);
      }
    }
  };

  // Run the corrector periodically to catch dynamically loaded content or Google Translate updates
  setInterval(() => {
    walkAndReplace(document.body);
  }, 1000);
}
```

File: src/index.css

```
@import url('https://fonts.googleapis.com/css2?family=Playfair+Display:ital,wght@0,400;0,600;0,700;1,400&family=Inter:wght@300;400;500;600&display=swap');
@import "tailwindcss";

@theme {
  --font-sans: "Inter", ui-sans-serif, system-ui, sans-serif;
  --font-serif: "Playfair Display", ui-serif, Georgia, serif;

  /* Swap white and black for a light premium theme. 'white' becomes dark text/borders, 'black'
  becomes light backgrounds. */
  --color-white: #1A211E;
  --color-black: #FDFCF8;

  --color-surface: #F4F1EA; /* Soft Linen */
  --color-card: #FFFFFF; /* Pure White for contrast */
  --color-primary: #556B5C; /* Muted Sage Green */
  --color-primary-container: #C8D6CD; /* Lighter Sage */
  --color-secondary: #A67D65; /* Dusty Terracotta */
  --color-muted: #6B6966; /* Warm Gray */

  --color-glass: rgba(255, 255, 255, 0.85);
  --color-border-glass: rgba(117, 143, 125, 0.15);
  --color-border-secondary: #E6E3DB;

  /* Pastel / Dusty tones for readability on light background */
  --color-red-400: #C27B7B;
  --color-red-500: #AD6363;
  --color-red-600: #964F4F;
  --color-red-700: #7A3D3D;

  --color-emerald-400: #73A387;
  --color-emerald-500: #5B8A6E;
  --color-emerald-600: #49735A;

  --color-green-500: #5B8A6E;

  --color-amber-500: #B88D4D;
  --color-amber-600: #9E753B;

  --color-blue-400: #7A9BBB;
  --color-blue-500: #5E80A3;

  --color-purple-500: #8A6E9E;

  --color-neutral-900: #F4F1EA;
  --color-neutral-200: #E3DFD6;
}

@layer base {
  body {
    @apply bg-surface text-white font-sans antialiased selection:bg-primary selection:text-black;
    background-image:
      radial-gradient(circle at 0% 0%, rgba(117, 143, 125, 0.3) 0%, transparent 60%),
      radial-gradient(circle at 100% 100%, rgba(199, 159, 134, 0.3) 0%, transparent 60%),
      radial-gradient(circle at 50% 50%, rgba(122, 155, 189, 0.15) 0%, transparent 60%);
    min-height: 100vh;
  }

  h1, h2, h3, h4, h5, h6 {
    @apply font-serif font-semibold tracking-normal text-[#1F2624];
  }
}

@layer components {
  .glass {
    @apply bg-card/80 backdrop-blur-[16px] border border-border-glass shadow-
    [0_8px_32px_rgba(117,143,125,0.06)];
  }
}
```

```

.glass-card {
  @apply bg-card/75 backdrop-blur-[16px] border border-border-glass p-6 transition-all duration-300
  rounded-3xl shadow-[0_8px_32px_rgba(117,143,125,0.08)] hover:shadow-
  [0_12px_48px_rgba(117,143,125,0.12)];
}

.btn-primary {
  @apply bg-primary text-[#FDFCF8] px-6 py-3 rounded-full font-bold transition-all
  hover:brightness-105 hover:-translate-y-0.5 active:translate-y-0 active:scale-95 text-xs uppercase
  tracking-widest shadow-[0_4px_16px_rgba(117,143,125,0.3)] hover:shadow-
  [0_8px_24px_rgba(117,143,125,0.4)];
}

.btn-outline {
  @apply border border-border-secondary text-muted px-6 py-3 rounded-full font-semibold transition-
  all hover:bg-white/5 hover:border-primary/30 hover:text-primary;
}

.btn-gold {
  @apply bg-primary text-[#FDFCF8] px-6 py-3 rounded-full font-bold transition-all
  hover:brightness-105 hover:-translate-y-0.5 active:translate-y-0 active:scale-95 shadow-
  [0_4px_16px_rgba(117,143,125,0.3)];
}

.label-bold {
  @apply text-[10px] uppercase tracking-[0.2em] font-bold text-muted;
}

.sos-button {
  @apply w-16 h-16 bg-red-500 text-black rounded-full flex items-center justify-center shadow-
  [0_0_30px_rgba(173,99,99,0.3)] animate-pulse hover:scale-110 transition-transform cursor-pointer z-
  [100];
}

.monolith-grid {
  background-image:
    linear-gradient(rgba(117, 143, 125, 0.25) 1px, transparent 1px),
    linear-gradient(90deg, rgba(117, 143, 125, 0.25) 1px, transparent 1px),
    linear-gradient(rgba(117, 143, 125, 0.08) 1px, transparent 1px),
    linear-gradient(90deg, rgba(117, 143, 125, 0.08) 1px, transparent 1px);
  background-size: 50px 50px, 50px 50px, 10px 10px, 10px 10px;
  background-position: center center;
}

.hide-scrollbar::-webkit-scrollbar {
  display: none;
}

.hide-scrollbar {
  -ms-overflow-style: none;
  scrollbar-width: none;
}

```

File: src/lib/firebase.ts

```
import { initializeApp } from 'firebase/app';
import { getAuth } from 'firebase/auth';
import { getFirestore, doc, getDocFromServer } from 'firebase/firestore';
import { getStorage } from 'firebase/storage';
import firebaseConfig from '../../../firebase-applet-config.json';

const app = initializeApp(firebaseConfig);
export const auth = getAuth(app);
// Use the firestoreDatabaseId from the config
export const db = getFirestore(app, (firebaseConfig as any).firestoreDatabaseId);
export const storage = getStorage(app);

/**
 * Validates connection to Firestore
 */
async function testConnection() {
  try {
    await getDocFromServer(doc(db, 'test', 'connection'));
    console.log("Firestore Connected successfully");
  } catch (error) {
    if (error instanceof Error && error.message.includes('the client is offline')) {
      console.error("Please check your Firebase configuration.");
    } else {
      console.warn("Initial connection test failed, but may succeed after auth:", error);
    }
  }
}

testConnection();

export interface FirestoreErrorInfo {
  error: string;
  operationType: 'create' | 'update' | 'delete' | 'list' | 'get' | 'write';
  path: string | null;
  authInfo: {
    userId: string;
    email: string;
    emailVerified: boolean;
    isAnonymous: boolean;
    providerInfo: { providerId: string; displayName: string; email: string; }[];
  }
}

export function handleFirestoreError(error: any, operationType: FirestoreErrorInfo['operationType'],
path: string | null = null) {
  if (error.code === 'permission-denied') {
    const errorInfo: FirestoreErrorInfo = {
      error: error.message,
      operationType,
      path,
      authInfo: {
        userId: auth.currentUser?.uid || 'anonymous',
        email: auth.currentUser?.email || '',
        emailVerified: auth.currentUser?.emailVerified || false,
        isAnonymous: auth.currentUser?.isAnonymous || true,
        providerInfo: auth.currentUser?.providerData.map(p => ({
          providerId: p.providerId,
          displayName: p.displayName || '',
          email: p.email || ''
        }))) || []
      }
    };
    throw new Error(JSON.stringify(errorInfo));
  }
  throw error;
}
```

File: src/lib/workspace.ts

```
declare const google: any;

const CLIENT_ID = import.meta.env.VITE_CLIENT_ID;

const SCOPES = [
  'openid',
  'email',
  'profile',
  'https://www.googleapis.com/auth/spreadsheets',
  'https://www.googleapis.com/auth/drive.file',
].join(' ');

let tokenResponse: any = null;

export const getAccessToken = (): Promise<string> => {
  return new Promise((resolve, reject) => {
    if (!CLIENT_ID) {
      reject(new Error('VITE_CLIENT_ID is not configured in .env'));
      return;
    }

    if (tokenResponse && Date.now() < tokenResponse.expires_at) {
      resolve(tokenResponse.access_token);
      return;
    }

    try {
      const client = google.accounts.oauth2.initTokenClient({
        client_id: CLIENT_ID,
        scope: SCOPES,
        callback: (response: any) => {
          if (response.access_token) {
            tokenResponse = {
              ...response,
              expires_at: Date.now() + (response.expires_in * 1000)
            };
            resolve(response.access_token);
          } else {
            reject(new Error('Failed to get access token: ' + (response.error || 'Unknown error')));
          }
        },
      });
      client.requestAccessToken();
    } catch (error) {
      reject(error);
    }
  });
};

/**
 * Uploads a file to Google Drive
 */
export const uploadToDrive = async (fileName: string, content: string, mimeType: string = 'text/plain') => {
  const token = await getAccessToken();

  const metadata = {
    name: fileName,
    mimeType: mimeType,
  };

  const form = new FormData();
  form.append('metadata', new Blob([JSON.stringify(metadata)], { type: 'application/json' }));
  form.append('file', new Blob([content], { type: mimeType }));

  const response = await fetch('https://www.googleapis.com/upload/drive/v3/files?uploadType=multipart', {
    method: 'POST',
  });
}
```

```

    headers: {
      Authorization: `Bearer ${token}`,
    },
    body: form,
  });

  if (!response.ok) {
    throw new Error('Failed to upload to Drive: ' + await response.text());
  }

  return response.json();
};

/**
 * Exports data to a new Google Sheet
 */
export const exportToSheets = async (title: string, headers: string[], rows: any[][]) => {
  const token = await getAccessToken();

  // 1. Create a new Spreadsheet
  const createRes = await fetch('https://sheets.googleapis.com/v4/spreadsheets', {
    method: 'POST',
    headers: {
      Authorization: `Bearer ${token}`,
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({
      properties: { title }
    }),
  });

  if (!createRes.ok) {
    throw new Error('Failed to create spreadsheet: ' + await createRes.text());
  }

  const spreadsheet = await createRes.json();
  const spreadsheetId = spreadsheet.spreadsheetId;

  // 2. Add data
  const values = [headers, ...rows];
  const updateRes = await fetch(`https://sheets.googleapis.com/v4/spreadsheets/${spreadsheetId}/values/Sheet1!A1:append?valueInputOption=RAW`, {
    method: 'POST',
    headers: {
      Authorization: `Bearer ${token}`,
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({
      values
    }),
  });

  if (!updateRes.ok) {
    throw new Error('Failed to update spreadsheet: ' + await updateRes.text());
  }

  return spreadsheet;
};

```

File: src/main.tsx

```
import {StrictMode} from 'react';
import {createRoot} from 'react-dom/client';
import App from './App.tsx';
import './index.css';

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

File: src/vite-env.d.ts

```
/// <reference types="vite/client" />
```

File: tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "experimentalDecorators": true,
    "useDefineForClassFields": false,
    "module": "ESNext",
    "lib": [
      "ES2022",
      "DOM",
      "DOM.Iterable"
    ],
    "skipLibCheck": true,
    "moduleResolution": "bundler",
    "isolatedModules": true,
    "moduleDetection": "force",
    "allowJs": true,
    "jsx": "react-jsx",
    "paths": {
      "@/*": [
        "./*"
      ]
    },
    "allowImportingTsExtensions": true,
    "noEmit": true
  }
}
```

File: vite.config.ts

```
import tailwindcss from '@tailwindcss/vite';
import react from '@vitejs/plugin-react';
import path from 'path';
import {defineConfig, loadEnv} from 'vite';

export default defineConfig(({mode}) => {
  const env = loadEnv(mode, '.', '');
  return {
    plugins: [react(), tailwindcss()],
    define: {
      'process.env.GEMINI_API_KEY': JSON.stringify(env.GEMINI_API_KEY),
    },
    resolve: {
      alias: {
        '@': path.resolve(__dirname, '.'),
      },
    },
    server: {
      // HMR is disabled in AI Studio via DISABLE_HMR env var.
      // Do not modify "file watching is disabled to prevent flickering during agent edits.
      hmr: process.env.DISABLE_HMR !== 'true',
    },
  };
});
```